

UNIVERSIDAD DE LOS ANDES
MÉRIDA, VENEZUELA

Python



Autor: Linis B. Guerrero P





UNIVERSIDAD
DE LOS ANDES

UNIVERSIDAD DE LOS ANDES
MÉRIDA
VENEZUELA

Un curso intensivo de Python 3

Una introducción rápida a la programación con Python 3

Una Guía rápida a la programación con Python 3

Autor: Linis B. Guerrero P.

Traducido: Prof. Linis B. Guerrero P.

Mérida, Venezuela

Septiembre, 2023



UNIVERSIDAD
DE LOS ANDES

UNIVERSIDAD DE LOS ANDES
MÉRIDA
VENEZUELA

Un curso intensivo de Python 3

Una introducción rápida a la programación con Python 3

Una Guía rápida a la programación con Python 3

Trabajo presentado por el Profesor Linis B. Guerrero P.

Autor: Linis B. Guerrero P.

Traducido: Prof. Linis B. Guerrero P.

MÉRIDA, VENEZUELA

Septiembre, 2023

Un curso intensivo de Python 3

Una introducción rápida a la
programación con Python

Una Guía rápida a la programación con Python 3

Agradecimientos

Borrador

Índice general

Índice general	V
Índice de figuras	VII
Índice de tablas Cuadros	VIII
Capítulos	Página
Agradecimientos	IX
Prefacio	IX
A Quién va dirigido este libro	X
¿Qué esperas aprender en este libro?	X
Navegando en este libro	XII
¿Por qué Python?	XV
Recursos en línea	XVI
Ejemplos de código	XVI
Convenciones utilizadas en este libro	XVII
Libros online Ula	XVIII
Como contactar con nosotros	XVIII
Agradecimientos	XIX
Prefacio	XIX
1. Introducción entre-png-jpg - y - gif	XX
1.0.1. CREAR IMÁGENES CON UN CANAL ALFA	3

2. Introducción a la Librería Pygame	6
2.1. Pygame	7
2.1.1. Instalación de la Librería PyGame	8
2.1.2. Módulos de la Librería PyGame	9
2.2. Mi primer Juego	10
2.3. Actualizando Mi primer Juego	17
2.3.1. Coordenadas de píxeles	20
2.3.2. Objetos de superficie y la ventana	21
2.3.3. Colores	21
2.3.4. Colores transparentes	22
2.4. Módulo pygame.Rect y objetos Rect	24
2.5. Módulo pygame.draw y funciones primitivas para dibujo	25
2.6. Animación	29
2.6.1. Nuestra primera animación	30
2.6.2. Velocidad de Fotogramas	32
2.6.3. Dibujar y cargar imágenes con pygame.image.load() y blit()	33
2.6.4. Crear texto con el módulo pygame.font	34
2.6.5. Reproducir sonidos	37
2.7. Módulo pygame.event y Cola de eventos	39
2.7.1. Recuperar eventos	40
2.7.2. Eventos de movimiento del mouse	43
2.7.3. Tipos de eventos del mouse	43
2.7.4. Manejo de eventos de teclado	44
2.7.5. Eventos personalizados con pygame.event.Event	47
2.8. Opciones para pantalla	48
2.8.1. Pantalla completa	49
2.8.2. Redimensionar ventanas Pygame	51
2.8.3. Ventanas sin bordes	53
2.8.4. Parámetros adicionales para visualización	54
2.9. Resumen	57
3. Movimientos, vectores y colisiones	58
3.1. Velocidad de fotogramas	59
3.2. Movimiento en línea recta, vertical y horizontal	60
3.2.1. El tiempo de pygame	62
3.3. Movimiento en cualquier dirección, y colisión sencilla	65
3.4. Explorando vectores	69

3.4.1. Vectores y el movimiento	72
3.5. Resumen	77
4. Interacción del Jugador con el juego	78
4.1. Comprender el teclado para controlar el juego	80
4.1.1. Detección de pulsaciones de teclas	80
4.2. Movimiento direccional con teclas	83
4.2.1. Movimiento rotacional con teclas	86
4.3. Implementación del control con el mouse	88
4.3.1. Movimiento rotacional con el mouse	89
4.4. Implementación del control por palanca de mando	93
4.5. Resumen	94
SEGUNDA PARTE: Mis Proyectos	95
5. Invasión Alienígena	98
5.1. Una Clase base para el juego	98
5.1.1. Agregamos la velocidad de fotogramas	101
5.1.2. Configurar el color de fondo	102
5.2. Crear una clase de configuración	103
5.3. Agregar la imagen de la nave	104
5.3.1. Una clase Nave()	105
5.4. Refactorización	108
5.4.1. El método <code>_chequear_eventos()</code>	109
5.4.2. El método <code>_actualizar_pantalla()</code>	109
5.5. Ejercicios	110
5.6. Pilotando la nave	110
5.7. Refactorización <code>_chequear_eventos()</code>	118
5.7.1. Ejecutar el juego en modo de pantalla completa	119
5.8. Resumiendo el trabajo	120
5.8.1. <code>invasión_alien.py</code>	121
5.8.2. <code>configurar.py</code>	121
5.8.3. <code>nave.py</code>	121
5.9. INTÉNTALO TÚ MISMO	122
5.10. Disparar balas	122
5.10.1. Disparar balas	125
5.10.2. Limitar el número de balas	128
5.10.3. Organizando la clase <code>InvasionAlien()</code>	129
5.11. INTÉNTALO TÚ MISMO	130

5.12. Resumen	130
6. Alienígenas en el cielo	131
6.1. Revisar el proyecto	132
6.2. El primer alienígena	132
6.2.1. La clase alienígena	133
6.2.2. Una instancia de la clase Alien	134
6.2.3. Construyendo la flota alienígena	135
6.3. INTÉNTALO TÚ MISMO	140
6.4. La flota se mueve	140
6.5. INTÉNTALO TÚ MISMO	144
6.6. Detección de colisiones de alienígenas con las balas	145
6.7. Hacer balas más poderosas para realizar pruebas	146
6.7.1. Reposición de la flota	147
6.7.2. Refactorización <code>_actualizar_balas()</code>	148
6.8. INTÉNTALO TÚ MISMO	149
6.9. Terminando el juego	149
6.10. Detección de colisiones	150
6.11. Identificar cuándo deben ejecutarse las partes del juego	155
6.12. INTÉNTALO TÚ MISMO	155
6.13. Resumen	156
7. PUNTUACIÓN	157
7.1. Agregar el botón Play	157
7.2. INTÉNTALO TÚ MISMO	163
7.3. Subir de nivel	164
7.4. INTÉNTALO TÚ MISMO	166
7.5. Puntuación	167
7.5.1. Visualización del número de naves	178
7.6. INTÉNTALO TÚ MISMO	181
7.7. Resumen	181
8. Empaquetando el juego, y creando un archivo EXE	183
8.0.1. Creando paquetes en Windows	183
8.0.2. Usando <code>cx_Freeze</code>	184

Índice de figuras

2.1.	Hola, Mundo en Pygame	18
2.2.	Ventana dibujos del módulo <code>pygame.draw</code>	27
2.3.	La ardilla mueve hacia el gato	30
2.4.	Simulando movimiento del perro	32
2.5.	Ventana agregando texto con módulo <code>font.SysFont()</code>	36
2.6.	Eventos del mouse	42
2.7.	Ventana sin borde y sin títulos	54
3.1.	Juego Pac-Man	58
3.2.	Vector $\overrightarrow{AB}$	70
4.1.	Vectores diagonales combinando vectores simples.	85
4.2.	Al girar una superficie cambia su tamaño.	88
5.1.	La nave para Invasión Alienígena.	105
5.2.	La nave en la parte inferior central de la pantalla.	108
5.3.	La nave después de disparar una serie de balas.	127
6.1.	El extraterrestre que usaremos.	133
6.2.	Aparece el primer alienígena en la pantalla.	135
6.3.	Aparece la primera fila de extraterrestres en pantalla.	137
6.4.	Aparece la flota completa.	140
6.5.	La flota ha sido parcialmente derribada.	146
6.6.	Balas más poderosas.	147
7.1.	Aparecer el botón Play.	160
7.2.	La puntuación en la esquina superior derecha.	169
7.3.	Puntuación redondeada con separadores de coma.	173
7.4.	En la parte superior central de la	176
7.5.	El nivel actual aparece justo	178

7.6. Las naves disponibles aparecen	180
---	-----

Borrador

Índice de cuadros

Borrador

Prefacio

En proceso

El concepto de prefacio de un libro es una introducción escrita por el autor o por otra persona, que se coloca al principio del libro y tiene como objetivo proporcionar información sobre la obra y su contexto.

El prefacio puede incluir detalles sobre la motivación del autor para escribir el libro, cómo surgió la idea, el proceso de creación, la investigación o la influencia de otras obras de arte o personas. También puede incluir una descripción del contenido del libro y cómo se relaciona con otros temas o trabajos similares. En resumen, el prefacio es una introducción al libro y su contexto que ayuda al lector a comprender mejor la obra.

Un prefacio es, en literatura, un texto de introducción y de presentación, ubicado al inicio de un libro. En el prefacio se da a conocer el plan y los puntos de vista utilizados durante la elaboración del escrito, a la par que allí también corresponde prevenir sobre posibles objeciones o reservas, responder a críticas ya formuladas a las ediciones anteriores o a los avances de la obra, y finalmente también dar ideas sobre el mensaje que el autor quiere transmitir con este documento; por ejemplo, y si el escrito planteara inquietudes sociales, el mensaje principal o más trascendente podría estar vinculado con la organización social, la pobreza, la desigual distribución de recursos, la prostitución, la violencia, el consumo de drogas y el narcotráfico, etc, o con varios de los temas citados.

El prefacio generalmente es corto cuando el mismo se orienta y se centra a ser una advertencia, y usualmente es largo cuando también incluye prolegómenos, motivaciones profundas o casuales, antecedentes, etc.

A Quién va dirigido este libro

El objetivo de este libro es enseñar de manera didáctica las herramientas fundamentales para programar en Python. Está dirigido a personas de cualquier edad que deseen aprender a programar, y que tal vez nunca hayan programado en Python o nunca hayan programado en absoluto. Se hacen comentarios breves respecto al vocabulario común usado con los lenguajes de alto nivel en informática, para familiarizar al lector que se inicia en Python. Dando las pautas para instalar Python y un entorno para editar código. Se exponen los conceptos e instrucciones básicos de programación con ejemplos de código, para ir formando una base sólida en los conceptos de programación, habilidad y seguridad para su desarrollo como programador. Se trabajan ejemplos que sirvan de referencia para crear programas¹ como juegos, visualizaciones de datos, y aplicaciones web, mientras se desarrolla una base en programación que te servirá bien por el resto de tu vida.

El contenido del libro es ideal para profesores de todos los niveles que quieren ofrecer a sus estudiantes una introducción a la programación basada en proyectos. Si estás tomando una clase universitaria y quieres una introducción más amigable a Python que el texto que te han asignado, este libro puede hacer que tu clase sea más fácil. Si estás buscando cambiar de carrera, este curso de Python puede ayudarte a hacer la transición hacia una carrera profesional más satisfactoria.

¿Qué esperas aprender en este libro?

El propósito de este libro es darle las herramientas para que se convierta en un buen programador de Python y en general en un buen programador; además, aprenderás y adoptarás buenos hábitos para desarrollar Python.

En la Parte I de este libro, aprenderá conceptos básicos de programación necesarios para escribir programas en Python. Estos conceptos son casi los mismos que aprenderías al comenzar a estudiar cualquier lenguaje de programación.

Aquí también aprenderá sobre tipos de datos y las formas en que puede

¹Un programa es una serie de instrucciones ordenadas, codificadas en un lenguaje de programación que expresa uno o varios algoritmos y que puede ser ejecutado en una computadora.

almacenarlos en sus programas. Crearás colecciones de datos, como listas y diccionarios, y trabajará en esas colecciones de manera eficiente. Aprenderás usar bucles `while` y declaraciones `if` para probar ciertas condiciones, por lo que puede ejecutar bloques específicos de código mientras esas condiciones sean verdaderas y se ejecuten otras en caso que no sean verdaderas, una técnica que le ayuda a automatizar muchos procesos.

Aprenderá a aceptar aportaciones de los usuarios para que sus programas sean interactivos y mantener sus programas funcionando todo el tiempo que el usuario lo requiera, lo hará explorar cómo escribir funciones que hagan que partes de su programa sean reutilizables, por lo que solo tienes que escribir bloques de código que realicen ciertas acciones una vez, mientras usas ese código tantas veces como necesites. Luego ampliarás este concepto a comportamientos más complicados con clases, haciendo que programas bastante simples respondan a una variedad de situaciones. Aprenderá a escribir programas que manejen correctamente los errores comunes.

Después de trabajar en cada uno de estos conceptos básicos, escribirá una serie de programas cada vez más complejos utilizando lo que ha aprendido. Finalmente, dará tu primer paso hacia la programación intermedia al aprender a escribir pruebas para tu código, de modo que puedas desarrollar más sus programas sin preocuparte por introducir errores. Toda la información de la Parte I lo preparará para asumir proyectos más grandes y complejos.

En la Parte II, en proceso... aplicará lo aprendido en la Parte I a tres proyectos. Puede realizar cualquiera o todos estos proyectos, en el orden que mejor le convenga. En los capítulos 12 a 14 está el primer proyecto, un juego de disparos al estilo Space Invaders llamado Alien Invasion, que incluye varios niveles, cada vez más difíciles. Una vez que hayas completado este proyecto, debes estar en camino de poder desarrollar tus propios juegos en 2D. Incluso si no aspiras a convertirte en programador de juegos, trabajar en este proyecto es una forma divertida de poner a prueba parte de lo que aprendió en la Parte I.

En los capítulos del 15 al 17 se expone el segundo proyecto, en esta caso se trata de técnicas para visualización de datos. Los científicos de datos utilizan una variedad de técnicas de visualización para encontrar patrones y dar sentido a la gran cantidad de información disponible. Trabaja con conjuntos de datos conjuntos de datos que descargue de fuentes en línea y

conjuntos de datos que sus programas descarguen automáticamente. Una vez que haya completado este proyecto, podrá escribir programas que analicen grandes conjuntos de datos y creen representaciones visuales de muchos tipos diferentes de información.

El tercer proyecto se presenta en los Capítulos del 18 al 20, creará una pequeña aplicación web llamada Registro de aprendizaje. Este proyecto le permite llevar un diario organizado de la información que ha aprendido sobre un tema específico. Podrá mantener registros separados para diferentes temas y permitir que otros creen una cuenta y comiencen sus propios diarios. También aprenderá cómo implementar su proyecto para que cualquiera pueda acceder a él en línea, desde cualquier parte del mundo.

Navegando en este libro

Este proyecto esta ideado en dos partes, la primera parte enseña los conceptos básicos que necesitará para escribir programas en Python. Muchos de estos conceptos son comunes a todos los lenguajes de programación, por lo que te serán útiles a lo largo de tu vida como programador. Si ya tienes configurado Python y tienes un editor o IDE que te gusta, puedes comenzar en el capítulo ?? sin problemas.

Este libro está organizado de la siguiente manera:

- En el Capítulo ?? se hace una introducción breve a Python, algunos los términos y conceptos comunes en informática, específicamente los que se asocian con Python, y una descripción de algunos editores e IDEs compatibles con Python.
- En el Capítulo ?? se inicia enseñando la instalación y configuración tanto para Python, como para algunos editores e IDEs, y se describen las características más importantes. Se ejecutará la primera su experiencia con un fragmento de código Python y su primer programa Python, con el interprete de Python tanto en la terminal de Windows (cmd), como en el IDLE de Python o VSCode.
- En el Capítulo ?? se estudian los tipos de datos, y se asigna información a variables, es una buena lectura para empezar. Y se estudia lo referentes a cadenas o string.

- En el Capítulo ?? presentan las Tuplas, Listas, Diccionarios, conjuntos, y tipo Colecciones. Las listas pueden almacenar tanta información como desee en un solo lugar, lo que le permitirá trabajar con esos datos de manera eficiente. Podrás trabajar con cientos, miles e incluso millones de valores en tan solo unas pocas líneas de código. En la Sección ?? se empieza a utilizar los diccionarios de Python, que le permiten establecer conexiones entre diferentes estructuras de información. Al igual que las listas, los diccionarios pueden contener tanta información como necesite almacenar.
- En el Capítulo ?? aprenderá cómo aceptar entradas de los usuarios para que sus programas sean interactivos. También aprenderá sobre los bucles `while`, que ejecutan bloques de código repetidamente siempre que ciertas condiciones sigan siendo verdaderas.
- En el Capítulo ?? se introducen las estructuras de control de flujo: `if-elif-else`, para escribir código que responda de una manera si ciertas condiciones son verdaderas y responda de otra manera si esas condiciones no son verdaderas.
- En el Capítulo ?? escribirás funciones, que son bloques de código con nombre que realizan una tarea específica y se pueden ejecutar cuando las necesites.
- En el Capítulo ?? se inicia el trabajo con Módulos, Paquetes y programas,...
- En el Capítulo ?? se presentan las clases, que le permiten modelar objetos del mundo real. Escribirás código que represente perros, gatos, personas, automóviles, cohetes y más.
- En el Capítulo ?? se inicia el trabajo archivos, y se enseña a manejar errores para que sus programas no colapsen inesperadamente. Almacenará datos antes de que se cierre su programa y los volverá a leer cuando el programa se ejecute nuevamente. Aprenderá sobre las excepciones de Python, que le permiten anticipar errores y hacer que sus programas los manejen correctamente.
- En el Capítulo ?? aprenderá a escribir pruebas eficientes para comprobar que sus programas funcionan de la forma prevista. Como resultado, podrá ampliar sus programas sin preocuparse por introducir

nuevos errores. Probar tu código es una de las primeras habilidades que te ayudarán a pasar de programador principiante a programador intermedio.

- En proceso... El proyecto Invasión alienígena en los Capítulo ?? y ?? incluye una configuración para controlar la velocidad de fotogramas, lo que hace que el juego se ejecute de manera más consistente en diferentes sistemas operativos. Se utiliza un enfoque más simple para construir la flota de extraterrestres y también se ha limpiado la organización general del proyecto.
- En proceso... Los proyectos de visualización de datos se desarrolla en los Capítulo ?? a ?? utilizan las funciones más recientes de Matplotlib y Plotly. Las visualizaciones de Matplotlib presentan configuraciones de estilo actualizadas. El proyecto de caminata aleatoria tiene una pequeña mejora que aumenta la precisión de los gráficos, lo que significa que verá surgir una variedad más amplia de patrones cada vez que genere una nueva caminata. Todos los proyectos que presentan Plotly ahora utilizan el módulo Plotly Express, que le permite generar sus visualizaciones iniciales con solo unas pocas líneas de código. Puede explorar fácilmente una variedad de visualizaciones antes de comprometerse con un tipo de trama y luego concentrarse en refinar los elementos individuales de esa trama.
- En proceso... El proyecto Learning Log se presenta en los Capítulo ?? a ?? se creó utilizando la última versión de Django y se diseñó utilizando la última versión de Bootstrap. Se ha cambiado el nombre de algunas partes del proyecto para que sea más fácil seguir la organización general del proyecto. El proyecto ahora está implementado en Platform.sh, un moderno servicio de alojamiento para proyectos de Django. El proceso de implementación está controlado por archivos de configuración YAML, que le brindan un gran control sobre cómo se implementa su proyecto. Este enfoque es consistente con la forma en que los programadores profesionales implementan proyectos modernos de Django.
- En proceso...El Apéndice xx dirige a los lectores a varios de los recursos en línea más populares para obtener ayuda.

Gracias por leer el curso intensivo de Python! Si tiene algún comentario

o pregunta, no dude en ponerse en contacto; Soy Julian en Celcie.Ula.ve jajajaja En proceso...

¿Por qué Python?

Python es un lenguaje increíblemente eficiente: sus programas harán más en menos líneas de código de lo que requerirían muchos otros lenguajes. Es un lenguaje de programación interpretado cuya filosofía hace hincapié en una sintaxis muy limpia y un código legible, que le te ayudará a escribir código "limpio". Su código será más fácil de leer, más fácil de depurar y más fácil de ampliar y desarrollar, en comparación con otros lenguajes.

Python es una muy buena alternativa para empezar a programar puesto que es un lenguaje muy sencillo, fácil y con una curva de aprendizaje buena. La sintaxis es fácil de entender puesto que es cercana al lenguaje natural.

La gente usa Python para muchos propósitos: crear juegos, crear aplicaciones web, resolver problemas comerciales y desarrollar herramientas internas en todo tipo de empresas interesantes, en el campo del aprendizaje automático y la inteligencia artificial. Actualmente se utiliza en campos científicos, para investigación académica y trabajos aplicados.

Una de las razones más importantes por las que recomiendo el uso de Python es por la comunidad Python, que incluye un grupo de personas increíblemente diversa y acogedora. La comunidad es esencial para los programadores porque la programación no es una actividad solitaria. La mayoría de nosotros, incluso los programadores más experimentados, necesitamos pedir consejo a otras personas que ya han resuelto problemas similares. Tener una comunidad bien conectada y de apoyo es fundamental para ayudarlo a resolver problemas, y la comunidad de Python apoya plenamente a las personas que están aprendiendo Python como su primer lenguaje de programación o que llegan a Python con experiencia en otros lenguajes. Python es un gran lenguaje para aprender, ¡así que comencemos!

Recursos en línea

Al estudiar este libro, puede visitar el sitio web para obtener documentación detallada sobre clases, funciones, y muchos ejemplos. También hay un curso con vídeos creado por

En proceso.....

No Starch Press tiene más información sobre este libro disponible en línea en También mantengo un amplio conjunto de recursos complementarios en

Estos recursos incluyen lo siguiente:

Instrucciones de configuración Las instrucciones de configuración en línea son idénticas a las del libro, pero incluyen enlaces activos en los que puede hacer clic para seguir los diferentes pasos. Si tiene algún problema de configuración, consulte este recurso. Actualizaciones Python, como todos los lenguajes, está en constante evolución. Mantengo un conjunto completo de actualizaciones, por lo que si algo no funciona, consulte aquí para ver si las instrucciones han cambiado.

Soluciones a los ejercicios Debe dedicar mucho tiempo a realizar los ejercicios de las secciones “Pruébelo usted mismo”. Sin embargo, si estás estancado y no puedes progresar, las soluciones para la mayoría de los ejercicios están en línea. Hojas de referencia También está disponible en línea un conjunto completo de hojas de referencia descargables para una referencia rápida a los conceptos principales.

Ejemplos de código

En proceso...

El material suplementario (ejemplos de código, cuadernos IPython, etc.) está disponible para su descarga en:

https://github.com/amueller/introduction_to_ml_with_python.

Este libro se desarrolló con la finalidad de ayudarle en su trabajo. En general, el código en los ejemplo del libro lo puede usar en sus programas y documentación. No necesita contactar con nosotros para obtener permiso a menos que esté reproduciendo una parte significativa del código. Por ejemplo, escribir un programa que usa varios trozos de código de este libro no requiere

permiso. Vender o distribuir un

Si cree que el uso que hace de los ejemplos de código queda fuera del uso legítimo o del permiso otorgado arriba, no dude en contactarnos en permissions@oreilly.com.

Convenciones utilizadas en este libro

En proceso..

En este libro se utilizan las siguientes convenciones tipográficas:

Italic

Indica nuevos términos, direcciones URL, direcciones de correo electrónico, nombres de archivo y extensiones de archivo.

Constant width

Usado para listados de programas, así como dentro de párrafos para referirse a elementos de programas tales como nombres de variables o funciones, bases de datos, tipos de datos, variables de entorno, sentencias y palabras clave. También se utiliza para comandos y nombres de módulos y paquetes.

Constant width bold

Muestra comandos u otro texto que el usuario debe escribir literalmente.

Constant width italic

Muestra texto que debe reemplazarse por valores proporcionados por el usuario o por valores determinados por el contexto.



Este elemento significa sugerencia.



Este elemento significa una nota general.



Este icono indica una advertencia o precaución.

Libros online Ula

En proceso..

Como contactar con nosotros

En proceso...

Dirija sus comentarios y preguntas sobre el libro al editor:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

Tenemos una página web para este libro, con la fe de erratas, ejemplos y cualquier información adicional. Puede acceder a esta página en:

<http://bit.ly/intro-machine-learning-python>.

Para comentar o hacer preguntas técnicas sobre este libro, envíe un correo electrónico a bookquestions@oreilly.com.

Para obtener más información sobre nuestros libros, cursos, conferencias y noticias, consulte nuestro sitio web en <http://www.oreilly.com>.

Síguenos en Twitter: <http://twitter.com/oreillymedia>.

Míranos en YouTube: <http://www.youtube.com/oreillymedia>.

Agradecimientos

De Linis

Sin la ayuda y el apoyo de un gran grupo de personas, este libro nunca habría existido.

Me gustaría agradecer a los editores,

Estoy eternamente agradecido por la acogedora comunidad científica de código abierto de Python, especialmente los contribuyentes

También estoy agradecido por las discusiones con muchos de mis colegas y compañeros que me ayudaron a comprender los desafíos del aprendizaje automático y me dio ideas para la estructura un libro de texto.

En el aspecto personal, quiero agradecer a mis padres, por su continuo apoyo y aliento. También quiero agradecer a muchas personas en mi vida cuyo amor y amistad me dieron la energía y el apoyo para emprender una tarea tan desafiante.

De Julian

Me gustaría agradecer, sin cuya ayuda y orientación este proyecto ni siquiera habría existido. Gracias a Carlos Dávila, Oswaldo por leerlo en los primeros días. Gracias a la gente ULA Ciencias por su infinita paciencia. Y finalmente gracias a Dios , por su apoyo eterno e infinito.

Capítulo 1

Introducción entre-png-jpg - y - gif

GUI frente a CLI

Los programas Python que puedes escribir con las funciones integradas de Python solo tratan con texto. a través de las funciones `print()` y `input()`.

Su programa puede mostrar texto en la pantalla y permitir que el usuario escriba texto desde el teclado. Este tipo de programa tiene una interfaz de línea de comando, o CLI (que se pronuncia como la primera parte de “climb” y rima con “sky”). Estos Los programas son algo limitados porque no pueden mostrar gráficos, tener colores o usar el ratón.

Estos programas CLI solo obtienen información del teclado con la función `input()` y incluso entonces el usuario debe presionar Enter antes de que el programa pueda responder a la entrada. Esto significa en tiempo real (es decir, continuar ejecutando código sin esperar al usuario) los juegos de acción son imposibles de ejecutar. hacer. Pygame proporciona funciones para crear programas con una interfaz gráfica de usuario o GUI (pronunciado “pega-joso”). En lugar de una CLI basada en texto, los programas con una GUI basada en gráficos pueden mostrar una ventana con imágenes y colores.

¡Hurra! ¡Acabas de crear el videojuego más aburrido del mundo! Es solo una ventana en blanco con "Hola". ¡Mundo! en la parte superior de la ventana (en lo que se llama la barra de título de la ventana, que contiene el texto del título). Pero crear una ventana es el primer paso para crear juegos gráficos. Cuando haces clic en el botón X en la esquina de la ventana, el programa finalizará y la ventana desaparecerá. Llamar a la función `print()` para que aparezca texto en la ventana no funcionará porque `print()` es una

función para programas CLI. Lo mismo ocurre con `input()` para obtener la entrada del teclado, del usuario. Pygame utiliza otras funciones para entrada y salida que se explican más adelante en este capítulo. Por ahora, veamos cada línea de nuestro programa "Hola mundo" con más detalle.

Borrador

Primera Parte: FUNDAMENTOS DE PYGAME

En proceso....
Comentar el trabajo con IDLE y en Windows...

Proyectos Python, un Juego: Invasión Alienígena En el primer Capítulo de la Segunda Parte del libro, desarrollaremos un proyecto de un juego llamado Alienígena Invasión. Para desarrollar este juego se usará la librería **Pygame**, una colección módulos de Python que facilita la creación juegos, con sofisticados gráficos, animación e incluso sonido, para que puedas gestión y desarrollar su proyectos.

Este libro se dirige a todos los makers y entusiastas de la programación que deseen aprender a desarrollar videojuegos con Pygame. También interesará a los principiantes en el lenguaje Python, así como a los informáticos que quieran perfeccionar sus conocimientos sobre esta librería, que se utiliza no solo en el campo de los videojuegos, sino también en el de la simulación.

No es necesario dominar el lenguaje Python para leer este libro, porque el autor comienza presentando los conceptos básicos antes de indicar cómo dar sus primeros pasos con Pygame y detallar la estructura de un juego con Pygame. Posteriormente, a lo largo de los capítulos, se estudian los aspectos principales del desarrollo de videojuegos en dos dimensiones. El lector también va a estudiar cómo gestionar un bucle de juego, cómo dominar los aspectos gráficos con el módulo `pygame.draw`, cómo añadir sonido con el módulo `pygame.mixer`, incluso cómo gestionar el tiempo y especialmente las colisiones entre objetos gráficos gracias al concepto de `sprite`. El autor también ofrece una introducción al 3D y al concepto de motor de juego.

Finalmente, los dos últimos capítulos brindan al lector una documenta-

ción precisa de los principales módulos de Pygame que se utilizan en el libro, así como de los que se utilizan con frecuencia en el desarrollo usando Pygame.

Hay elementos adicionales para su descarga en el sitio web www.ediciones-eni.com.

¿Alguna vez has abierto tu computadora y echado un vistazo al interior de la carcasa? No es necesario que lo hagas ahora, pero lo harás. Descubra que está construido a partir de una serie de piezas necesarias para ofrecer su experiencia informática. la tarjeta de video genera una imagen y envía una señal a su monitor. La tarjeta de sonido mezcla el sonido y envía audio a sus parlantes. Luego están los dispositivos de entrada, como el teclado, el mouse y los joysticks, y un variedad de otros artilugios electrónicos, todos los cuales son esenciales para crear un juego. En los primeros días de las computadoras domésticas, los programadores con malos cortes de pelo y gafas de montura gruesa tenían enfrentarse a cada uno de los componentes del ordenador. El programador del juego tuvo que leer la información técnica. manual de cada dispositivo para escribir el código de computadora para comunicarse con él, todo antes de trabajar en el juego real. La situación empeoró cuando los fabricantes lanzaron diferentes dispositivos. y versiones de dispositivos existentes con nuevas capacidades. Los programadores querían soportar tantos dispositivos como fuera posible. posible, por lo que había un mercado más grande para sus juegos, pero se encontraron empantanados en los detalles de trabajar con estas nuevas tarjetas gráficas y de sonido. También fue una molestia para el público comprador de juegos, que había para marcar cuidadosamente la casilla para ver si tenían la combinación correcta de dispositivos para que el juego funcionara. Las cosas se volvieron un poco más fáciles con la introducción de sistemas operativos gráficos como Microsoft. Ventanas. Le dieron al programador del juego una forma única de comunicarse con los dispositivos. Significaba que el programador podría tirar los manuales técnicos porque los fabricantes suministraban drivers, pequeños Programas que manejan la comunicación entre el sistema operativo y el hardware. Avancemos rápidamente hasta tiempos más recientes, cuando los programadores todavía tienen malos cortes de pelo pero llantas más delgadas. sus gafas. La vida de un programador de juegos todavía no es fácil. Aunque existe una forma común de Al comunicarse con gráficos, audio y entradas, aún puede resultar complicado escribir juegos debido a la variedad de hardware en el mercado. La PC familiar barata que mamá compró en el supermercado local es muy diferente de la máquina de alta gama adquirida por un directivo de la empresa. Es esta variedad la que lo hace tan un esfuerzo para inicializar el hardware

y prepararlo para su uso en el juego. Afortunadamente, ahora que Pygame está aquí, tienen una forma de crear juegos sin tener que preocuparse por estos detalles (y los programadores de juegos tienen tiempo salir y cortarse el pelo decentemente). En este capítulo le presentaremos Pygame y le explicaremos cómo usarlo para crear una pantalla gráfica, y leer el estado de los dispositivos de entrada.

Una palanca de mando o joystick (del inglés joy, alegría, y stick, palo) es un periférico de entrada que consiste en una palanca que gira sobre una base e informa su ángulo o dirección al dispositivo que está controlando.

Blitting

El método de superficie de objetos que probablemente utilizarás con más frecuencia es blit, que es un acrónimo de bit transferencia en bloque. Blitting simplemente significa copiar datos de imágenes de una superficie a otra. Lo usarás para ¡Dibujar fondos, fuentes, personajes y casi todo lo demás en un juego!

También puede borrar solo una parte de la superficie agregando un objeto de estilo Rect al parámetro que define la región de origen. Aquí hay dos formas de utilizar el método blit:

```
pantalla.blit(fondo, (0,0))
```

que proyecta una superficie llamada fondo en la esquina superior izquierda de la pantalla. Si el fondo tiene el mismo dimensiones como pantalla, no necesitaremos llenar la pantalla con un color sólido; l otra manera es

```
pantalla.blit(ogro, (300, 200), (100*frame_no, 0, 100, 100))
```

1.0.1. CREAR IMÁGENES CON UN CANAL ALFA

Si tomó una fotografía con una cámara digital o la dibujó con algún software de gráficos, probablemente no tenga un canal alfa. Agregar un canal alfa a una imagen generalmente implica un poco de trabajo con el software de gráficos.

Para agregar un canal alfa a la imagen de un pez, usé Photoshop, pero también puedes usar otro software, como GIMP (www.gimp.org) para hacer esto.

Para obtener una introducción a GIMP, consulte Beginning GIMP de Akkana Peck: De principiante a profesional (Apress, 2006). Una alternativa a agregar un canal alfa a una imagen existente es crear una imagen con una

representación 3D, paquete como 3ds Max de Autodesk o la alternativa gratuita, Blender (www.blender.org). Con este tipo de software puede generar directamente una imagen con un fondo invisible (también puede crear varios fotogramas de animación o vistas desde diferentes ángulos). Esto puede producir los mejores resultados para un juego de apariencia elegante, pero puede hacer mucho con la técnica manual del canal alfa. Intente tomar una fotografía de su gato, perro o pez dorado y ¡Haciendo un juego con esto!

1.0.1.1. Almacenamiento de imágenes

Hay varias formas de almacenar una imagen en su disco duro. A lo largo de los años, muchos formatos para archivos de imagen, cada uno con sus pros y sus contras. Afortunadamente un pequeño número han surgido como los más útiles, dos en particular: JPEG y PNG. Ambos están bien apoyados en software de edición de imágenes y probablemente no tendrá que utilizar otros formatos para almacenar imágenes en un juego.

- JPEG (Grupo conjunto de expertos en fotografía): los archivos de imágenes JPEG suelen tener la extensión .jpg o, a veces, .jpeg. Si utiliza una cámara digital, los archivos que produce probablemente ser archivos JPEG, porque fueron diseñados específicamente para almacenar fotografías. Ellos usan un proceso conocido como compresión con pérdida, que es muy bueno para reducir el tamaño del archivo. La desventaja de la compresión con pérdida es que puede reducir la calidad de la imagen, pero generalmente es tan sutil que no notarás la diferencia. La cantidad de compresión puede también se puede ajustar para lograr un equilibrio entre calidad visual y compresión. Pueden ser excelentes para fotografías, pero los archivos JPEG son malos para cualquier cosa con bordes duros, como fuentes o diagramas, porque la compresión con pérdida tiende a distorsionar este tipo de imágenes. Si usted tiene para cualquiera de estos tipos de imágenes, **PNG probablemente sea una mejor opción.**

- PNG (Portable Network Graphics): los archivos PNG son probablemente las imágenes más versátiles, porque el formato pueden almacenar una amplia variedad de tipos de imágenes y aún así comprimirse muy bien. También admiten canales alfa, lo cual es una verdadera ayuda para los desarrolladores de juegos. La compresión que utilizan los archivos PNG no tiene pérdidas, lo que significa que las imágenes almacenadas como archivos PNG se exac-

tamente igual que los originales. La desventaja es que incluso con una buena compresión probablemente será más grande que los archivos JPEG. Además de JPEG y PNG, Pygame su

Borrador

Capítulo 2

Introducción a la Librería Pygame



Pygame es una librería o paquete de Python para crear juegos, se basa en otra librería llamada Simple DirectMedia Layer (SDL), que fue escrita por Sam Lantinga mientras trabajaba para Loki Software o Loki Games (una empresa de juego desaparecida en 2002) para simplificar la tarea de portar juegos de una plataforma a otra. Proporcionó una forma común de crear una visualización en múltiples plataformas, así como trabajar con gráficos y dispositivos de entradas.

Debido a su sencillez para trabajar con él, se hizo muy popular entre los desarrolladores de juegos. Cuando se lanzó en 1998 y desde entonces se ha utilizado para muchos juegos comerciales y de pasatiempo. SDL fue escrito

en C, un lenguaje comúnmente utilizado para juegos debido a su velocidad y capacidad para trabajar con el hardware. Pero desarrollar en C, o su sucesor C++, puede ser lento y propenso a errores. Entonces los programadores produjeron enlaces a sus lenguajes favoritos, y SDL ahora se puede utilizar desde casi cualquier lenguaje que exista. Uno de esos enlaces es Pygame, que permite a los programadores de Python utilizar la potente biblioteca SDL.

Ambos de código abierto tanto Pygame como SDL, y han estado en desarrollo activo, una gran cantidad de programadores han trabajado para refinar y mejorar esta excelente herramienta para crear juegos.

2.1. Pygame

Pygame es un conjunto de módulos de Python para desarrollar videojuegos. Este programa de desarrollo e informática proporciona a los usuarios una biblioteca Simple DirectMedia Layer que contiene gráficos de ordenador y clips de sonido. La principal ventaja de este software es su portabilidad y su soporte multiplataforma. Con esta aplicación, los usuarios pueden distribuir sus juegos basados en Python y probar los trabajos de otros.

En las siguientes secciones se presenta un breve tutorial o guía sobre las principales características y funciones de Pygame. En caso de que no hayas instalado Python ni Pygame en tu computadora, en la próxima sección se dan las instrucciones de instalación, si ya ha instalado ambos entonces puedes saltar y continuar en la siguiente.

La librería Pygame al igual que Python viene con varios módulos que proporcionan funciones adicionales para sus programas; incluye varios módulos con funciones para dibujar gráficos, reproducir sonidos, manejar la entrada del mouse y otros.

Este capítulo se describen algunos los módulos y funciones básicos que Pygame proporciona, y se asume que conoce un poco de la programación básica de Python. Si tiene problemas con algunos conceptos de programación, puedes leer el libro "Inventa tus propios juegos de computadora con Python", en línea en <http://invy.com/book>. Este libro está dirigido a principiantes totales en programación; o revisare le Libro Python 3 de Linis Guerrero, una compilación de varios textos.

Para obtener información más detallada sobre Pygame, sobre los módulos

que proporciona Pygame se recomienda revisar la documentación en línea en la web www.pygame.org/docs/.

2.1.1. Instalación de la Librería PyGame

Para descargar e instalar la librería PyGame en su sistema operativo puede visitar la web <https://www.pygame.org/news>. En esta web encontrará el método para descargar la versión que desee instalar en su sistema. Además en esta página podrá conocer más sobre pygame sus módulos y las nuevas noticias acerca de sus versiones.

Por supuesto que PyGame requiere de Python (Python 3); Si aún no lo tienes, puedes descargarlo desde <https://www.python.org/>. Se recomienda ejecutar la última versión de Python, porque suele ser más rápida y tiene mejores funciones que las anteriores.

La mejor manera de instalar pygame es con la herramienta pip (que es la que usa Python para instalar paquetes), en la consola o terminal mcd (símbolo del sistema). Tenga en cuenta que esta herramienta viene con Python en versiones recientes. Usamos el indicador `--user` para indicarle que se instale en el directorio de inicio, en lugar de hacerlo globalmente.

```
python -m pip install -U pygame==2.5.2 --user
```

Una vez que haya instalado el paquete apropiado, puede probarlo abriendo el intérprete de Python e ingresando las siguiente línea de código:

```
>>> import pygame
```

Si Pygame se instaló correctamente, debería ver la versión mostrada:

```
pygame 2.5.2 (SDL 2.28.3, Python 3.12.0)  
Hello from the pygame community. https://www.pygame.org/contribute.html
```

Además puede ver la versión de pygame con `print(pygame.ver)`:

```
>>> print(pygame.ver)  
2.5.2
```

Al momento de escribir este artículo, la versión 2.5.2 es la más reciente, pero es posible que encuentre una versión posterior, que seguro estará disponible cuando estés leyendo este libro. El código de ejemplo seguirá funcionando, ya que las versiones más nuevas de Pygame tienden a ser compatibles con versiones anteriores. (Tenga en cuenta que pygame ha dejado de admitir Python 2).

2.1.2. Módulos de la Librería PyGame

El paquete PyGame contiene una serie de módulos que se pueden utilizar de forma independiente. Hay un módulo para cada uno de los dispositivos que puedes usar en un juego, y muchos otros para crear juegos de manera muy sencilla.

Siempre tendrás algún tipo de exhibición, por lo tanto, el módulo de visualización es esencial y definitivamente necesitará algún tipo de información, ya sea del teclado, joystick o mouse. Otros módulos se utilizan con menos frecuencia, pero en combinación te brindan una de las herramientas de creación de juegos más poderosas que existen.

En la Tabla 2.1.2 se presentan algunos módulos de Pygame:

Tabla 2.1.2. Módulos de la librería Pygame:

Nombre	Propósito del módulo
pygame.cdrom	Accede y controla unidades de CD
pygame.cursors	Carga imágenes del cursor
pygame.display	Accede a la pantalla
pygame.draw	Dibuja formas, líneas y puntos
pygame.event	Gestiona eventos externos
pygame.font	Utiliza fuentes del sistema
pygame.image	Carga y guarda una imagen
pygame.joystick	Utiliza joysticks y dispositivos similares
pygame.key	Lee las pulsaciones de teclas del teclado
pygame.mixer	Carga y reproduce sonidos
pygame.mouse	Administra el mouse
pygame.movie	archivos de película
pygame.music	Funciona con música y transmisión de audio
pygame.overlay	Accede a superposiciones de vídeo avanzadas
pygame.rect	Contiene funciones de Pygame de alto nivel
pygame.rect	Gestiona áreas rectangulares
pygame.sndarray	manipula datos de sonido
pygame.sprite	Gestiona imágenes en movimiento
pygame.surface	Gestiona las imágenes y la pantalla
pygame.surfarray	Manipula los datos de píxeles de la imagen
pygame.time	Gestiona el tiempo y la velocidad de fotogramas
pygame.transform	Cambia el tamaño y mueve imágenes

Sin embargo, no hay garantía que todos los módulos de la Tabla 2.1.2 estén presentes para todas las plataformas. Es posible que el hardware en el que se ejecuta el juego no tenga ciertas capacidades, o que no estén instalados los controladores (drivers) necesarios. Si este es el caso, Pygame configurará

el módulo como `None`, lo que facilita la prueba.

El siguiente fragmento de código detectará si el módulo `pygame.font` está disponible o no:

```
if pygame.font is None:
    print("El módulo font (fuentes) no esta disponible!")
    exit()
```

A través de el espacio de nombres de `pygame` se accede a estos módulos; por ejemplo, `pygame.display` se refiere al módulo de visualización.

2.2. Mi primer Juego

Siguiendo la tradición de usar el popular "¡Hola, mundo!" al iniciarse en un nuevo lenguaje. En nuestro primer juego o programa con `pygame` que vamos a crear, usaremos el texto "¡Hola, Mundo Pygame!". En este caso escribimos el texto "¡Hola, Mundo Pygame!" en la pantalla como nombre del juego. No se desanime, a pesar de ser una línea de texto simplemente, y no muy interesante para empezar a crear juegos, iniciamos con una ventana gráfica en su escritorio y luego seguiremos con imágenes atractivas.

Vamos a crear nuestro primer archivo o script `mi_primer_pygame.py`, que abre una ventana gráfica en su escritorio con el nombre "¡Hola, Mundo Pygame!", al inicio con un fondo de pantalla negro, pero le iremos agregando elementos para hacerlo más interesante.

```
mi_primer    import pygame
_pygame.py   from pygame.locals import *
              from sys import exit

pygame.init()
screen = pygame.display.set_mode((640, 480), 0, 32)
pygame.display.set_caption("¡Hola, Mundo Pygame!")

while True:  # bucle principal del juego
    for in pygame.evento.get(): # bucle de eventos
        if evento.type == QUIT:
            pygame.quit()
            exit()

    pygame.display.update()
```

Cuando ejecutes este programa, aparecerá una ventana como la ventana que se muestra en la figura 2.1, pero con fondo negro. Hasta ahora, muy bien, ¡Acabas de crear un videojuego muy simple!. Es solo una ventana con fondo

negro, con el nombre ¡Hola, Mundo Pygame! en la parte superior (se le llama la barra de título de la ventana, por defecto `windows pygame`).

El primer paso para crear juegos gráficos es crear una ventana; y finalizar el juego haciendo clic en el botón **X** en la esquina superior derecha de la ventana, el programa finalizará y la ventana desaparecerá.



Llamar a la función `print()` para que aparezca texto en la ventana no funcionará porque `print()` es una función para programas CLI (interfaz de línea de comando); lo mismo sucede con `input()` para obtener la entrada del teclado del usuario.

Pygame utiliza otras funciones para entrada y salida que se explican más adelante.

Ahora continuamos describiendo con mayor detalle cada línea del código fuente de nuestro programa.

- La primera línea es una declaración simple que importa la librería `pygame` para que nuestro programa pueda utilizar los módulos y las funciones que contiene. Tenga en cuenta que cuando importa el módulo `pygame`, se importa automáticamente todos los módulos están Pygame, como `pygame.image` y `pygame.mixer.music`. Todas las funciones de Pygame relacionadas con gráficos, sonido, y otras características que proporciona Pygame se encuentran en el módulo `pygame`. No es necesario importar estos módulos dentro de módulos con declaraciones de importación adicionales.

- La segunda línea también es una declaración de importación. Sin embargo, en lugar del formato de nombre del módulo de importación, utiliza el formato `from modulename import *`: `from pygame.locals import *`.

Normalmente, si desea llamar a una función que está en un módulo, debe usar el formato `modulename.functionname()` después de importar el módulo. Sin embargo, con `from modulename import *`, puede omitir parte del nombre del módulo y simplemente usar el `functionname()`, semejante con las funciones integradas de Python.

La razón por la que utilizamos esta forma de declaración de importación para `pygame.locals` es porque contiene varias variables constantes que son fáciles de identificar como parte del módulo `pygame.locals` sin `pygame.locals` en frente de ellos.

Este módulo contiene varias constantes usadas por `pygame`¹. Su contenido

¹Para obtener documentación completa sobre los módulos de Pygame, consulte www.pygame.org/docs/.

se coloca automáticamente en el espacio de nombres (namespace) del módulo `pygame`. Sin embargo, en una aplicación se puede usar `pygame.locals` para incluir solo las constantes de `pygame` con la sentencia:

```
from pygame.locals import *
```

y usar constantes como: mejorar y completar referencias `QUIT` del módulo `pygame.event`; el módulo `pygame.key` enumera las constantes y modificadores del teclado (`K_*` y `MOD_*`) relacionados con los atributos `key` y `mod` de los eventos `KEYDOWN` y `KEYUP`; el módulo `pygame.display` contiene indicadores como `FULLSCREEN` utilizados por `pygame.display.set_mode()`; el módulo `pygame.time`, define `TIMER_RESOLUTION`.

Las dos primeras líneas de código en el programa son líneas que comenzarán casi en todo programa que escribes con Pygame.✓

- La tercera línea se importa `exit` de `sys` (un módulo en la biblioteca estándar), para cerrar la ventana `pygame`, luego de cerrar `pygame`, al recibir el objeto evento `QUIT` (al llamarlo hará que la ventana de Pygame desaparezca). Si usa `import sys` en lugar de `from sys import *` en su programa, tendrías que llamar a `sys.exit()` en lugar de `exit()`. Sin embargo, la mayoría de las veces es mejor usar el nombre completo del módulo y la función para tener claro en qué módulo se usa y evitar confusiones.

- La cuarta línea es el llamado a la función `pygame.init()`, que siempre debe llamarse después de importar el módulo `pygame` y antes de llamar a cualquier otra función de Pygame.

Inicializa el módulo `pygame.init()`, debe llamarse primero para que muchas funciones de Pygame trabajen. Si alguna vez ve un mensaje de error como `pygame.error: fuente no inicializado`, verifique que seguro olvidó llamar a `pygame.init()` al comienzo de su programa.



El script llamará a salir cuando el usuario haga clic en el botón cerrar; de lo contrario, el usuario no tendría forma de cerrar la ventana! Si te encuentras en una situación en la que no puedes cerrar la ventana de Pygame, es posible que puedas detener Python en seco presionando **Ctrl+C**. Esta línea bastante simple de código Python en realidad hace un trabajo muy importante

- La quinta línea es una llamada a la función `pygame.display.set_mode()`, que devuelve el objeto `pygame.Surface` para la ventana (los objetos de superficie se describen más adelante en la sección ??).

Primer parámetro:

Observe que pasamos un valor de tupla de dos números enteros (640, 480) a la función, que le indica a la el ancho y el alto de la ventana en píxeles. En esta caso creará una ventana con 640 píxeles de ancho y 480 píxeles de alto.

Una llamada a función como `pygame.display.set_mode(640, 480)` provocará un error que se ve así: `TypeError`: el argumento debe ser una tupla de 2 elementos, no entero. El objeto `pygame.Surface` (los llamaremos simplemente objetos Surface para abreviar) devuelto es almacenado en una variable llamada `DISPLAYSURF`. Lo primero que hacemos con nuestra superficie recién creada es agregarle el nombre con el método `set_caption` en el módulo de visualización para configurar la barra de título del ventana de Pygame. Configuramos el título como "¡Hola, Mundo Pygame!", solo para que sea un guión válido del juego.

Segundo parámetro:

El parámetro 32: Esta es la profundidad de color de la superficie de visualización. Especifica el número de bits utilizados para representar cada píxel en la superficie de visualización, utilizados para almacenar colores en la pantalla. En este caso, 32 bits se utilizan, lo que significa que cada píxel puede tener 2^{32} (o 4.294.967.296) colores diferentes.

Un bit, o dígito binario, es la unidad de almacenamiento fundamental en una computadora. Los bits tienen exactamente dos valores potenciales, 1 o 0, y están organizados en la memoria como grupos de 8. Un grupo de 8 bits se llama byte. No te preocupes si esto te suena a tecno-charla. Python tiende a ocultar este tipo de cosas al programador. Usaremos el valor 32 para nuestra profundidad de bits porque nos da la mayor cantidad de colores.

En la tabla 2.2 se resume los valores de profundidad de color en bits, puede consultar para conocer otros posibles valores de profundidad en bits.

Tabla 2.2. Valores de profundidad en bits:

Profundidad	Número de colores
8 bits	256 colores, elegidos entre una paleta de colores más amplia
15 bits	32,768 colores, con un bit libre
16 bits	65.536 colores
24 bits	16.7 millones de colores, con un a spare bit
32 bits	16,7 millones de colores, con un 8 bit libre

Es posible tener otras profundidades de bits, pero estas son las más común.



Algunas veces, Pygame no puede generar la visualización exacta que configuramos, tal vez se debe a que la tarjeta gráfica no admite esa configuración. Afortunadamente, Pygame elegirá una pantalla que sea compatible con el hardware y emula la pantalla que realmente hemos configurado.

Hay más parámetros indicadores para la función `.set_mode()`. Los parámetros indicadores (flags) de `pygame.display.set_mode()`, que activar o desactivar una función de la ventana Pygame; y se puede combinar varios parámetros indicadores con el operador bitwise OR (`|`) (bit a bit).

Pero por ahora no los trataremos, solo se dará el valor de 0, que el valor por defecto. En la sección 2.8.4 se gestiona los parámetros `NOFRAME`, `RESIZABLE`, `DOUBLEBUF` y `HWSURFACE`, para tener una idea puede consultar la tabla 2.8.4, que resume los parámetros indicadores de visualización de la función `pygame.display.set_mode()` que puede utilizar.

Table 2.8.4. Parámetros indicador de visualización:

Parámetro indicador	Propósito
FULLSCREEN	Crea una visualización que ocupa toda la pantalla
DOUBLEBUF	Crea una visualización de "doble búfer"
	Recomendado para HWSURFACE u OPENG
HWSURFACE	Crea una pantalla acelerada por hardware
	Debe combinarse con FULLSCREEN
OPENG	Crea una visualización renderizable OpenGL
RESIZABLE	Crea una visualización redimensionable
NOFRAME	Elimina el borde y la barra de título de la ventana

- La línea 6 establece el texto del título que aparecerá en la parte superior de la ventana al llamando de la función `pygame.display.set_caption()`. El valor de cadena "¡Hola, Mundo Pygame!" es pasado en el llamado de función para que el texto aparezca como título de la ventana.

- La línea 7 es un bucle `while` que tiene como condición simplemente el valor `True`, es el bucle principal del juego. El trabajo de este bucle es repetir continuamente el bloque; es decir, nunca sale del bucle, pues su condición para salir se evalúa como `False`. La única forma de salir del bucle es ejecutando una instrucción `break` (que mueve la ejecución a la primera línea después de la bucle), o forzarlo a salir de alguna otra manera, en este caso sale con

`exit()` (`sys.exit()`) que finaliza el programa al hacer clic en el botón X en la esquina superior derecha de la ventana, el programa finalizará y la ventana desaparecerá.

Todos los juegos tendrán un bucle similar a este, que normalmente se repite una vez por actualización de pantalla. Dentro del bucle principal del juego tenemos otro bucle: el bucle de eventos, que la mayoría de los juegos ejecutarán. Todos los juegos de este libro tienen estos bucles `while True` junto con un comentario que llama es el “bucle principal del juego”.

Un bucle de juego, también llamado bucle principal es un bucle donde el código hace tres cosas:

1. Maneja eventos.
2. Actualiza el estado del juego.
3. Dibuja el estado del juego en la pantalla.

El estado del juego es simplemente una forma de referirse a un conjunto de valores para todas las variables de un juego. En muchos juegos, el estado incluye los valores de las variables que rastrean el la salud y la posición del jugador, la salud y la posición de los enemigos, qué marcas se han realizado en un tablero, la puntuación o a quién le toca. Cada vez que sucede algo como el jugador recibe un daño (lo que reduce su valor de salud), o un enemigo se mueve a algún lugar, o algo sucede en el mundo del juego, decimos que el estado del juego ha cambiado.

Si alguna vez has jugado un juego que te permite guardar, el "estado de guardado" es el estado del juego en el punto en el que lo has guardado. En la mayoría de los juegos, pausar el juego evitará que cambie el estado del juego. Dado que el estado del juego generalmente se actualiza en respuesta a eventos (como clics del mouse o pulsaciones del teclado) o el paso del tiempo, el bucle del juego está constantemente comprobando y volviendo a comprobar muchas veces un segundo para cualquier evento nuevo que haya ocurrido.

Gestión de eventos externos

Dentro del bucle principal `while` del juego, hay otro bucle: el bucle de eventos (`for event in pygame.event.get()`), que analiza si han creado eventos y se hace con función `pygame.event.get()` (mayoría de los juegos ejecutarán).

```
while True:    # bucle principal del juego
    for evento in pygame.event.get(): # bucle de eventos
```



```
if evento.type == QUIT:
    pygame.quit()
    exit()
```

Un evento es la forma en que Pygame informa que algo sucedió fuera de su código. Los eventos son creados para muchas cosas, desde presionar teclas hasta recibir información de Internet, y están en cola para que Usted los maneje. La función `get()` en el módulo `pygame.event` devuelve cualquier evento que haya ocurrido, y que se puede recuperar con un bucle `for`.

En este `script`, solo estamos interesado en el evento `QUIT`, que es generado por Pygame cuando el usuario hace clic en el botón cerrar en la ventana de Pygame (si no se incluye tendremos un juego inmortal, una ventana que no cierra). Entonces, si el tipo de evento es `QUIT`, se llama a `exit()` para cerrar, y todos los demás eventos son ignorados.

En un juego, por supuesto, tendríamos que gestionar una mayor cantidad de eventos. Pygame crea otros eventos para informarle sobre cosas como el movimiento del mouse y la pulsación de teclas (en la sección 2.7 se estudian con mas detalles los eventos).

- La siguiente línea, el bloque del condicional `if evento.type == QUIT`, que evalúa si el evento es la salida, en caso afirmativo cierran el juego: Es importante llamar a `pygame.quit()` y `exit()` para cerrar completamente Pygame. La función `pygame.quit()` es algo así como lo opuesto a la función `pygame.init()`, al ejecutarla desactiva la librería `pygame` (cierra `pygame`), y `exit()` cierra de la ventana.

Bucle principal del juego: while

- En la última línea del bloque `while`, tenemos el código que evita los destellos de pantalla cuando creas una imagen: `pygame.display.update()`. Esto es porque Pygame primero construye una imagen en un buffer de respaldo, que es una visualización invisible en la memoria, antes de ser desplegado.

Si no tuviéramos este paso, el usuario vería destellos individuales a medida que ocurren, parpadearían de lo más desagradable. Para los programadores de juegos, ¡el parpadeo es el enemigo!. Afortunadamente, una llamada a `pygame.display.update()` es todo lo que necesitamos para asegurarnos de que la imagen que tenemos creado en la memoria se muestra al usuario sin parpadeo.

2.3. Actualizando Mi primer Juego

Es hora de hacer un juego mas atractivo. Ahora le agregaremos dos imágenes, una para usar como fondo de pantalla `pygame.jpg`, y la otra `cubo.jpg` que usaremos puntero nuestro el mouse (cursor del ratón).

Puede descargar estos archivos para este y otros ejemplos desde código fuente o crearlos para a su necesidad. Si no tienes acceso a Internet en este momento, puedes usar archivos de imágenes que tenga en su disco duro o crearlos con cualquier edición gráfica o fotográfica software. Cualquier imagen está bien para el fondo, siempre que tenga un tamaño mínimo de 640 por 480, que es el tamaño de pantalla que hemos creado (si es más grande, se recortará el exceso). Para el cursor del mouse, necesitará una imagen más pequeña, que encaje cómodamente dentro del fondo; un buen tamaño es 80 por 80.

```
mi_primer
pygame.py

background_image_filename = 'pygame.jpg'
mouse_image_filename = 'cubo.jpg'

import pygame
from pygame.locals import *
from sys import exit

pygame.init()
screen = pygame.display.set_mode((640, 480), 0, 32)
pygame.display.set_caption("¡Hola, Mundo Pygame!")

background = pygame.image.load(background_image_filename).convert()
mouse_cursor = pygame.image.load(mouse_image_filename).convert_alpha()

while True:
    for evento in pygame.event.get():
        if evento.type == QUIT:
            exit()

    screen.blit(background, (0,0))
    x, y = pygame.mouse.get_pos()
    x-= mouse_cursor.get_width()/2
    y-= mouse_cursor.get_height()/2
    screen.blit(mouse_cursor, (x, y))
    pygame.display.update()
```

Al ejecutar el archivo, obtendrá una ventana similar a la que se muestra en la Figura 2.1.

- Las dos primeras líneas establecen los nombres de los archivos de imágenes; si está utilizando imágenes diferentes, debe reemplazar los nombres de los archivos con la ubicación de mismo.
- El siguiente par de líneas nuevas en el programa se usa la función

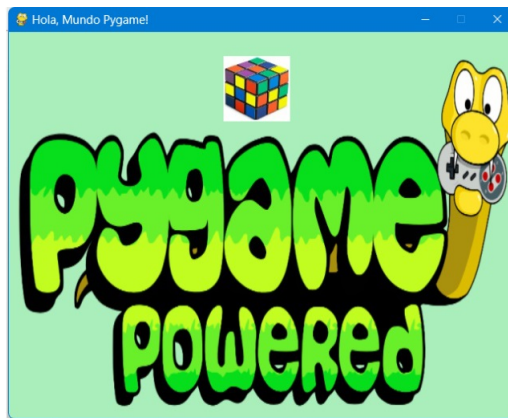


Figura 2.1: Hola, Mundo en Pygame

`pygame.image.load()` para cargar las dos imágenes, para el fondo y para el cursor del ratón. Pasamos los nombres de los archivos de las imágenes almacenadas al inicio del `script`:

```
background = pygame.image.load(background_image_filename).convert()
mouse_cursor = pygame.image.load(mouse_image_filename).convert_alpha()
```

La función de carga lee un archivo de su disco duro y devuelve una superficie que contiene datos de la imagen. Estos son el mismo tipo de objetos que nuestra pantalla, pero representan imágenes almacenadas en la memoria.

La primera llamada a `pygame.image.load()` lee la imagen de fondo y luego llama inmediatamente a la función `convert()`, que es una función miembro para objetos de superficie. Esta función convierte la imagen al mismo formato que nuestra pantalla, por ser más rápido dibujar imágenes si la pantalla tiene la misma profundidad.

El cursor del mouse se carga de manera similar, pero llamamos la función `convert_alpha()` en lugar de `convert()`. Esto se debe a que la imagen del cursor del mouse contiene información alfa, lo que significa que partes de la imagen pueden ser translúcidas o completamente invisibles. Sin información alfa en la imagen de nuestro mouse, estamos limitados a un Cuadrado o rectángulo antiestético como el cursor de nuestro mouse! (en la sección ?? se cubrirá alfa y los formatos de imagen con más detalle).

Siguiendo con la descripción del código fuente del programa: en el bloque del `while` (bucle principal) y después del bucle de eventos, se agrega una línea de código: `screen.blit(background, (0,0))`, esta función transfiere

la imagen de fondo a la pantalla ("blit" significa copiar una imagen sobre otra).

- Esta línea utiliza la función `blit()` miembro el objeto `Surface` de pantalla, que toma una imagen (en este caso, nuestro fondo de 640×480) y una tupla que contiene la posición de destino. El fondo nunca se moverá; solo queremos que cubra toda la ventana de Pygame, así que nos dirigimos a la coordenada $(0, 0)$, que está en la parte superior izquierda de la pantalla.



Es importante que la imagen abarque cada parte de la pantalla. Si no lo hace, pueden producirse efectos visuales extraños cuando presenta animaciones, y el juego puede verse diferente en cada computadora en la que se ejecuta.

- En las siguientes líneas código son para dibujar `mouse_cursor` debajo del puntero habitual del mouse:

```
screen.blit(background, (0,0))
x, y = pygame.mouse.get_pos()
x-= mouse_cursor.get_width()/2
y-= mouse_cursor.get_height()/2
screen.blit(mouse_cursor, (x, y))
pygame.display.update()
```

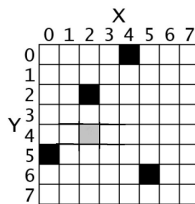
Obtener la posición del mouse es muy sencillo; el módulo `pygame.mouse` contiene todo lo que necesitamos para trabajar con el mouse. La función `pygame.mouse.get_pos()` devuelve una tupla que contiene las coordenadas del mouse.

- La primera línea descomprime esta tupla en dos valores por conveniencia: `x` e `y`. Podríamos usar estos dos valores como coordenadas cuando desplazamos el cursor del mouse, pero eso colocaría la esquina superior izquierda de la imagen debajo del mouse, y queremos que el centro de la imagen esté debajo del mouse. Así que hacemos un poco de cálculo (¡no temas!) para ajustar `x` e `y` para que la imagen del mouse se mueva hacia arriba la mitad de su altura y hacia la izquierda la mitad de su ancho. Usando estas coordenadas coloca el centro de la imagen justo debajo del puntero del mouse, lo que se ve mejor. Al menos lo hace para la imagen del cubo de rubik; si desea utilizar una imagen de puntero más típica, ajuste las coordenadas para que la punta se encuentra debajo de las coordenadas reales del mouse.

- La línea de código `screen.blit(mouse_cursor, (x, y))`, se utiliza nuevamente la función `blit()`, como el caso de la imagen de fondo; toma una imagen y una tupla que contiene la posición de destino.✓

2.3.1. Coordenadas de píxeles

La ventana que crea el programa `mi_primer_pygame` está compuesta simplemente por pequeños puntos cuadrados en tu pantalla llamada píxeles. Cada píxel comienza en negro, pero se puede configurar en un color diferente. Nuestro objeto de Superficie tiene 640 píxeles de ancho y 480 píxeles de alto. Para ilustrar suponga que tenemos un objeto de Superficie con 8 píxeles de ancho por 8 píxeles de alto. Si esa pequeña superficie de 8x8 se amplía para que cada píxel pareciera un cuadrado en una cuadrícula, y agregamos números para los ejes X e Y, luego una buena representación del mismo podría verse algo como esto:



Podemos referirnos a un píxel específico utilizando un sistema de coordenadas cartesianas. Cada columna del eje X y cada fila del eje Y tendrá una "dirección" que es un número entero de 0 a 7, de modo que puede localizar cualquier píxel especificando los números enteros de los ejes X e Y. Por ejemplo, en la imagen de 8x8 de arriba, podemos ver que los píxeles en las coordenadas XY (4, 0), (2, 2), (0, 5) y (5, 6) se han pintado de negro, el píxel en (2, 4) se ha pintado de gris, mientras que todos los otros píxeles están pintados de blanco. Las coordenadas XY también se llaman puntos. Si has tomado una clase de matemática y aprendió sobre las coordenadas cartesianas, puede notar que el eje Y comienza en 0 en el origen y luego aumenta hacia abajo, en lugar de aumentar a medida que sube. Así funcionan coordenadas en Pygame (y en casi todos los lenguajes de programación).

El marco de Pygame a generalmente representa las coordenadas cartesianas como una tupla de dos números enteros, como (4, 0) o (2, 2). El primer número entero es la coordenada X y el segundo es la coordenada Y. (Las coordenadas cartesianas se tratan con más detalle en el capítulo 12 de *Inventa tus propios juegos de computadora*)

2.3.2. Objetos de superficie y la ventana

Los objetos de superficie son objetos que representan una imagen 2D rectangular. Los píxeles del objeto Superficie se puede cambiar llamando a las funciones para dibujo de Pygame (descritas más adelante en este seccion ??) y luego mostrado en la pantalla. El borde de la ventana, la barra de título y los botones no forman parte de objeto de superficie la pantalla.

En particular, el objeto Surface devuelto por `pygame.display.set_mode()` se llama superficie de visualización. Todo lo que se dibuje en el objeto de superficie de visualización se mostrará en la ventana cuando se llama a la función `pygame.display.update()`. Es mucho más rápido dibujar en un objeto Superficie (que sólo existe en la memoria de la computadora) que dibujar una Superficie de pantalla de la computadora. La memoria de la computadora es mucho más rápida de cambiar que los píxeles de una monitor.

A menudo su programa dibujará varias cosas diferentes en un objeto Surface. Una vez que hayas terminado dibujando todo en el objeto superficie de la pantalla para esta iteración del bucle del juego (llamado marco, tal como se llama una imagen fija en un DVD en pausa) en un objeto de superficie, se puede dibujar en la pantalla. La computadora puede dibujar cuadros muy rápidamente y nuestros programas a menudo se ejecutarán de un lado a otro 30 fotogramas por segundo (es decir, 30 FPS). Esto se llama "velocidad de fotogramas" y se explica más adelante en ??.

El dibujo sobre objetos de superficie se tratará en las “Funciones de dibujo primitivas” y “Funciones de dibujo”. Imágenes” más adelante en este capítulo.

2.3.3. Colores

Hay tres colores primarios de luz: rojo, verde y azul (El rojo, el azul y el amarillo son los colores primarios para pinturas y pigmentos, pero el monitor de la computadora usa luz, no pintura), combinando diferentes cantidades de estos tres colores puedes formar cualquier otro color. En Pygame, se representar colores con tuplas de tres números enteros. El primer valor de la tupla es el color rojo. Un valor entero de 0 significa que no hay rojo en este color y un valor de 255 significa que tiene la cantidad máxima de rojo ese el color. El segundo valor es para verde y el tercer valor es para azul.

Estas tuplas de tres números enteros que se utilizan para representar un color suelen denominarse valores RGB, debido a que puedes usar cualquier combinación de 0 a 255 para cada uno de los tres colores primarios, esto

significa que Pygame puede dibujar 16.777.216 colores diferentes (es decir, 256 x 256 x 256 colores). Sin embargo, si intenta utilizar un número mayor que 255 o un número negativo, obtendrá un error parecido a "ValueError: argumento de color no válido". Por ejemplo, crearemos la tupla (0, 0, 0) y la almacenaremos en una variable llamada NEGRO. Con sin cantidad de color (luz) rojo, verde o azul, el color resultante es completamente negro. El color negro es la ausencia de cualquier color. La tupla (255, 255, 255) para una cantidad máxima de rojo, verde y azul para dar como resultado blanco. El color blanco es la combinación completa de rojo, verde y azul. La tupla (255, 0, 0) representa la cantidad máxima de rojo pero no la cantidad de verde y azul, por lo que el color resultante es rojo. De manera similar, (0, 255, 0) es verde y (0, 0, 255) es azul.

Puedes mezclar la cantidad de rojo, verde y azul para formar otros colores. Aquí están

En la Tabla 2.3.3 se presentan algunos valores de RGB para los colores más comunes:

Table 2.3.3. Algunos colores más comunes:

Color	Valores de RGB
Agua	(0, 255, 255)
Negro	(0, 0, 0)
Azul	(0, 0, 255)
Fucsia	(255, 0, 255)
Gris	(128, 128, 128)
Verde	(0, 128, 0)
Lima	(0, 255, 0)
Granate	(128, 0, 0)
Azul marino	(0, 0, 128)
Oliva	(128, 128, 0)
Púrpura	(128, 0, 128)
Rojo	(255, 0, 0)
Plata	(192, 192, 192)
Verde azulado	(0, 128, 128)
Blanco	(255, 255, 255)
Amarillo	(255, 255, 0)

2.3.4. Colores transparentes

Cuando miras a través de una ventana de vidrio que tiene un tinte rojo intenso, todos los colores detrás de ella tienen un un tono rojo. Puede imitar este efecto agregando un cuarto valor entero de 0 a 255 a tus valores de color. Este valor se conoce como valor **alfa**. Es una medida de qué tan opaco es un color (es decir, no transparente). Normalmente, cuando dibujas un píxel en un objeto superficie, el nuevo color reemplaza por completo a cualquier

color que ya estuviera allí. Pero con colores que tienen un valor alfa, puedes en su lugar, simplemente agregar un tinte de color al color que ya está allí.

Por ejemplo, esta tupla de tres números enteros para el color verde es (0, 255, 0). Pero si añadimos un cuarto entero para el valor **alfa**, podemos convertirlo en un color verde medio transparente: (0, 255, 0, 128). Un valor **alfa** de 255 es completamente opaco (es decir, no es transparente en absoluto). Los colores (0, 255, 0) y (0, 255, 0, 255) se ven exactamente iguales. Un valor **alfa** de 0 significa que el color es completamente transparente. Si dibujas cualquier color que tenga un valor **alfa** de 0 en un objeto Superficie, no tendrá ningún efecto, porque este color es completamente transparente e invisible.

Para dibujar usando colores transparentes, debes crear un objeto Superficie con el método `convert_alpha()`. Por ejemplo, el siguiente código crea un objeto Superficie que se pueden dibujar colores transparentes en:

```
anotherSurface = DISPLAYSURF.convert_alpha()
```

Una vez que se han dibujado cosas en el objeto Superficie almacenado en otra Superficie, entonces `anotherSurface` se puede "borrar" (es decir, copiar) en `DISPLAYSURF` para que aparezca en la pantalla (Consulte la sección "Dibujar imágenes con `pygame.image.load()` y `blit()`" más adelante ??).

Es importante tener en cuenta que no puede utilizar colores transparentes en objetos Surface que no hayan sido devueltos por una llamada `convert_alpha()`, incluida la superficie de visualización que se devolvió `pygame.display.set_mode()`.

Si tuviéramos que crear una tupla de colores para dibujar el legendario Unicornio Rosa Invisible, usaríamos (255, 192, 192, 0), que termina pareciendo completamente invisible como cualquier otro color que tiene un 0 como valor **alfa**., después de todo, es invisible.

2.3.4.1. Objetos `pygame.Color`

Necesitas saber cómo representar un color porque las funciones de dibujo de Pygame necesitan una manera de saber qué color quieres usar. Una tupla de tres o cuatro números enteros es unidireccional. Otra forma es como un objeto `pygame.Color`. Puede crear objetos de color llamando a la función `pygame.Color` constructora y pasando tres o cuatro números enteros, puedes almacenar este objeto Color en variables del mismo modo que puedes almacenar tuplas en variables. Intente escribir lo siguiente en el shell interactivo:


```
>>> importar pygame
>>> pygame.Color(255, 0, 0)
(255, 0, 0, 255)
>>> miColor = pygame.Color(255, 0, 0, 128)
>>> miColor == (255, 0, 0, 128)
True
```

Cualquier función de dibujo en Pygame que tenga un parámetro para el color, tiene la forma de tupla o la forma de objeto de color para pasar el argumento. A pesar de que son tipos de datos diferentes, un objeto `Color` es igual a una tupla de cuatro números enteros si ambos representan el mismo color (al igual que `42 == 42.0` se evaluará como Verdadero).

Ahora que sabes cómo representar colores (como un objeto `pygame.Color` o una tupla de tres o cuatro números enteros para rojo, verde, azul y, opcionalmente `alfa`) y coordenadas (como una tupla de dos números enteros para X e Y), aprendamos sobre los objetos `pygame.Rect` para que podamos comenzar a usar las funciones para dibujar de Pygame.

2.4. Módulo `pygame.Rect` y objetos `Rect`

Pygame tiene dos formas de representar áreas rectangulares (al igual que hay dos formas de representar colores). La primera es una tupla de cuatro números enteros:

1. La coordenada X de la esquina superior izquierda.
2. La coordenada Y de la esquina superior izquierda.
3. El ancho (en píxeles) del rectángulo.
4. Luego la altura (en píxeles) del rectángulo.

La segunda forma es como un objeto `pygame.Rect`, al que llamaremos objetos `Rect` para abreviar.

Por ejemplo, el siguiente código crea un objeto `Rect` con una esquina superior izquierda en (10, 20) que tiene 200 píxeles de ancho y 300 píxeles de alto:

```
>>> import pygame
>>> rectangulo = pygame.Rect(10, 20, 200, 300)
>>> rectangulo == (10, 20, 200, 300)
True
```

Lo útil de esto es que el objeto `Rect` calcula automáticamente las coordenadas para las otras características del rectángulo. Por ejemplo, si necesita saber la coordenada X del borde derecha del objeto `pygame.Rect` que almacenamos

en la variable `spamRect`, simplemente puede acceder al atributo derecho del objeto `Rect`:

```
>>> rectangulo.right
210
```

El código Pygame para el objeto `Rect` calculó automáticamente que si el borde izquierdo está en la X coordenada 10 y el rectángulo tiene 200 píxeles de ancho, entonces el borde derecho debe estar en la X coordenada 210. Si reasigna el atributo `right`, todos los demás atributos se eliminan automáticamente, recalculado:

```
>>> rectangulo.right= 350
>>> rectangulo.right
150
```

En la Tabla 2.4 se muestra una lista de todos los atributos que proporcionan los objetos `pygame.Rect` (en el ejemplo, la variable que almacena el objeto `Rect` es `rectangulo`):

Table 2.4. Lista de los atributos del objeto `Rect`:

Atributo	Descripción
<code>rectangulo.left</code>	El valor int de la coordenada X del lado izquierdo del rectángulo
<code>rectangulo.right</code>	El valor int de la coordenada X del lado derecho del rectángulo
<code>rectangulo.top</code>	El valor int de la coordenada Y del lado superior del rectángulo
<code>rectangulo.bottom</code>	El valor int de la coordenada Y del lado inferior
<code>rectangulo.centerx</code>	El valor int de la coordenada X del centro del rectángulo
<code>rectangulo.centery</code>	El valor int de la coordenada Y del centro del rectángulo
<code>rectangulo.width</code>	El valor int del ancho del rectángulo
<code>rectangulo.height</code>	El valor int de la altura del rectángulo
<code>rectangulo.size</code>	Una tupla de dos enteros: (ancho, alto)
<code>rectangulo.topleft</code>	Una tupla de dos enteros: (izquierda, arriba)
<code>rectangulo.topright</code>	Una tupla de dos enteros: (derecha, arriba)
<code>rectangulo.bottomleft</code>	Una tupla de dos enteros: (izquierda, abajo)
<code>rectangulo.bottomright</code>	Una tupla de dos enteros: (derecha, abajo)
<code>rectangulo.midleft</code>	Una tupla de dos enteros: (izquierda, centro)
<code>rectangulo.midright</code>	Una tupla de dos enteros: (derecha, central)
<code>rectangulo.midtop</code>	Una tupla de dos enteros: (centerx, top)
<code>rectangulo.midbottom</code>	Una tupla de dos enteros: (centrox, abajo)

Desarrollaremos las prácticas de este módulo en la siguiente sección.

2.5. Módulo `pygame.draw` y funciones primitivas para dibujo

Pygame proporciona varias funciones para dibujar diferentes formas en un objeto superficie. Estas formas, son como rectángulos, círculos, elipses,

líneas o píxeles individuales, a menudo se denominan primitivas para dibujo (drawing primitives).

Abra el editor de archivos de IDLE, escriba el siguiente programa y guárdelo como dibujos.py

```
dibujos.py import pygame, sys
from pygame.locals import *
pygame.init()

# Configurando ventana
DISPLAYSURF = pygame.display.set_mode((640, 480), 0, 32)
pygame.display.set_caption('Dibujando Polígonos, Rectas,
                             Rectángulos, Círculos')

# Configurando colores
BLACK = ( 0, 0, 0)
WHITE = (255, 255, 255)
RED = (255, 0, 0,0)
GREEN = ( 0, 255, 0,0)
BLUE = ( 0, 0, 255)
SUAVE = (153,217,235)
AZUL = (153,217,234)

# dibujando el objeto Superficie (surface object)
DISPLAYSURF.fill(SUAVE)

pygame.draw.polygon(DISPLAYSURF, BLACK, ((40, 50), (300, 50),
                                           (300, 300), (160, 440),
                                           (40, 300)), 4)

# Recta
pygame.draw.line(DISPLAYSURF, BLUE, (80, 80), (250, 80), 4)
pygame.draw.line(DISPLAYSURF, WHITE, (250, 80), (80, 300), 2)
pygame.draw.line(DISPLAYSURF, RED, (80, 300), (250, 300), 4)

# Rectas y Polígono
pygame.draw.lines(DISPLAYSURF, GREEN, True, ((440, 60), (470, 240),
                                              (340, 90)), 4)

pygame.draw.circle(DISPLAYSURF, WHITE, (550, 150), 70, 4)
pygame.draw.rect(DISPLAYSURF, BLUE, (320, 260, 200, 60), 2)
pygame.draw.ellipse(DISPLAYSURF, RED, (360, 360, 200, 60), 2)

# Bucle del juego
while True:
    for evento in pygame.event.get():
        if evento.type == QUIT:
            pygame.quit()
            sys.exit()
    pygame.display.update()
```

Al ejecutar este archivo, tendremos una ventana semejante a la siguiente figura 2.2: El juego estará ejecutándose hasta que el usuario cierre la ventana (haciendo clic en X).

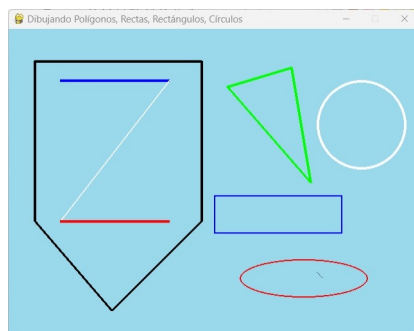


Figura 2.2: Ventana dibujos del módulo `pygame.draw`

Observe que usamos variables constantes para cada uno de los colores. Esto hace que nuestro código sea más legible, porque ver VERDE en el código fuente es mucho más fácil de entender que representar el color verde como una tupla (0, 255, 0).

Las funciones de dibujo reciben el nombre de las formas que dibujan. Los parámetros que pasas estas funciones les dicen en qué parte del objeto Superficie dibujar la forma, el tamaño, el color y qué tan gruesas deben ser las líneas.

Puedes ver cómo se llaman estas funciones en el archivo de programa `dibujos.py`, pero aquí hay una breve descripción de cada función:

- `fill(color)` es un método, no es una función sino un método de `pygame.Surface`. Rellenará completamente todo el objeto Superficie con cualquier valor de color que le pase como parámetro de color (actúa sobre el objeto `DISPLAYSURF.fill(WHITE)`).

- `pygame.draw.polygon (superficie, color, lista de puntos, ancho)`: dibuja un polígono, que es una figura formada por sólo segmentos de rectas (lados planos). Los parámetros de superficie y color le indican a la función en qué superficie dibuja el polígono y de qué color hacerlo. El parámetro lista de puntos es una tupla o lista de puntos (es decir, una tupla o lista de dos números enteros de tuplas para coordenadas XY). El polígono se dibuja trazando líneas entre cada punto, y el punto que viene después en la tupla, luego se traza una línea desde el último punto hasta el primero punto. También puedes pasar una lista de puntos en lugar de una tupla de puntos. El parámetro de ancho es opcional. Si lo omites, el polígono que se dibuje se

rellenará adentro, al igual que nuestro polígono verde en la pantalla se llena de color. Si pasas un número entero para el parámetro ancho, solo se dibujará el contorno del polígono, el número entero representa cuántos píxeles de ancho tendrá el contorno del polígono.

Pasando 1 para el ancho el parámetro creará un polígono delgado, mientras que pasar 4, 10 o 20 lo hará más grueso. Si pasa el número entero 0 para el parámetro de ancho, el polígono se completará (como si omitieras el parámetro de ancho por completo).

Todas las funciones de dibujo de `pygame.draw` tienen parámetros de ancho opcionales al final, y funcionan de la misma manera que el parámetro de ancho de `pygame.draw.polygon()`. Probablemente un mejor nombre para el parámetro ancho hubiera sido espesor (*thickness*), ya que ese El parámetro controla el grosor (*thick*) de las líneas que dibujas.

- `pygame.draw.line(superficie, color, punto_inicial, punto_final, ancho)`: esta función dibuja una línea entre los parámetros `punto_inicial (x,y)`, `punto_final (x,y)`.

- `pygame.draw.lines(superficie, color, cerrado, lista de puntos, ancho)`: esta función dibuja una serie de líneas de un punto al siguiente, muy parecido a `pygame.draw.polygon()`. La única diferencia es que si pasa `False` para el parámetro `cerrado`, no dibuja línea del último punto al primer punto en la lista de puntos del parámetro. Si pasa `True`, entonces dibuja una línea desde el último punto hasta el primero.

- `pygame.draw.circle(superficie, color, punto_centro, radio, ancho)`: esta función dibuja un círculo. El centro del círculo está en el parámetro `center_point`. El número entero pasado por el parámetro de radio establece el tamaño del círculo. El radio de un círculo es la distancia desde el centro hasta el borde. (El radio de un círculo es siempre la mitad del diámetro). Al pasar el valor de 70 para el parámetro de radio se dibujará un círculo que tiene un radio de 70 píxeles.

- `pygame.draw.rect(superficie, color, rectángulo_tupla, ancho)`: esta función dibuja un rectángulo. La `tupla_rectángulo` es una tupla de cuatro números enteros (para las coordenadas XY de la esquina superior izquierda, y el ancho y alto) o se puede pasar un objeto `pygame.Rect` en cambio. Si la `tupla_rectángulo` tiene el mismo tamaño en ancho y alto, se

formará un cuadrado. ser dibujado.

- `pygame.draw.ellipse(superficie, color, rectángulo_limitador, ancho)`: esta función dibuja un elipse (que es como un círculo aplastado o estirado). Esta función tiene todas las características habituales, sin embargo para indicarle a la función dónde dibujar la elipse y el tamaño, debe especifique el rectángulo delimitador de la elipse. Un rectángulo delimitador es el rectángulo más pequeño que se puede dibujar alrededor de una forma. Aquí hay un ejemplo de una elipse y su rectángulo delimitador:

El parámetro `bounding_rectangle` puede ser un objeto `pygame.Rect` o una tupla de cuatro números enteros. Tenga en cuenta que no especifica el punto central de la elipse como lo hace para la función `pygame.draw.circle()`.

Finalizando el juego

- La línea de llama a la función `pygame.display.update()` actualiza el objeto Superficie en la pantalla del monitor del usuario.

Una vez que haya terminado de llamar las funciones de dibujo para que el objeto Superficie de visualización tenga el aspecto deseado, debe llamar a `pygame.display.update()`. Dado que el objeto Superficie no ha cambiado, la imagen es la misma azul claro, se vuelve a dibujar en la pantalla cada vez que se llama a `pygame.display.update()`. Después de terminar la línea, el bucle `while` infinito, comienza nuevamente desde el inicio. Esto es todo el programa. ✓

Lo único que debes recordar es que `pygame.display.update()` solo hará que superficie de visualización aparecen en la pantalla (es decir, el objeto superficie que devolvió al llamada a `pygame.display.set_mode()`). Si quieres que las imágenes de otros objetos Surface aparezcan en la pantalla, debe usar en método `blit()` para copiarlos en el objeto Superficie de visualización. En la sección 2.6.3 se explica con más detalles este método.

2.6. Animación

Ahora que sabemos cómo hacer que Pygame dibuje en la pantalla figuras o dibujos, veamos cómo hacer imágenes animadas en la pantalla o ventana de juego.

Las imágenes animadas son el resultado de dibujar una imagen en la pantalla, y una fracción de segundo después dibujar una segunda imagen ligeramente diferente a la primera en la pantalla, que aparente un movimiento.

Considere las siguientes tres imágenes en la figura 2.3, y suponga que se muestran rápidamente una después de la otra en la pantalla, para el usuario,

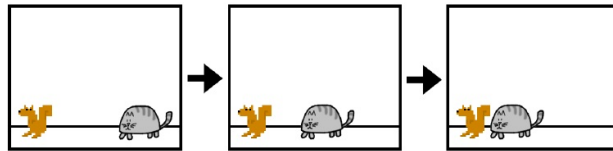


Figura 2.3: La ardilla mueve hacia el gato

parecería que el gato se mueve hacia la ardilla. Es solo ilusión visual. Para la computadora, son sólo un montón de píxeles.

2.6.1. Nuestra primera animación

A continuación prestamos un programa de ejemplo que muestra una animación simple. Se trata en una imagen de un perro que aparenta moverse en la pantalla.

Escriba este código en su editor y guárdelo como `animacion_perro.py`, también necesita que los dos archivos de imagen `perro.png` y `perro_regresa.png` se encuentren en la misma carpeta que el programa.

```
animacion
_perro.py  # -*- coding: utf-8 -*-
"""
Created on Mon Apr  3 17:07:13 2023
@author: Linis
"""
# El perro regresa

import pygame, sys
from pygame.locals import *
pygame.init()

# Configuración de Ventana
DISPLAYSURF = pygame.display.set_mode((450, 370), 0, 32)
pygame.display.set_caption('Animaciones y algo mas')

FPS = 30 # Configurando fotograma por segundos
fpsReloj = pygame.time.Clock() # velocidad de FPS

# Configuración de colores
WHITE = (255,255,255)
```

```

CELESTE = (153,217,235)
RED = (225,0,0)
GREEN = (0,255,0,180)

# Cargando imagenes y posición inicial
perro = pygame.image.load('perro.png') # Tamaño 85x76
perrox = 20
perroy = 20
direction = 'derecho'
perroR = pygame.image.load('perro_regresa.png')
perroRx = 365
perroRy = 20

while True: # Bucle principal del juego
    DISPLAYSURF.fill(WHITE)

    if direction == 'derecho':
        perrox += 5
        if perrox == 365:
            direction = 'izquierdo'
            perroRx = 365

    if direction == 'izquierdo':
        DISPLAYSURF.blit(perroR, (perroRx, perroRy))
        if direction == 'izquierdo':
            perroRx -= 5
            if perroRx == 10:
                direction = 'derecho'
                perrox = perroRx
    else:
        DISPLAYSURF.blit(perro, (perrox-5, perroy-5))

    for evento in pygame.event.get():
        if evento.type == QUIT:
            pygame.quit()
            sys.exit()

    pygame.draw.line(DISPLAYSURF, GREEN, (10, 90), (440, 90), 4)
    pygame.draw.circle(DISPLAYSURF, CELESTE, (300, 50), 40, 4)
    pygame.draw.ellipse(DISPLAYSURF, RED, (20, 15, 200, 73), 2)

    pygame.display.update()
    fpsReloj.tick(FPS)

```

Aunque no es el mejor ejemplo de animación, se observa que el perro se mueve en la pantalla, simulando movimiento.

Al ejecutar este archivo, tendremos una ventana semejante a la siguiente en la figura 2.4: A continuación describimos en programa en detalle:

- En la línea 7 se define una constante igual a 30 FPS, es la configuración de fotogramas por segundo (FPS), en la próxima sección se da una referencia teórica básica al respecto.

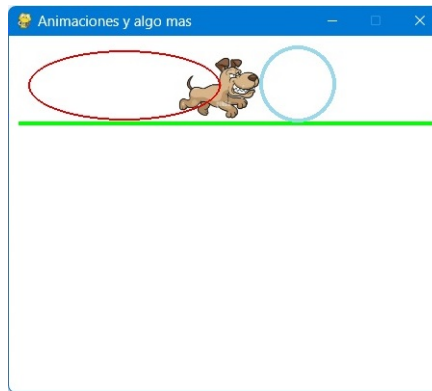


Figura 2.4: Simulando movimiento del perro

2.6.2. Velocidad de Fotogramas

La velocidad de fotogramas o frecuencia de actualización es la cantidad de imágenes que el programa dibuja por segundo, y se mide en fotogramas por segundo (FPS). En los monitores de computadora el nombre común para FPS es hertz, muchos monitores tienen una velocidad 60 hertz o 60 fotogramas por segundo. Una velocidad de fotogramas baja en videojuegos pueden hacer que el juego parezca entrecortado o nervioso.

Si el programa tiene demasiado código para ejecutar dibujos en pantalla con suficiente frecuencia, entonces el FPS baja. Pero los juegos de este libro son lo suficientemente simple como para que esto no sea un problema ni siquiera en computadoras antiguas.

- En la siguiente línea tenemos un objeto del módulo `pygame.time`, `fpsReloj = pygame.time.Clock()`; un objeto con la finalidad de hacer que nuestro programa se ejecute en un máximo de FPS. Este objeto `fpsReloj` asegura que nuestros programas de juego no se ejecuten demasiado rápido al instalar pequeñas pausas en cada iteración del bucle del juego.

Si no tuviéramos estas pausas, nuestro juego (programa) se ejecutaría tan rápido como la computadora pudiera ejecutarlo. Esto suele ser demasiado rápido para el jugador y a medida que las computadoras se vuelvan más rápidas, también ejecutarán el juego más rápido.

Una llamada al método `tick(FPS)` del objeto `fpsReloj` en el bucle del juego garantiza que el juego se ejecute a la misma velocidad sin importar qué tan rápido sea la computadora en la que se ejecuta.

El método `fpsReloj.tick(FPS)` debe llamarse al final del ciclo del juego, después de llamar a `pygame.display.update()`. La duración de la pausa se calcula en función de cuánto tiempo ha pasado desde la llamada anterior a `tick()`, que habría tenido lugar al final de la iteración anterior del bucle del juego. La primera vez que se llama al método `tick()`, no hay pausar en absoluto.

Intente modificar la variable constante `FPS` para ejecutar el mismo programa a diferentes velocidades de fotogramas. Configurarlo con un valor más bajo haría que el programa se ejecutara más lento, y a un valor más alto haría el programa se ejecuta más rápido.

- Luego tenemos las líneas de código: `pygame.image.load("Imagen.png")`, esta función carga los archivos `perro.png` y `perro_regreresa.png` (formato de imagen), y devuelve un objeto `Surface` al llamarlo.

2.6.3. Dibujar y cargar imágenes con `pygame.image.load()` y `blit()`

Para cargar las imágenes se le pasa la cadena `"perro.png"` a la función `pygame.image.load()`, luego al llamado de la función `pygame.image.load()` devolverá un objeto `Surface` con la imagen dibujada.

Este objeto `Surface` será un objeto separado del objeto `Surface` de visualización, por lo que el objeto `Surface` de la imagen debe ser `blit` (es decir, copiar) sobre el objeto `Surface` de visualización (display `Surface` object).

- Ahora en el cuerpo del `while`, tenemos la línea de código `DISPLAYSURF.blit(perro, (perrox, perroy))` un método del objeto `DISPLAYSURF`. El método `blit()` se usa para copiar el objeto `Surface` de la imagen en el objeto `Surface` de visualización, `perro` en `DISPLAYSURF`. Blitting consiste en dibujar el contenido de una superficie sobre otra (copiar una imagen en una nueva ubicación).

El método `DISPLAYSURF.blit(perro, (perrox, perroy))`, tiene dos parámetros, el primero es el objeto `Superficie` de la imagen, que se debe copiar en el objeto `Surface` `DISPLAYSURF`. El segundo parámetro es una tupla de dos números enteros para los valores `X` e `Y` de la esquina superior izquierda donde se debe copiar la imagen.

Si `perrox` y `perroy` estuvieran configurados en 100 y 200, y el ancho de la imagen fuera 125 y la altura era 79, esta llamada `blit()` copiaría esta

imagen en `DISPLAYSURF` de tal forma que la esquina superior izquierda del objeto queda con coordenada XY (100, 200) y la esquina inferior derecha con coordenada XY (225, 279).

El resto del ciclo del juego consiste simplemente en cambiar las variables `perrox`, `perroy`, la dirección para que el perro se mueve en línea horizontal en la ventana a la altura de la coordenada Y 20, y retorne el perro con el objeto `perroR` de la segunda imagen.

Se agregaron tres dibujos, una línea recta color verde, un círculo y una elipse, opcional. Y el código finaliza con las dos líneas de código:

```
pygame.display.update()
fpsReloj.tick(FPS)
```

Esto es todo el programa, ✓

Puede descargar el archivo movimientos de formas, para ver movimiento de dibujos círculos y elipses...

2.6.4. Crear texto con el módulo `pygame.font`

Pygame tiene un modulo para gestionar fuentes `pygame.font`, este módulo proporciona otras funciones además de objetos de fuente, que ocasionalmente puede necesitar. Es posible que lo uses para mostrar instrucciones del juego, opciones de menú, etc.

El módulo `pygame.font`, usa el tipo de fuente TrueType fonts (TTFs), que es usado en mayoría de los sistemas para representar texto fluido y de alta calidad. Hay muchas fuentes de este tipo instalado en su computadora que puede ser utilizado por el módulo de `font`.

En `pygame` se necesitan seis pasos para hacer que aparezca un texto en la pantalla con el módulo `pygame.font`, a continuación los describimos e ilustramos con el texto `texto = ";Cuidado!"` perro bravo:

1. Crear un objeto fuente en `pygame` con `pygame.font`. Ejemplo: `fuente = pygame.font.Font('freesansbold.ttf',40)` (o `fuente = pygame.font.SysFont("arial", 50)` una fuente común que es fácil de leer, Pygame buscará una fuente con el nombre "arial" en las fuentes instaladas; si no la encuentra, se devolverá una fuente predeterminada).

Los parámetros de la función constructora `pygame.font.Font()` son dos, el nombre de la fuente que desea crear (tipo font), y el tamaño de la fuente (medido en píxeles, como los procesadores lo miden tamaño de la fuente).

`freesansbold.ttf` es una fuente que viene con Pygame (archivo de fuente TrueType extensión `.ttf`). Y para `pygame.font.SysFont` puede utilizar cualquiera de las fuentes que tienes instaladas en tu computadora. Puede obtener una lista de las fuentes instaladas en su sistema llamando a `pygame.font.get_fonts()`. Las fuentes también se pueden crear directamente a partir de archivos `.ttf` mediante llamando a `pygame.font.Font()`, que toma un nombre de archivo.

Una vez que haya creado un objeto fuente, puede usarlo para construir un objeto Surface del texto.

2. Crear un objeto Surface del texto con el método `render()`. Ejemplo: `textoSurface = fuente.render(";Cuidado! perro bravo", True, YELLOW)` (el objeto superficie que contiene el texto).

Los parámetros del método `render()` son la cadena texto a renderizar (representar), un valor booleano para especificar si queremos suavizado², el color del texto y el color del fondo. El fondo es opcional, si desea un fondo transparente, simplemente omita el parámetro de color de fondo en la llamada al método.

3. Crear un objeto Rect a partir del objeto Surface del texto, mediante el método `get_rect()`. Ejemplo: `textoRect = textoSurface.get_rect()`.

Este objeto Rect tendrá el ancho y alto configurado correctamente para el texto que se representó, pero los atributos para las coordenadas del rectángulo de la esquina superior izquierdo serán (0,0).

4. Establecer la posición del objeto Rect cambiando uno de sus atributos. Por ejemplo, configuramos el centro del objeto `textoRect.center = (225, 150)`.

5. Ahora solo hace falta copiar (Blit) el objeto Surface del texto en el objeto Superficie de visualización devuelto por `pygame.display.set_mode()`. Ejemplo: `DISPLAYSURF.blit(textoSurface, textoRect)`

6. Finalmente se llamar a `pygame.display.update()` para actualizar la su-

²El suavizado es una técnica gráfica para hacer que el texto y las formas parezcan menos bloques, se logra haciendo un poco borroso los bordes. Se necesita un poco más de tiempo de cálculo para dibujar con suavizado, aunque los gráficos se vean mejor, su programa puede funcionar más lento (pero sólo un poco). Para hacer que el texto de Pygame use anti-aliasing, simplemente se pase True al segundo parámetro del método `render()`.

perficie de visualización aparecerá la pantalla.

Todos estos pasos y códigos los agregamos al archivo `animacion_perro.py`:

```
animacion
_perro.py
recorte
.
texto = '¡Cuidado! Perro bravo'
fuente = pygame.font.SysFont("arial",50)
textoSurface = fuente.render(texto, True,YELLOW) # omitir el color de fondo
textoRect = textoSurface.get_rect()
textoRect.center = (225, 150)
# salvar archivo de imagen de textoSurface en formato .pnf
pygame.image.save(textoSurface, "textoSurface.png")

while True: # Bucle principal del juego
    DISPLAYSURF.fill(WHITE)
    DISPLAYSURF.blit(textoSurface, textoRect)
    .
    recorte
    .
    pygame.display.update()
    fpsReloj.tick(FPS)
```

Al ejecutar el archivo se obtendrá una ventana de juego semejante a la siguiente figura 2.5:



Figura 2.5: Ventana agregando texto con módulo `font.SysFont()`

En este archivo se agregó una línea de código:

```
pygame.image.save(textoSurface, "textoSurface.png")
```

antes del bucle del `while`, este código permite guardar el archivo en formato PNF en el directorio en que se encuentra el programa, y además se puede ver con cualquier visualizador en su computador, verifique. Descarga de archivo en

Para obtener detalles completos sobre el módulo de fuente, consulte la documentación en www.pygame.org/docs/ref/font.html.



Las fuentes instaladas varían de una computadora a otra y no siempre es posible confiar en una fuente en particular. Si Pygame no encuentra la fuente que ha configurado, usará una fuente predeterminada, y tal vez puede verse un poco diferente. La solución es distribuir el juego con archivos extensión .ttf, pero asegúrate de tener permiso o que sean libres.

2.6.5. Reproducir sonidos

Ya hemos recorrido una buena parte de los conceptos básicos de Pygame, ahora estudiaremos como reproducir sonidos a partir de archivos de sonido almacenados; pero no se preocupe, este trabajo es muy fácil incluso es más sencillo que mostrar imágenes a partir de archivos de imágenes.

- Primero, debes crear un objeto de sonido con la función constructora `pygame.mixer.Sound()`. Pygame puede cargar archivos WAV, MP3 u OGG. Ejemplo: `sonidoObj = pygame.mixer.Sound("perro_ladra.wav")` el parámetro de esta función es una cadena, el nombre del archivo de sonido.

- Segundo, reproducir el sonido del objeto de sonido, esto se logra llamado al método `play()`. Ejemplo: `sonidoObj.play()`. La ejecución del programa continúa inmediatamente después de llamar a `play()`. El sonido se reproduce inmediatamente, no espera el pasar a la siguiente línea de código.

- Tercero, para detener el objeto de sonido que se reproduce se llama al método `stop()`. Ejemplo: `sonidoObj.stop()` Este método no tiene argumentos.

Aquí hay un código de muestra:

```
sonido.py import pygame
pygame.init()
sonidoObj = pygame.mixer.Sound("perro_ladra.wav") #
sonidoObj.play()
import time
time.sleep(3) # Espera durante 3 segundo y deja que el sonido se reproduzca
sonidoObj.stop()
```

El método `time.sleep()` espera y permite la ejecución del programa durante un número específico de segundos dados.

2.6.5.1. Sonido o música de fondo

Los objetos de sonido son buenos para que se reproduzcan efectos de sonido cuando el jugador recibe premio, corta un espada, o compra o colecciona una moneda. Pero los juegos podrían ser mejores si tuvieran música de fondo, independientemente de lo que estuviera pasando en el juego.

Pygame solo puede cargar un archivo de música para reproducir en segundo plano a la vez.

La función `pygame.mixer.music.load()` se usa para cargar un archivo de música de fondo, y le pasa una cadena como argumento, el nombre del archivo de sonido a carga. Igualmente este archivo puede tener formato WAV, MP3 o MIDI.

- Primero, para comenzar a reproducir el archivo de sonido cargado como música de fondo, se llame a la función `pygame.mixer.music.play(-1, 0.0)`. El argumento -1 hace que la música de fondo se repita siempre cuando llega al final del archivo de sonido. Si lo configuras como un número entero 0 o mayor, entonces la música solo se repetirá esa cantidad de veces en lugar de repetirse para siempre.

- El segundo argumento 0.0 significa tiempo para iniciar a reproducirse el sonido. Si pasa un número entero más grande o flotante, la música comenzará a reproducirse después de ese número de segundos. Por ejemplo, si pasas 13.5 para el segundo parámetro, el archivo de sonido comenzará a reproducirse en el punto 13.5 segundos en desde el principio.

- Para detener la reproducción de la música de fondo inmediatamente, llame a `pygame.mixer.music.stop()` función. Esta función no tiene argumentos.

Aquí hay un código de ejemplo de los métodos y funciones de sonido:

```
musica
_fondo.py    import pygame, sys
              pygame.init()

              # Cargando y reproduciendo un efecto de sonido:
              sonidoObj = pygame.mixer.Sound("perro_ladra.wav")
              sonidoObj.play()
              import time
              time.sleep(2) # espera y deja que el sonido se reproduzca durante 2 segundo
              sonidoObj.stop()
```

Pero para continuar con la ilustración didáctica, agregamos este código al archivo `animacion_perro.py`:

```

animacion
_perro.py
    .
    recorte
    .
    textoRect = textoSurface.get_rect()
    textoRect.center = (225, 150)

    # Configuración de Sonido

    pygame.mixer.music.load("perro_ladra.wav")
    pygame.mixer.music.play(-1, 0.0)

    while True: # Bucle principal del juego
        DISPLAYSURF.fill(WHITE)
        .
        recorte
        .
        pygame.display.update()
        fpsReloj.tick(FPS)

```

Al ejecutar obtendrás un juego igual al que se muestra en la página 36, pero ahora escuchará ladrar el perro. ✓

Descargar archivo en proyecto...

2.7. Módulo `pygame.event` y Cola de eventos

En Pygame el método `pygame.event` nos facilita todos los eventos que han ocurrido en una cola de eventos. Esta cola se puede recorrer con un `for in`, para recuperar los eventos de `pygame.event.get()`, luego procesar o ignorar estos eventos. Una vez que un evento se procesa o se ignora, se elimina de la cola.

En los juegos (programas) que hemos tratado hasta ahora solo hemos manejamos el evento "QUIT" (SALIR), que es esencial a menos que quieras tener ventanas Pygame que no cierran, ventanas inmortales!

Pygame crea otros eventos para informarle de cosas que suceden, como el movimiento del ratón y pulsaciones de teclas. Eventos que se pueden generar en cualquier momento, sin importar el momento actual de ejecución del programa.

Por ejemplo, en un juego de combate, el código podría dibujar un tanque en la pantalla, al mismo tiempo que el usuario presiona un botón ejecutar cierta acción, digamos un disparo. Debido a que no puedes reaccionar a los eventos en el instante en que suceden, Pygame almacena en una cola hasta que esté listo para manejarlos (normalmente al comienzo de la página principal bucle de juego).

Puedes idearlo se puede imaginar una fila de personas esperando para entrar a un edificio, cada uno de ellos con una información específica sobre un evento. Cuando el jugador presiona el botón de disparo, llega el evento del joystick, que lleva información sobre qué tecla se presionó. De manera similar, cuando el jugador suelta el botón de disparo, llega un clon del mismo evento del joystick con información sobre el botón que se soltó. Podrían ir seguidos de un evento de mouse y un evento del teclado. En la próxima 2.7.1 se trata la recuperación de estos eventos con detalles.

2.7.1. Recuperar eventos

El módulo `pygame.event` almacena los evento que suceden fuera de su código; recordemos que los eventos son generados por muchas situaciones externas al código, estas van desde presionar teclas, mover el mouse, hasta recibir información de internet; estos se van organizando en cola hasta que el usuario los maneje.

La función `pygame.event.get()` devuelve cualquier evento ocurrido o esperado. Para recuperar todos o cualquier eventos se usa un bucle `for`. En los programas anteriores sólo se esperaba el evento `QUIT`, que es generado por Pygame cuando el usuario hace clic en el botón cerrar en la ventana de Pygame. Entonces, si el tipo de evento es `QUIT`, llamamos a `pygame.quit()` para cerrar Pygame, y todos los demás eventos son ignorados, inmediatamente se cierra ventana con `sys.exit()`.

Además, se llamó a `pygame.event.get()` para recuperar todos los eventos y sacarlos de la cola, lo cual es como abrir la puerta y dejar entrar a todos. Esto probablemente sea la mejor manera de lidiar con los eventos, ya que garantiza que hayamos manejado todo antes de pasar a dibujar algo en la pantalla, pero hay otras formas de trabajar con la cola de eventos.

Pygame tiene una función que hace esperar la ocurrencia del próximo evento antes de regresar, se trata de `pygame.event.wait()`. Para ilustrar su funcionamiento, puede imaginarse esperado en la puerta hasta que llegue alguien. Generalmente esta función no se utiliza casi en juegos, debido a que si usted llama a `pygame.event.wait()`, Pygame pausa el `script` hasta que suceda algo, pero puede ser útil para aplicaciones Pygame donde coopere con otros programas en su sistema, como reproductores multimedia.

Una alternativa es la función `pygame.event.poll()`, que devuelve un único evento si hay uno en espera, o un evento ficticio (dummy) `NOEVENT`

si no hay eventos en la cola. Cualquiera que sea el método que utilice, es importante no permitir que se acumulen, porque la cola de eventos tiene un tamaño limitado y los eventos se perderán si la cola se desborda (overflows).

Es necesario llamar al menos a una de las funciones de manejo de eventos a intervalos regulares para que Pygame puede procesar eventos internamente.

Si no utiliza ninguna de las funciones de manejo de eventos, puedes llamar a la función `pygame.event.pump()` en lugar de un bucle de eventos.

Estos objetos de evento contienen pocas variables miembro que describen el evento que ocurrió. La información que contienen varía dependiendo del evento. Lo único común en todo objetos evento es el tipo, que es un valor que indica el tipo de evento; y es el primero valor que se consulta para que puedas decidir qué hacer con ella.

La Tabla 2.7.1 enumera algunos eventos estándar que puede recibir.

Table 2.7.1. Eventos estándar:

Evento	Propósito	Parámetros
QUIT (SALIR)	El usuario ha hecho clic en el botón cerrar	no tiene
ACTIVEEVENT	Pygame ha sido a activado u oculto	gain, state (ganancia, estado)
KEYDOWN	Se ha presionado la tecla	unicode, clave, mod
KEYUP	La tecla KEYUP ha sido liberada	key, mod (clave, modo)
MOUSEMOTION	Se ha movido el ratón	pos, rel = distancia , botones
MOUSEBUTTONDOWN	Se presionó el botón del mouse	pos, buttons (posición, botón)
MOUSEBUTTONUP	Se libero el botón del mouse	pos, button (posición, botón)
JOYAXISMOTION	Se movió el joystick o pad	joy, axis = eje, valor
JOYBALLMOTION	La bola de Joy se movió	joy, ball = esférica, rel
JOYHATMOTION	Se movió el hat del joystick	joy, hat , valor
JOYBUTTONDOWN	Se presionó el botón del joystick o pad	joy, button (botón)
JOYBUTTONUP	Se libero el botón del joystick o pad	joy, button (botón)
VIDEORESIZE	Se cambió el tamaño de la ventana de Pygame	tamaño, ancho, alto
VIDEOEXPOSE	Parte o toda la ventana de Pygame quedó expuesta	no tiene
USEREVENT	Se ha producido un evento de usuario	código

Para ilustrar y ver todos los eventos que se generan, considere el siguiente script `pygame_eventos.py` de Pygame usa `pygame.event.get()` para recuperar los evento.

```
pygame_    from pygame.locals import *
eventos.py from sys import exit

pygame.init()
tamañoVentana = (450, 370)
#CELESTE = (153,217,235)
CELESTE = (170,238,187)
screen = pygame.display.set_mode(tamañoVentana, 0, 32)

while True:
    screen.fill(CELESTE)
    for evento in pygame.event.get():
        print(evento)
        if evento.type == QUIT:
            pygame.quit()
            exit()
    pygame.display.update()
```

Al ejecutar archivo `pygame_eventos.py`, verá una ventana color azul celeste simple. Al mover el ratón sobre la ventana pygame, MOUSEMOTION comienza a transmitir el movimiento del mouse, que se crean cada vez que el mouse cambia de posición (ver Figura 2.6).

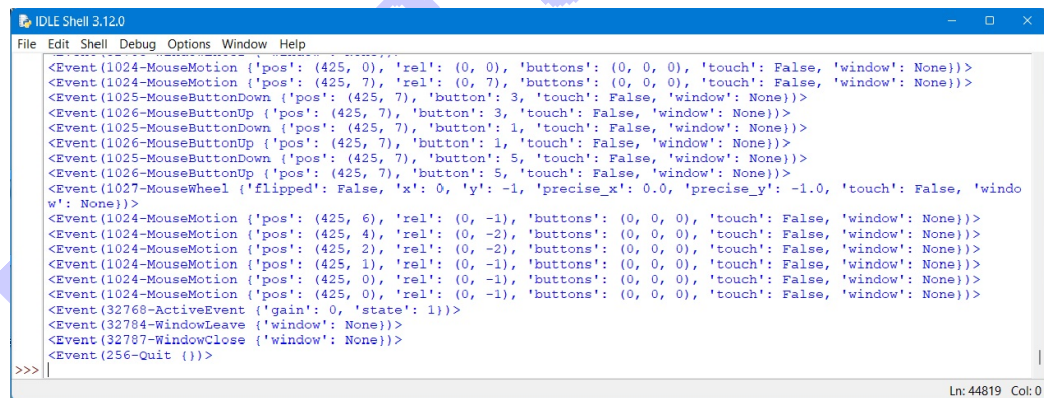


Figura 2.6: Eventos del mouse

Estos eventos especifican la posición actual del mouse, qué tan lejos se ha desplazado el mouse desde el último evento de movimiento y qué botones están presionados actualmente.

Puedes obtener la posición actual del mouse con el módulo `pygame.mouse`, pero corres el riesgo de perder información sobre lo que ha estado haciendo

el jugador. Este es un problema particular en computadoras de escritorio que realizan mucho trabajo en segundo plano y ocasionalmente pueden pausar el juego por un breve período de tiempo.

Para un cursor de mouse, sólo necesita saber dónde está el mouse al comienzo de cada cuadro, por lo que es razonable usar `pygame.mouse.get_pos()`. Si estuvieras usando el movimiento del mouse para conducir un tanque y los botones para disparar, sería mejor trabajar con eventos para que el juego pueda monitorear más de cerca lo que el jugador ha estado haciendo.

2.7.2. Eventos de movimiento del mouse

Como ha visto, los eventos `MOUSEMOTION` se emiten cada vez que mueve el mouse sobre la ventana de Pygame. Contienen estos tres valores:

- `pos`: una tupla que contiene la posición del mouse cuando se generó el evento.
- `rel`: una tupla que contiene la distancia que se ha movido el mouse desde el último evento de movimiento del mouse (a veces es llamado `mouse mickies`).
- `buttons` (botones): una tupla de tres números que corresponden a los botones del mouse. Entonces `buttons[0]` es el botón izquierdo del mouse, `buttons[1]` es el botón central y `buttons[2]` es el botón derecho. Si se presiona el botón, su valor se establece en 1; si no se presiona, el valor será 0. Se pueden presionar varios botones a la vez.

2.7.3. Tipos de eventos del mouse

Además de los eventos `MOUSEMOTION`, el mouse genera dos tipos de eventos más `MOUSEBUTTONDOWN` y `MOUSEBUTTONUP`. Si hace clic en el mouse, primero verá en el script de la cola de mensajes, el evento inactivo (down event), seguido de un evento suelta (up event) cuando quita el dedo del botón. Entonces, ¿por qué tienen los dos eventos?; si usted usa el botón del mouse como disparador para disparar un cohete, solo necesitarías uno de los eventos, pero es posible que tengas un tipo diferente de arma, como una pistola automática que dispara continuamente mientras el se mantiene pulsado el botón. En este caso, comenzarías acelerar la pistola automática con evento presionar y disparar hasta que obtengas el evento liberar.

Ambos tipos de eventos del botón del mouse tienen los siguientes dos valores:

- **buttons:** el número del botón que se presionó. Un valor de 1 indica si presionó el botón de la izquierda del mouse, un valor 2 indica que se presionó el botón central y 3 indica que presionó el botón derecho.
- **pos:** una tupla que contiene la posición del mouse cuando se generó el evento.

2.7.4. Manejo de eventos de teclado

El teclado y el joystick tienen eventos similares a los de KEYDOWN y KEYUP, que se transmiten cuando se pulsa una tecla (down) y cuando se suelta (up) la tecla.

En el siguiente archivo se ilustra el proceso cómo podría responder a los eventos KEYDOWN y KEYUP para mover algo en la pantalla con las teclas del cursor.

2.7.4.1. Usar eventos de teclado para mover un fondo

Al ejecutar este archivo, verá una ventana que contiene una imagen de fondo simple. Presiona arriba, abajo, izquierda o derecha y el fondo se deslizará en esa dirección una unidad.

```
movimientos      archivo_imagen_fondo = 'pygame.jpg'
_fondo.py

import pygame
from pygame.locals import *
from sys import exit
pygame.init()

#CELESTE = (153,217,235)    # Color azul Celeste
CELESTE = (170,238,187)
pantalla = pygame.display.set_mode((640, 480), 0, 32)
fondo = pygame.image.load(archivo_imagen_fondo).convert()
x, y = 0, 0
mover_x, mover_y = 0, 0

while True:
    for evento in pygame.event.get():
        if evento.type == QUIT:
            pygame.quit()
            exit()
        if evento.type == KEYDOWN:
            if evento.key == K_LEFT:
                mover_x = -1
            elif evento.key == K_RIGHT:
```

```

        mover_x = +1
    elif evento.key == K_UP:
        mover_y = -1
    elif evento.key == K_DOWN:
        mover_y = +1

elif evento.type == KEYUP:
    if evento.key == K_LEFT:
        mover_x = 0
    elif evento.key == K_RIGHT:
        mover_x = 0
    elif evento.key == K_UP:
        mover_y = 0
    elif evento.key == K_DOWN:
        mover_y = 0

x+= mover_x
y+= mover_y
pantalla.fill(CELESTE)
pantalla.blit(fondo, (x, y))

pygame.display.update()

```

El programa empieza igual que los anteriores juegos, importa, inicializa Pygame, y carga un imagen de fondo. El bucle de eventos en este `script` es diferente porque maneja los eventos `KEYDOWN` y `KEYUP`.

Estos eventos de teclas contienen los mismos tres valores:

- `key` (Tecla): este es un número que representa la tecla que se presionó o liberó. Cada tecla (física) del teclado tiene una constante que comienza con `K_`. Las teclas del alfabeto son `K_a` hasta la `K_z`, pero también hay constantes para todas las demás teclas (`key`), como `K_SPACE` y `K_RETURN`. Para obtener una lista completa de las constantes del teclado que puede consultar en <https://www.pygame.org/docs/ref/key.html>

- `mod`: este valor representa las teclas que se utilizan en combinación con otras teclas, como `SHIFT` (Mayúscula), `Alt` y `Ctrl`. Cada una de estas teclas modificadoras está representada por una constante que comienza con `KMOD_`, como `KMOD_SHIFT`, `KMOD_ALT` y `KMOD_CTRL`. La información del modificador se puede decodificar usando el operador bitwise `AND` (bit a bit). Por ejemplo, `mod & KMOD_CTRL` se evaluará como `True` si la tecla `Ctrl` se presiona. Puede encontrar una lista completa de las teclas modificadoras en www.pygame.org/docs/ref/key.html.

La información del modificador se puede decodificar usando un operador

AND (excepto KMOD_NONE, que debe compararse usando iguales ==). Por ejemplo:

- Unicode: este es el valor Unicode de la tecla que se presionó. Se produce combinando la tecla presionada con cualquiera de las teclas modificadoras que se presionaron. Hay un valor Unicode para cada símbolo en el alfabeto inglés y otros idiomas.

Generalmente no usarás estos valores en un juego, porque las teclas tienden a usarse más como interruptores que para ingresar texto. Una excepción sería ingresar a una tabla de puntuación, donde se quiere que el jugador escriba letras que no estén en inglés, así como mezclar mayúsculas y minúsculas.

Dentro del controlador de KEYDOWN verificamos las cuatro constantes de teclas que corresponden al cursor. Si se presiona K_LEFT, entonces el valor de `mover_x` se establece en `-1`; si se presiona K_RIGHT, está establecido en `+1`. Este valor se agrega luego a la coordenada `x` del fondo para moverlo hacia la izquierda o hacia la derecha. También hay un valor `mover_y`, que se establece si se presiona K_UP o K_DOWN, que se moverá el fondo verticalmente.

También es necesario manejar el evento KEYUP, porque queremos que el fondo deje de moverse cuando el usuario suelta la tecla del cursor. El código dentro del controlador para eventos KEYUP es similar al `down`, pero establece `mover_x` o `mover_y` nuevamente a cero para evitar que el fondo se mueva.

Después del ciclo de eventos, todo lo que tenemos que hacer es agregar los valores `mover_x` y `mover_y` a `x` e `y`, luego dibuja el fondo en `(x, y)`. Y se usa `fondo.fill(CELESTE)`, para borrar la pantalla y dejarla en azul celeste. Esta línea es necesaria porque si movemos la imagen de fondo ya no cubre toda la pantalla, lo cual supongo que técnicamente significaría que ya no es un fondo.

2.7.4.2. Bloquear y permitir eventos

En muchas ocasiones no es necesario gestionar todos los eventos en el juegos, y es sano suspender los eventos que no se necesitan. Existen formas alternativas gestionar eventos. Por ejemplo, si estás usando `pygame.mouse.get_pos()`, no necesitará al evento MOUSEMOTION, con la función `pygame.event.set_blocked(pygame.MOUSEMOTION)` o `pygame.event.set_blocked([MOUSEMOTION])` se bloquea.

- `set_blocked()`, si pasas un evento de pygame o una lista de eventos, todos esos eventos se bloquearán. Si le pasa `None` como entrada, todos los eventos de la cola se bloquearán.

Por ejemplo, la siguiente línea deshabilitará todas las entradas del teclado, bloqueando los eventos `KEYDOWN` y `KEYUP`:

```
pygame.event.set_blocked([KEYDOWN, KEYUP])
```

- `set_allowed()`, hace lo contrario que el método anterior, desbloquear los eventos. Requiere un único tipo de evento o una lista de tipos de eventos pygame en la cola que se le permitirá. Además si le pasa `None`, permitirán todos los eventos.

Ejemplo: `pygame.event.set_allowed(pygame.QUIT)`.

- `get_blocked()`, con esta función puedes preguntarle a Pygame si un evento o una lista de eventos están bloqueados. Devolverá un valor booleano (`True` o `False`) si un evento está bloqueado.

Ejemplo: `print(pygame.event.get_blocked([KEYUP, QUIT]))` o `print(pygame.event.get_blocked(pygame.QUIT))`.

2.7.5. Eventos personalizados con `pygame.event.Event`

Generalmente es Pygame el que crea todos los eventos, pero puedes crear sus propios eventos. Podría usar esta capacidad para reproducir demostraciones (replicando la entrada del reproductor) o simular los efectos de un gato caminando sobre el teclado.

Para enviar un evento a la cola, primero construye un objeto de evento con `pygame.event.Event` y luego se enlista con `pygame.event.post`. El evento se colocará al final de la cola listo para ser recuperado en el bucle de eventos.

A continuación se ilustra cómo simular al jugador presionando la barra espaciadora:

```
mi_evento = pygame.event.Event(KEYDOWN, tecla = K_SPACE, mod=0, unicode=u' ')
pygame.event.post(mi_evento)
```

El constructor de eventos toma el tipo de evento, como uno de los eventos en la Tabla 2.7.1, seguido de los valores que debe contener el evento. Como estamos simulando el evento `KEYDOWN`, debe proporcionar todos los valores (parámetros) que el controlador de eventos esperaría que estuvieran allí. Si lo prefieres, puede proporcionar estos valores como un diccionario.

Esta línea creará el mismo objeto de evento:


```
mi_evento = pygame.event.Event(KEYDOWN, {"key": K_SPACE,
                                           "mod": 0, "unicode": ' '})
```

Además de simular eventos generados por Pygame, puedes crear eventos completamente nuevos. Todo lo que tienes que hacer es usar un valor por encima de `USEREVENT` para el evento. `USEREVENT` es el valor máximo que Pygame utilizará para sus propios eventos IDs.

A veces esto puede resultar útil si quiere hacer un bucle de eventos antes de pasar a dibujar en la pantalla. Aquí hay un ejemplo de un evento personalizado para responder a una acción de teclado:

```
mi_evento = pygame.event.Event(Accion_Teclado,
                               mensaje= ";Acción de teclado!")
pygame.event.post(mi_evento)
```

El manejo de eventos de usuario se realiza de la misma manera que los eventos habituales que genera Pygame, simplemente verifique el tipo de evento para ver si coincide con su evento personalizado. Así es como podrías manejar un evento `Acción_Teclado`:

```
for event in pygame.event.get():
    if event.type == Acción_Teclado:
        print(event.mensaje)
```

2.8. Opciones para pantalla

La resolución de pantalla es una decisión que también depende de cuánta acción tendrás en el juego, cuántos objetos estarán moviéndose en pantalla al mismo tiempo, cuanto mas objetos más lento será la ejecución del juego. Es posible que tengas que compensar seleccionando una resolución más baja que permita acelerar las cosas para lograr que funcionen como se quiere.

Generalmente es más fácil utilizar una resolución de pantalla fija (tamaño de pantalla) porque puede simplificar el código. Sin embargo la mejor solución podría ser dejar que el jugador decida en qué resolución quiere ejecutar, de modo que pueda ajustar la pantalla hasta que tengan una calidad aceptable entre lo visual y la fluidez con la que se ejecuta el juego. Si sigues esta ruta, tendrás que asegurarte de que tu juego se vea bien en todas las resoluciones potenciales. Pero este trabajo no se inicia hasta que llegue el momento de escribir tu juego. Por ahora simplemente seleccione una resolución que funcione mientras experimentas.

2.8.1. Pantalla completa

El primer parámetro de la ventana a crear es el tamaño. Un tamaño de 640x480 píxeles crea una pequeña ventana que cabe cómodamente en la mayoría de las computadoras de escritorios, y para ejecutar y depurar el juego en esa ventana es excelente, sin embargo la mayoría de los juegos ocupan toda la pantalla, y no tienen los bordes ni la barra de título habituales.



Si algo sale mal con el `script` con modo de pantalla completa, a veces puede resultar difícil solucionarlo, vuelve a su escritorio. Es mejor probarlo primero en modo ventana. También debe proporcionar una forma alternativa de salir del script porque el botón de cerrar no está visible en el modo de pantalla completa. Puede agregar un evento auxiliar para salir, por ejemplo al pulsar la letra `q`

```
if evento.type == KEYDOWN:
    if evento.key == K_q:
        pygame.quit()
        exit()
```

El modo de pantalla completa suele ser más rápido porque su `cript` no tiene que cooperar con otras ventanas en su escritorio. Para configurar el modo de pantalla completa, puede usar el parámetro indicador `FULLSCREEN` el segundo parámetro de `set_mode()`.

```
pantalla = pygame.display.set_mode((640, 480), FULLSCREEN, 32)
```

Cuando trabaje con pantalla completa, su tarjeta de vídeo probablemente cambiará a un modo de vídeo diferente, que cambiará el ancho y el alto de la pantalla, y potencialmente cuántos colores puede mostrar al mismo tiempo. Las tarjetas de vídeo solo admiten algunas combinaciones de tamaño y cantidad de colores, pero Pygame te ayudará si intentas seleccionar un modo de vídeo que la tarjeta no admite directamente.

Si el tamaño de pantalla que configurado no es compatible, Pygame seleccionará el siguiente tamaño y copia la pantalla en el centro, lo que puede generar bordes negros en la parte superior e inferior de la misma. Para evitar estos límites, se selecciona una de las resoluciones estándar que prácticamente todas las tarjetas de vídeo soportan: (640, 480), (800, 600) o (1024, 768).

Para ver exactamente qué resoluciones de visualización soporta, puede utilizar `pygame.display.list_modes()`, que devuelve una lista de tuplas que contienen resoluciones soportadas.

Probemos esto desde el intérprete interactivo:

```
>>> importar pygame
>>> pygame.init()
>>> pygame.display.list_modes()
```

```
[(1366, 768), (1360, 768), (1280, 768), (1280, 720),  
(1280, 600), (1024, 768), (800, 600), (640, 480)]
```

Si la tarjeta de video no puede brindarle la cantidad de colores que configuro, Pygame ajustara los colores en la superficie de la pantalla automáticamente, lo que puede resultar en una ligera caída en la calidad de la imagen.

En el siguiente script se muestra cómo pasar del modo de ventana al modo de pantalla completa.

Si presiona la tecla F, la pantalla ocupará toda la pantalla (puede haber un retraso de algunos segundos mientras esto sucede). Presione F por segunda vez y la pantalla volverá a una ventana.

```
pantalla
_completa.py  archivo_imagen_fondo = 'pygame.jpg'

import pygame
from pygame.locals import *
from sys import exit
pygame.init()
#CELESTE = (153,217,235)    # Color azul Celeste
CELESTE = (170,238,187)
#pantalla = pygame.display.set_mode((640, 480), FULLSCREEN, 32)

pantalla = pygame.display.set_mode((640, 480), 0, 32)

fondo = pygame.image.load(archivo_imagen_fondo).convert()

Fulls_pantalla = False

while True:
    for evento in pygame.event.get():
        if evento.type == KEYDOWN:
            if evento.key == K_q:
                pygame.quit()
                exit()
            if evento.type == QUIT:
                pygame.quit()
                exit()
            if evento.type == KEYDOWN:
                if evento.key == K_f:
                    Fulls_pantalla = not Fulls_pantalla
                    if Fulls_pantalla:
                        pantalla = pygame.display.set_mode((640, 480), FULLSCREEN, 32)
                    else:
                        screen = pygame.display.set_mode((640, 480), 0, 32)

    pantalla.fill(CELESTE)
    pantalla.blit(fondo, (0,0))

    pygame.display.update()
```

2.8.2. Redimensionar ventanas Pygame

Ocasionalmente, es posible que el usuario quiera cambiar el tamaño de una ventana de Pygame, lo que normalmente logra haciendo clic en la esquina de la ventana y arrastrando con el mouse. Es esta tarea bastante fácil, usando el parámetro indicador `RESIZABLE` cuando llamas a `set_mode()`. Pygame informa a su código que el usuario ha cambiado el tamaño de la ventana enviando un evento `VIDEORESIZE` que contiene el nuevo ancho y alto de la ventana. Cuando obtenga uno de estos eventos, debe llamar a la función `pygame.display.set_mode()` nuevamente para configurar las nuevas dimensiones del tamaño de ventana.

En el siguiente script se muestra cómo tratar los eventos `VIDEORESIZE` para redimensionar una ventana:

```
redimens
_ventana.py

archivo_imagen_fondo = 'pygame.jpg'

import pygame
from pygame.locals import *
from sys import exit
pygame.init()
tamana = (640, 480)
CELESTE = (170,238,187)    # Color azul Celeste

# Inicializar la pantalla
pantalla = pygame.display.set_mode(tamana, RESIZABLE, 32)
fondo = pygame.image.load(archivo_imagen_fondo).convert()
pygame.display.set_caption("Redimensión de Ventana")
# Inicializar la variable para manejar el tamaño de la pantalla
pantalla_ancho, pantalla_alto = tamana

# Bucle principal
while True:
    # Manejar eventos
    for evento in pygame.event.get():
        if evento.type == QUIT:
            pygame.quit()
            exit()
        if evento.type == KEYDOWN:
            if evento.key==K_q:
                pygame.quit()
                exit()

    # redimensionar ventana e imagen de fondo
    if evento.type == VIDEORESIZE:
        tamana = evento.size
        pantalla = pygame.display.set_mode(tamana, RESIZABLE, 32)
        pantalla_ancho, pantalla_alto = tamana
        (wp, hp) = tamana
        imagen_redimensionada = pygame.transform.scale(fondo, (wp, hp))
        fondo = imagen_redimensionada
        pygame.display.set_caption("Redimensión de Ventana" +str(evento.size))
```

```
# Dibujar el fondo en la pantalla
pantalla.fill(CELESTE)
for y in range(0, pantalla_alto, fondo.get_height()):
    for x in range(0, pantalla_ancho, fondo.get_width()):
        pantalla.blit(fondo,(x, y))

pygame.display.update()
```

Al ejecutar este **script**, mostrará una ventana Pygame con imagen de fondo `pygame.jpg`. Si hace clic en la esquina o el borde de la ventana y arrastra con el mouse, el **script** genera un evento `VIDEORESIZE`. En el controlador de ese evento se llama la función `set_mode()`, y se crea una nueva superficie de pantalla que coincide con las nuevas dimensiones.

El evento `VIDEORESIZE`, contiene los siguientes valores:

- **size** (tamaño): es una tupla que contiene las nuevas dimensiones de la ventana; `size[0]` es el ancho y el `size[1]` es la altura.
- **w**: este valor contiene el nuevo ancho de la ventana. Es el mismo valor que el `size[0]`, pero puede ser más conveniente.
- **h**: este valor contiene la nueva altura de la ventana. Es el mismo valor que `size[1]`, pero puede ser más conveniente.

Además, se ha configurado el objeto superficie de imagen de fondo, con la función `pygame.transform.scale(fondo, (w, h))`, a las nuevas dimensiones. Es importante tener presente que la imagen se puede ver un poco distorsionada y tal vez no se verá con una buena calidad, así es recomendable modificar el tamaño de imagen con un editor de imágenes en lugar de hacerlo con PyGame, pero al final es decisión tuya. También se pueden manejar como opción repetir el fondo en cuadrículas, múltiples copias de fondo para completar la nueva ventana, el siguiente código genera las cuadrículas para la imagen de fondo:

```
for y in range(0, pantalla\_alto, fondo.get\_height()):
    for x in range(0, pantalla\_ancho, fondo.get\_width()):
        pantalla.blit(fondo,(x, y))
```

Las dos llamadas al rango producen las coordenadas necesarias para colocar la imagen de fondo en cuadrículas (múltiples).

Generalmente los juegos se ejecutan en pantalla completa, por lo que las pantallas redimensionables quizás no sean una característica que utilices. ¡Pero está ahí en tu caja de herramientas si lo necesitas! ✓

2.8.3. Ventanas sin bordes

Generalmente, cuando creas una ventana de Pygame estándar con barras de título y frontera. Sin embargo, es posible crear una ventana que no tenga estas características para que el usuario no pueda mover ni cambiar el tamaño de la ventana, ni cerrarla mediante el botón de cerrar. Un ejemplo de tal uso es la ventana utilizada para las pantallas de presentación.

Algunos juegos pueden tardar un poco en cargar porque contienen muchos archivos de imagen y sonido. Si no hay nada visible en la pantalla mientras esto sucede, el jugador puede sentir que el juego no funciona e intentar reiniciarlo. Para configurar una visualización sin bordes, se usa el indicador `NOFRAME` a llamar la función `pygame.display.set_mode()`.

Por ejemplo, el siguiente script muestra una ventana sin títulos y sin bordes:

```
sin
_bordes.py from sys import exit
pygame.init()

# Inicializar el tamaño de la pantalla
tamana = (640, 480)
CELESTE = (170,238,187) # Color azul Celeste

# Inicializar la pantalla sin marcos
pantalla = pygame.display.set_mode(tamana, NOFRAME, 32)

fondo = pygame.image.load(archivo_imagen_fondo).convert()

# Bucle principal
while True:
    # Manejar eventos
    for evento in pygame.event.get():
        if evento.type == KEYDOWN:
            if evento.key==K_q:
                pygame.quit()
                exit()

    # Dibujar el fondo en la pantalla
    pantalla.fill(CELESTE)
    pantalla.blit(fondo,(0,0))

    pygame.display.update()
```

Al ejecutar archivo `sin_bordes.py`, verá una ventana con el fondo de imagen `pygame.jpg`, sin bordes, y sin títulos, como una presentación, semejante a la siguiente figura 2.7:

Se manejan los objetos eventos (`pygame.event.get()`) para cerrar el juego, creando una salida al presiona la tecla q (`K_q`).



Figura 2.7: Ventana sin borde y sin títulos

2.8.4. Parámetros adicionales para visualización

La función `pygame.display.set_mode( )` tiene otras opciones importante que puedes usar, pero deben ser usadas con más cuidado, debido a que pueden causar problemas de compatibilidad en algunas plataformas y afectar el rendimiento si se usan incorrectamente. Por ejemplo, hay un parámetro indicador de modo de visualización, el **hardware** es que no es compatible con todas las plataformas. Por lo general es mejor usar el valor 0 para ventanas en pantallas y **FULLSCREEN** para pantalla completa, debido a que garantizan el buen funcionamiento en todas las plataformas.

Sin embargo, si sabes lo que está haciendo, puede configurar algunas opciones avanzadas para obtener un rendimiento adicional.

Por ejemplo, si configura el parámetro indicador **HWSURFACE**, creará lo que se llama una superficie de **hardware**³. Este es un tipo especial de superficie de visualización que se almacena en la memoria de su tarjeta gráfica. Sólo se puede utilizar en combinación con el parámetro indicador **FULLSCREEN**, de la

³En Pygame, una superficie "hardware" se refiere a una superficie de visualización que se beneficia de la aceleración por hardware. Esto significa que la superficie de visualización se gestiona y procesa directamente por la tarjeta gráfica de tu computadora, lo que puede resultar en un rendimiento más rápido y eficiente en comparación con una superficie de visualización que se maneja exclusivamente por software.

Al utilizar una superficie "hardware" en Pygame, puedes aprovechar la capacidad de la tarjeta gráfica para realizar operaciones de dibujo y renderizado de manera más eficiente, lo que puede ser especialmente útil para aplicaciones gráficamente intensivas como juegos o simulaciones.

siguiente forma:

```
pantalla = pygame.display.set_mode(tamaño, HWSURFACE | FULLSCREEN, 32)
```

Esta línea crea una superficie de **hardware** con doble búfer⁴.

Las superficies de **hardware** también benefician al parámetro indicador **DOUBLEBUF**. Esto crea efectivamente dos superficies de **hardware**, pero solo una es visible en cualquier momento. Las superficies de **hardware** pueden ser más rápidas que las superficies creadas en la memoria (normal) del sistema, porque pueden aprovechar más funciones de su tarjeta gráfica para acelerar el blitting. La desventaja de las superficies de **hardware** es que no son compatibles con todas las plataformas. Suelen funcionar en plataformas Windows pero no tan bien en otras.

La siguiente línea crea un doble buffer superficie del hardware:

```
pantalla = pygame.display.set_mode(SCREEN_SIZE,
                                   DOUBLEBUF | HWSURFACE | FULLSCREEN, 32)
```

A continuación se describe una explicación detallada de cada parámetro indicador de visualización:

`pygame.display.set_mode()`: Esta función inicializa una ventana o pantalla para la visualización. Recibe tres parámetros: el tamaño de la ventana, los parámetros indicadores de visualización y la profundidad de color. **DOUBLEBUF | HWSURFACE | FULLSCREEN**: son los parámetros indicadores de visualización, que se combinan utilizando el operador **bitwise OR (|)**.

- **DOUBLEBUF**: Este parámetro indicador configura un doble buffer de visualización. Esto significa que la superficie de visualización se divide en dos partes: un buffer frontal donde se dibuja el juego y un buffer posterior donde se dibuja el juego antes de ser intercambiado con el buffer frontal. Esto ayuda a prevenir el **flicker**⁵ y el **tearing**⁶ de la visualización. En el contexto de

⁴En informática, un búfer (del inglés, buffer) es un espacio de memoria, en el que se almacenan datos de manera temporal, normalmente para un único uso (generalmente ocupan un sistema de cola FIFO); su principal función es evitar que el programa o recurso que los requiere, ya sea hardware o software, se quede sin datos durante una transferencia (entrada/salida) de datos irregular o por la velocidad del proceso.

⁵El "flicker" se refiere a un efecto visual donde los objetos en la pantalla parecen parpadear o titilar. Esto puede ocurrir cuando la pantalla no se actualiza de manera suave y consistente, lo que hace que los objetos se vean intermitentes o inestables.

⁶El "tearing" se refiere a un efecto visual donde la imagen en la pantalla se ve dividida

Pygame, "flicker" y "tearing" se refieren a problemas de visualización que pueden ocurrir cuando se mueven o actualizan objetos en la pantalla.

- **HWSURFACE**: Este parámetro indicador configura una superficie de visualización acelerada por hardware. Esto significa que la superficie de visualización se maneja por el hardware, lo que puede mejorar el rendimiento.

- **FULLSCREEN**: Este parámetro indicador configura una visualización en pantalla completa. Esto significa que la ventana cubrirá toda la pantalla y no tendrá una borde ni título.

Tanto el flicker como el tearing pueden ser problemas comunes en juegos y aplicaciones gráficas, y pueden requerir técnicas como el doble búfer, la sincronización vertical o el uso de aceleración de hardware para resolverlos.

Normalmente, cuando llamas a `pygame.display.update()`, se copia una pantalla completa en la memoria, lo que lleva un poco de tiempo. Las superficies con doble buffered le permiten cambiar a la nueva pantalla al instante y así hace que su juego se ejecute un poco más rápido.

Por último el parámetro indicador de visualización que puede utilizar es **OPENGL** (<http://www.opengl.org/>) y que lo desarrollaremos con más detalles en la sección ???. Se trata de una biblioteca de gráficos que utiliza el acelerador de gráficos 3D, y que se encuentra en casi todas las tarjetas gráficas. La desventaja de usar el parámetro indicador **OPENGL** es que ya no podrás usar las funciones de gráficos 2D de Pygame.

Además, si utiliza una pantalla con doble buffer, debe llamar a

```
pygame.display.flip()
```

en lugar de `pygame.display.update()`. Esto hace que la visualización instantánea cambie en lugar de copiar los datos de la pantalla.

En la tabla 2.8.4 se presenta un resumen de los parámetros indicadores utilizadas en la creación de la ventana de la función `pygame.display.set_mode()`.

o partida en dos o más partes. Esto ocurre cuando la pantalla se actualiza mientras el juego está dibujando un nuevo cuadro, lo que resulta en que se muestren partes de dos cuadros diferentes al mismo tiempo.

2.9. Resumen

Estos temas cubren los conceptos básicos para creación de juegos gráficos con la librería Pygame. Por supuesto, conocer y manipular estas funciones probablemente no sea suficiente para ayudarte a crear juegos. En los siguientes capítulos del libro se centran en analizar código fuente de juegos completo. Esto le dará más ideas formación de cómo "lucen" los programas de juego completos, para que pueda iniciar sus propios programas de juegos.

A asumimos que conoce los conceptos básicos de la programación en Python. Si tiene problemas para recordar esos conceptos puede revisar el libro un curso intensivo de Linis Guerrero, descarga gratis....

Capítulo 3

Movimientos, vectores y colisiones

Introducción a Python y PyGame módulo pygame para clases vectoriales

En el mundo real, los objetos se mueven de diferentes maneras y en cualquier dirección, dependiendo de lo que estén modelando, y en nuestro caso que son los juegos debe aproximar esos movimientos para crear una representación virtual convincente y agradable.

Algunos juegos pueden escapar con movimiento poco realista: por ejemplo Pac-Man (figura 3.1), se mueve en línea recta con una velocidad constante y puede cambiar de dirección en un instante, pero si aplicaras ese tipo de movimiento a un automóvil en un juego de conducción, sería destruir la ilusión. Después de todo, en un juego de conducción es de esperar que el coche tarde



Figura 3.1: Juego Pac-Man

algún tiempo en alcanzar su máxima capacidad velocidad y definitivamente no debería poder girar 180 grados en un instante!

Para juegos con un toque de realismo, el programador del juego debe tener en cuenta como se mueven los personajes(vehículos, animales, maquinas, etcétera).

Veamos un juego de típico conducción. Independientemente de si el vehícu-

lo es una bicicleta, un coche de rally o un semirremolque, siempre hay una fuerza del motor que lo impulsa hacia adelante. También hay otras fuerzas que actúan sobre él. La resistencia de las ruedas que varía según la superficie sobre la que conduzca, por lo que la resistencia del vehículo será diferente para el barro y para el asfalto. Y por supuesto también está la gravedad, que tira constantemente del coche hacia la tierra, generando fuerza de roce. Hay muchos factores que generan cientos de otras fuerzas que se combinan para crear el movimiento de un vehículo, afortunadamente para nosotros, los programadores de juegos, sólo tenemos que simular algunas de estas fuerzas para crear una ilusión convincente del movimiento.

Una vez escrito nuestro código de simulación, podemos aplicarlo a muchos objetos en el juego. La gravedad, por ejemplo, afectará a todo (a menos que el juego esté ambientado en el espacio), por lo que podemos aplicar el código relacionado con la gravedad a cualquier objeto, ya sea una granada de mano arrojada, un tanque que cae de un acantilado o un hacha volando por el aire.

En este capítulo se describe cómo mover objetos por la pantalla de forma predecible y cómo hacer ese movimiento consistente en las computadoras de otras personas.

En la próxima sección 3.1 trataremos la velocidad de fotogramas, muy importante para crear la ilusión convincente de la animación.

3.1. Velocidad de fotogramas

Lo primero que debemos saber sobre el movimiento en un juego de computadora es que nada se mueve realmente, al menos no en ningún sentido físico. Una pantalla de computadora o un televisor nos presenta una secuencia de imágenes, y cuando el tiempo entre las imágenes es lo suficientemente corto, nuestro cerebro combina las imágenes para crear la ilusión de fluido movimiento, como un libro animado.

El número de imágenes, o fotogramas, necesarios para producir un movimiento suave puede variar de persona a persona. Las películas usan 24 fotogramas (imágenes) por segundo, pero los juegos de computadora tienden a requerir un cuadro más, una tasa más rápida.

Treinta fotogramas por segundo es una buena velocidad, pero en términos generales cuanto mayor sea la velocidad de fotogramas, más suave se verá el movimiento. Para un evento deportivo, la escena de un crimen o si hay objetos que se mueven realmente rápido, entonces necesita una mayor velocidad de

fotogramas, que puede llegar a 60 FPS.

La velocidad de fotogramas de un juego también está limitada por el número de veces por segundo que el dispositivo de visualización (como su monitor) se puede actualizar. Por ejemplo, mi monitor LCD tiene una frecuencia de actualización de 60 hercios, lo que significa que actualiza la pantalla 60 veces por segundo.

Generar fotogramas más rápido de lo que puede generar la frecuencia de actualización es lo que se conoce como “tearing”, donde parte del cuadro siguiente se combina con un cuadro anterior, obtener una buena velocidad de fotogramas generalmente significa comprometer los efectos visuales, porque cuanto más trabajo haga su computadora, más lenta será la velocidad de fotogramas. La buena noticia es que la computadora de escritorio probablemente sea lo suficientemente rápido para generar las imágenes que necesita.

Todos los videos que vemos no son más que fotografías que se muestran a una determinada velocidad para crear el efecto de movimiento. Esas fotografías se llaman fotogramas y el número de fotogramas por segundo de un video es lo que llamamos velocidad de fotogramas.

En general, la velocidad de fotogramas estándar es de 24 a 30 fotogramas por segundo (FPS) para videos como películas y programas de televisión, y de 60 fotogramas por segundo para grabaciones de eventos deportivos, juegos y otras escenas que implican objetos en rápido movimiento.

Si se dice que un video es de 30 FPS, esto significa que cada segundo pasan treinta fotogramas consecutivos ante nuestros ojos.

3.2. Movimiento en línea recta, vertical y horizontal

Comencemos examinando el movimiento rectilíneo simple. Si movemos una imagen una cantidad fija en la ventana, entonces la imagen aparenta moverse. Para moverlo horizontalmente, agregaríamos a la coordenada x una velocidad o incremento, de igual forma para moverlo verticalmente el sumaríamos a la coordenada y un incremento. Las imágenes en movimiento en 2D a menudo se denominan **sprites**.

El archivo `movimiento_horizontal.py` presenta un algoritmo para mover una imagen horizontalmente; dibuja una imagen en una la coordenada x específica y luego agrega el valor de 10.0 a cada fotograma, de modo que en

el siguiente marco se habrá desplazado un poco hacia la derecha. Cuando la coordenada `x` llega al borde derecho de la ventana, se devuelve a 0 para que no desaparezca por completo.

Algoritmo para el movimiento simple en línea recta (horizontal):

```
movimiento
_horizontal.py import pygame
from pygame.locals import *
from sys import exit
archivo_image_fondo = 'pygame.jpg'

perro = 'perro.png'
pygame.init()

pantalla = pygame.display.set_mode((640, 480), 0, 32)
fondo = pygame.image.load(archivo_image_fondo).convert()
perro_sprite = pygame.image.load(perro)

# La coordenada x de nuestro sprite
x = 0.
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            exit()

    pantalla.blit(fondo, (0,0))
    pantalla.blit(perro_sprite, (x, 100))
    x += 1 # incremento en x, velocidad constante
    # Si la imagen llega al final de la pantalla, se devuelve
    # inicio de la ventana, en coordenada x = 0
    if x > 640:
        x -= 640
    pygame.display.update()
```

Si ejecuta, verá la imagen del perro deslizándose de izquierda a derecha. Esto es exactamente el efecto que estábamos buscando, pero hay una falla en el diseño. El problema es que no podemos saber exactamente cuánto tiempo llevará dibujar la imagen en la pantalla, y problema es que el `perro_sprite` se moverá muy rápido en computadoras potentes y más lento en máquinas computadoras menos potentes.

Parece razonablemente fluido porque estamos creando un marco extremadamente simple, pero en un juego el tiempo para dibujar un marco variará dependiendo de cuánto actividad que hay en pantalla. Y si queremos un juego que no ralentice (lento un proceso) justo cuando se vuelve interesante. El truco para resolver este problema es hacer que el movimiento se base en el tiempo, empleando el modulo de pygame para el tiempo `pygame.time.Clock()`.

Para el caso de manipular en forma manual el tiempo, se necesita saber cuanto tiempo tiene ha pasado desde el fotograma anterior para que podamos colocar todo en la pantalla siguiente.

3.2.1. El tiempo de pygame

El módulo `pygame.time.Clock` genera un objeto del tiempo o Reloj, y lo podemos usar para realizar seguimiento del tiempo. Para crear un objeto de reloj se llama a su función constructora `tiempo = pygame.time.Clock()`. Una vez creado el objeto reloj, debe llamar al método `tiempo.tick()`, que devuelve el tiempo transcurrido en milisegundos (hay 1.000 milisegundos en un segundo) desde la llamada anterior; ejemplo `tiempo_pasado = tiempo.tick()`, devuelve el tiempo transcurrido desde el ultimo llamado.

La función `tick()` también toma un parámetro opcional para la indicar la velocidad máxima de fotogramas, y es muy posible que necesite configurar este parámetro si el juego se ejecuta en el escritorio para que no utilice toda la potencia de procesamiento de la computadora:

```
# El juego se ejecutará a un máximo de 30 fotogramas por segundo.
tiempo_FPS= tiempo.tick(30)
```

Los milisegundos se utilizan a menudo para cronometrar eventos en los juegos porque puede ser más fácil manejar valores enteros en lugar de tiempos fraccionarios, y 1000 ticks de reloj por segundo generalmente son lo suficientemente precisos para la mayoría de las tareas del juego.

En lo personal prefiero trabajar en segundos cuando trato conceptos como la velocidad, porque 250 píxeles por segundo tiene más sentido para mí que 0,25 píxeles por milisegundo. La conversión de milisegundos a segundos es simple, dado en milisegundos se divide por 1000, y resulta el valor es segundos: `tiempo_pasado_segundos = tiempo_pasado/1000.0`

A continuación tenemos que calcular hasta qué punto el objeto se ha movido en el corto período de tiempo desde el último cuadro, y agrega ese valor a la coordenada x. La matemática para esto es sencilla, simplemente multiplica la velocidad del sprite por `tiempo_pasado_segundos`. Configurado el movimiento basado en el tiempo y el objeto se moverá a la misma velocidad independientemente de la velocidad de la computadora en la que lo ejecutas.

El siguiente código muestra el movimiento basado en el tiempo:

```
movimiento    import pygame
_tiempo.py    from pygame.locals import *
              from sys import exit

              archivo_image_fondo = 'pygame.jpg'

              pygame.init()
              pantalla = pygame.display.set_mode((640, 480), 0, 32)
              fondo = pygame.image.load(archivo_image_fondo).convert()
              sprite = pygame.image.load('perro.png')
```

```

# Configurando Reloj  objeto Reloj
tiempo = pygame.time.Clock()

# Velocidad en píxeles por segundo
velocidad = 300

# La coordenada x de nuestro sprite
x = 0.
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            exit()

    pantalla.blit(fondo, (0,0))
    pantalla.blit(sprite, (x, 100))
    tiempo_pasado = tiempo.tick()

    tiempo_pasado_seconds = tiempo_pasado/1000.0
    distance_moved= tiempo_pasado_seconds * velocidad
    x += distance_moved

    # Si la imagen llega al final de la pantalla, muévela hacia atrás
    if x > 640:
        x -= 640
    pygame.display.update()

```

Con este cálculo la imagen se mueve a una velocidad de 300 fotograma por segundo. El proceso se hizo manual, es la velocidad de un sprite en el juego.

Es importante entender la diferencia entre la velocidad de fotogramas y la velocidad de un sprite en el juego.



Si no está utilizando Python 3+, asegúrese de dividir por un valor de punto flotante de 1000,0; si no lo hace incluye el punto flotante, el resultado se redondeará hacia abajo al número entero más cercano. Entonces, ¿cómo usamos `time_passed_segundos` para mover un objeto? Lo primero que debemos hacer es elegir un velocidad para el sprite. Digamos que nuestro sprite se mueve a 300 píxeles por segundo. A esa velocidad, el sprite cubrir el ancho de una pantalla de 640 píxeles en 2,13 segundos (640 dividido por 300).

Velocidad de fotogramas (FPS)

La velocidad de fotogramas se refiere a cuántos fotogramas o frames se muestran por segundo en la pantalla. Esto determina la fluidez de la animación y el movimiento en general del juego. Por ejemplo, si un juego corre a 30 FPS, significa que se muestran 30 fotogramas cada segundo. A mayor

FPS, más fluida se verá la animación, pero también requiere más potencia de procesamiento. La velocidad de fotogramas es una propiedad global del juego, no de un sprite en particular. Se establece una vez para todo el juego

Velocidad de un sprite

La velocidad de un sprite se refiere a cuán rápido se mueve ese sprite en particular dentro del juego. Esto es independiente de la velocidad de fotogramas. Por ejemplo, puedes tener un sprite que se mueve a 100 píxeles por segundo, independientemente de si el juego corre a 30 FPS o 60 FPS. La velocidad de un sprite se puede definir de dos formas en Game Maker Studio:

- Fotogramas por segundo: el sprite avanza 1 fotograma cada cierto tiempo, independiente de los FPS del juego.
- Fotogramas por frame de juego: el sprite avanza 1 fotograma cada frame del juego, por lo que a mayor FPS, más rápido se verá la animación.

En resumen, la velocidad de fotogramas (FPS) determina la fluidez general del juego, mientras que la velocidad de un sprite determina que tan rápido se mueve y anima ese sprite en particular dentro del juego. Son conceptos relacionados pero independientes.

En el siguiente código se presenta dos imágenes dos sprites moviéndose en la pantalla, uno con velocidad por píxeles y el otro por FPS. El de arriba se mueve a 30 fotogramas, por segundo o tan suavemente como lo permita su computadora; el otro simula una computadora lenta actualizando sólo cada cinco fotogramas.

Deberías ver que aunque el movimiento es muy entrecortado para el segundo perro_sprite, en realidad no se mueven a la misma velocidad promedio. Entonces, para los juegos que usan movimiento basado en el tiempo, una velocidad de fotogramas lenta será resultará en una experiencia visual menos placentera, pero en realidad no ralentizará la acción.

El siguiente código presenta dos imágenes en la ventana, uno con velocidad de 30 FPS y el otro con velocidad píxeles por segundos, basado en el tiempo:

```
compara      archivo_image_fondo = 'pygame.jpg'
_velocidad.py import pygame
              from pygame.locals import *
              from sys import exit
              pygame.init()
              pantalla = pygame.display.set_mode((640, 480), 0, 32)
```

```

fondo = pygame.image.load(archivo_image_fondo).convert()
sprite = pygame.image.load('perro.png')
perro = pygame.image.load('perro.png')
perrox = 0
perroy = 200
direction = 'derecho'

# Configurando Reloj  objeto Reloj
tiempo = pygame.time.Clock()
velocidad = 30 # FPS
# Velocidad en pixeles por segundo
velocidad = 300
# La coordenada x de nuestro sprite
y = 300
x = 0.
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            exit()
    pantalla.blit(fondo, (0,0))
    pantalla.blit(perro, (perrox, 200))
    pantalla.blit(sprite, (x, 300))

    if direction == 'derecho':
        perrox += 1
        if perrox == 640:
            perrox = perrox - 640

    tiempo_pasado = tiempo.tick()
    tiempo_pasado_seconds = tiempo_pasado/1000.0
    distance_moved= tiempo_pasado_seconds * velocidad
    x += distance_moved # distancia en tiempo del ultimo

    # Si la imagen llega al final de la pantalla, muévela hacia atrás
    if x > 640:
        x -= 640

    pygame.display.update()
    tiempo.tick(velocidad) # todo el juego

```

Al ejecutar puede notar la diferencia de movimientos.

3.3. Movimiento en cualquier dirección, y colisión sencilla

El movimiento en línea recta es útil, pero un juego probablemente se sería bastante aburrido si todo se moviera en forma horizontal y vertical. Una simulación de movimientos de un objeto necesita moverse en cualquier dirección, para lograr este objetivo se hacen algunos ajustes en las coordenadas x y y .

El siguiente script simula un objeto que se mueve en dirección diagonal, agregando el movimiento basado en el tiempo en ambas coordenadas. Además, se detecta la colisión del sprite con los bordes de la ventana y se hace rebotar en sentido contrario, esta es una “detección de colisiones”, aunque muy trivial; de esta forma el sprite lugar de llevarlo de regreso a una posición inicial, cuando llega al borde de la ventana, el sprite rebota en el sentido contrario.

Para lograr este rebote, primero tenemos que detectar que el sprite ha tocado un borde, esto se hace con algunos cálculos matemáticos simples en las coordenadas. Si la coordenada x es menor o igual que 0, sabemos que se encuentra el lado izquierdo de la pantalla porque la coordenada del borde izquierdo es 0. Si x es mayor que la suma de la posición actual en x del objeto más el ancho del sprite, sabemos que el sprite ha tocado el lado derecho de la pantalla. El caso la coordenada y es similar, pero usamos la altura del sprite.

El siguiente archivo py, el sprite es un círculo de radio igual a 40 rebotando en la ventana pygame:

```
rebote      import pygame
_objeto.py import sys
            import time
            # Inicializar Pygame
            pygame.init()

            # Definir colores
            BLACK = (0, 0, 0)
            WHITE = (255, 255, 255)
            CELESTE = (170, 238, 187)

            tiempo= pygame.time.Clock()
            velocidad = 200

            # Tamaño de la pantalla
            ancho, alto = 640, 480
            SCREEN = pygame.display.set_mode((ancho, alto))
            pygame.display.set_caption("Rebote de objeto")

            # Tamaño del objeto
            Obj_ancho, Obj_alto = 40, 40

            # Posición inicial del objeto
            #obj_x = ancho//2 - Obj_ancho//2
            #obj_y = alto//2 - Obj_alto//2
            obj_x = 50
            obj_y = 100
            # Velocidad del objeto
            vel_x = 5
            vel_y = 5
```

```

# Bucle principal
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

    # Actualizar posición del objeto
    obj_x += vel_x
    obj_y += vel_y

    # Rebotar en los bordes
    if obj_x <= 40 or obj_x + Obj_ancho >= ancho:
        vel_x = -vel_x
    if obj_y <= 40 or obj_y + Obj_alto >= alto:
        vel_y = -vel_y

    # Limpiar pantalla
    SCREEN.fill(CELESTE)

    # Dibujar objeto
    #pygame.draw.rect(SCREEN, WHITE, (obj_x, obj_y, Obj_ancho, Obj_alto))
    pygame.draw.circle(SCREEN, WHITE, (obj_x, obj_y), 40)

    # Actualizar pantalla
    pygame.display.update()
    #pygame.display.flip()
    tiempo.tick(velocidad)

```

El sprite es un círculo de radio 40, rebotando en la ventana, una “detección de colisiones” simple.

En el siguiente código se agregamos un obstáculo en el centro de la ventana para generar colisión del objeto sprite con el obstáculo, se detecta la colisión y generar el rebote el sprite:

```

obstáculo
_rebote.py # -*- coding: utf-8 -*-
"""
Created on Mon Apr  3 17:07:13 2023

@author: Linis
"""
import pygame
import sys
import time
# Inicializar Pygame
pygame.init()

# Definir colores
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
CELESTE = (170, 238, 187)

# Tamaño de la pantalla

```

```

WIDTH, HEIGHT = 640, 480
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
#pygame.display.set_caption("Rebote de objeto")

# Tamaño del objeto
OBJ_WIDTH, OBJ_HEIGHT = 20, 20

# Posición inicial del objeto
obj_x = WIDTH // 2 - OBJ_WIDTH // 2
obj_y = HEIGHT // 2 - OBJ_HEIGHT // 2

# Velocidad del objeto y tiempo
tiempo= pygame.time.Clock()
velocidad = 200
vel_x = 5
vel_y = 5

pygame.display.set_caption("Colisión circular")

# Tamaño y posición del objeto circular
circle_radius = 40
circle_x = WIDTH//2
circle_y = HEIGHT//2

# Tamaño y posición del obstáculo rectangular en el centro
obstacle_width = 100
obstacle_height = 100
obstacle_x = WIDTH//2 - obstacle_width//2
obstacle_y = HEIGHT//2 - obstacle_height//2

# Bucle principal
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

    # Actualizar posición del objeto
    obj_x += vel_x
    obj_y += vel_y

    # Rebotar en los bordes
    if obj_x <= 20 or obj_x + OBJ_WIDTH >= WIDTH:
        vel_x = -vel_x
    if obj_y <= 20 or obj_y + OBJ_HEIGHT >= HEIGHT:
        vel_y = -vel_y

    # Limpiar pantalla
    SCREEN.fill(CELESTE)

    # Dibujar objeto circular
    pygame.draw.circle(SCREEN,WHITE,(obj_x, obj_y),20)
    obstacle_x = obj_x
    obstacle_y = obj_y
    obstacle_width = 2.5
    obstacle_height = 2.5
    pygame.draw.circle(SCREEN, BLACK, (circle_x, circle_y), 5)

```

```

# Detectar colisión entre el obstaculo y el pelota
distance_x = abs(circle_x - obstacle_x - obstacle_width/2)
distance_y = abs(circle_y - obstacle_y - obstacle_height/2)

if (distance_x <= (obstacle_width/2 + circle_radius) and
    distance_y <= (obstacle_height/2 + circle_radius)):
    # Colisión detectada
    print("Colisión detectada")
    # Rebotar en los bordes
    if obj_x == WIDTH // 2 or obj_x + OBJ_WIDTH >= WIDTH:
        vel_x = -vel_x
    if obj_y == HEIGHT // 2 or obj_y + OBJ_HEIGHT >= HEIGHT:
        vel_y = -vel_y

# Actualizar pantalla
pygame.display.flip()
tiempo.tick(velocidad)

```

Hemos visto que agregar un valor basado en el tiempo a las coordenadas x e y de un objeto crea un movimiento en cualquier dirección. Sin embargo la mejor manera de trabajar el movimiento es con vectores. En PyGame los vectores se tratan con clases, una clase `Vector2` para dos dimensiones y una clase `Vector3` para tres dimensiones, en la siguiente sección 3.4 se estudia el tema de clases de vectores pygame y sus métodos.

3.4. Explorando vectores

Para generar el movimiento diagonal usamos dos valores de velocidad, uno para la coordenada x y otra para y . Estos dos valores combinados forman un punto, y si tiene dirección es lo que se conoce como vector. Un vector es algo que los desarrolladores de juegos tomaron prestado de las matemáticas y se utilizan con frecuencia en Juegos 2D y 3D.

Los vectores son similares a los puntos en que ambos tienen un valor para x y otra para y (en 2D), pero se usan para diferentes propósitos. Un punto en la coordenada $(10, 20)$ siempre será el mismo punto en la pantalla, pero un vector de $(10, 20)$ significa sumar 10 a la coordenada x y 20 a la coordenada y desde la posición inicial. Entonces podrías pensar en un punto como si fuera un vector desde el origen $(0, 0)$.

Creando vectores

Puedes crear un vector a partir de dos puntos cualesquiera digamos $A(x_0, y_0) = (x_0, y_0)$ y $B(x_1, y_1) = (x_1, y_1)$, restando a los valores del primer

punto A los del segundo punto B : $\vec{AB} = \vec{AB}(x_1 - x_0, y_1 - y_0)$.

Ilustremos con un ejemplo de un juego ideal. Supongamos un personaje de un juego de batallas, un soldado del futuro llamado Alpha que debe destruir un centinela droide clase Beta con un rifle de francotirador. Alfa está escondiéndose detrás de un arbusto en la coordenada $A(10, 20)$ y apuntando a Beta en la coordenada $B(30, 35)$. Para calcular un vector $\vec{AB}$ dirigido al objetivo, Alfa tiene que restar los componentes de B a las componentes de A .

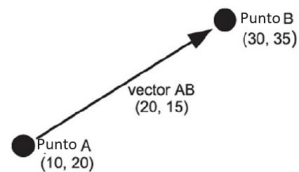


Figura 3.2: Vector $\vec{AB}$

Es decir, el vector $\vec{AB}$ es igual a $(30 - 10, 35 - 20) = (20, 15)$, resultando $\vec{AB}(20, 15)$. Esto nos dice que para llegar de A a B tendríamos que recorrer 20 unidades en la dirección x y 15 unidades en la dirección y , como se muestra en la figura 3.2. El juego necesitaría esta información para dirigir un proyectil, un arma, o un rayo láser entre los dos puntos.

Tratamiento de vectores en PyGame

En Python, los vectores se pueden representar utilizando listas o matrices. La biblioteca **NumPy** es una forma común de representar vectores y ofrece operaciones de álgebra lineal. Puedes crear vectores utilizando el método `numpy.array()` y puedes realizar operaciones básicas con vectores en Python, como sumas y restas.

Sin embargo, en Pygame los vectores se utilizan para representar posiciones y direcciones en el espacio, y la librería `pygame.math` proporciona dos clases `Vector2` y `Vector3` para vectores bidimensionales y tridimensionales respectivamente. Estas clases ofrecen operaciones numéricas elementales, como sumas, restas, y operaciones más complejas como productos escalares, productos vectoriales y ángulo entre vectores.

Para definir un vector, ahora podemos usar objetos generados por la clase `Vector2`. Por ejemplo, una llamada a `v1 = pygame.Vector2(10, 20)` pro-

duce un objeto `Vector2` llamado `v1`. Veamos algunos ejemplos del uso de la clase `Vectorial`:

```
import pygame
v1 = pygame.Vector2(10, 20)
v2 = pygame.Vector2(3, 4)
V_sum = v1 + v2
V_diff = v1 - v2
V_produc = v1*v2 # producto escalar
V_produc_cruz = v1.cross(v2) # producto cruz
```

Además, podemos referirnos a los componentes del vector individualmente como `v1.x` y `v1.y`.

Esta es una forma clara y didáctica para tratar vectores en PyGame. Claro que podemos utilizar NumPy, pero lo más didáctico por ahora es usar las clases vectoriales del modulo integrado de pygame `pygame.math`, `Vector2` y `Vector3`. A medida que avancemos en los ejemplos, cubriremos vectores y la matrices.

Los métodos de la clase vectorial de Pygame, como `Vector2` y `Vector3`, proporcionan funcionalidades esenciales para operar con vectores en dos y tres dimensiones respectivamente, permitiendo realizar cálculos y manipulaciones matemáticas sobre ellos de manera eficiente. Para mas información visitar la página oficial https://www.pygame.org/docs/ref/math.html#pygame.math.Vector2.as_polar.

En la siguiente tabla 3.4 se presenta la clase `Vector2` con los métodos más comunes y su propósito:

Table 3.4. Método de la clase vectorial `Vector2`:

Método	Propósito
<code>v1.dot(v2)</code>	calcula el producto escalar
<code>v1.cross(v2)</code>	calcula el producto cruzado o vectorial
<code>v1.magnitude()</code>	devuelve la magnitud euclidiana del vector
<code>v1.magnitude_squared()</code>	devuelve la magnitud al cuadrado del vector
<code>v1.length()</code>	devuelve la longitud euclidiana del vector
<code>v1.normalize()</code>	devuelve un vector con la misma dirección pero de longitud 1
<code>v1.normalize_ip()</code>	normaliza el vector en su lugar para que su longitud sea 1
<code>v1.is_normalized()</code>	prueba si el vector está normalizado es decir, tiene una longitud == 1
<code>v1.scale_to_length(float)</code>	escala el vector a una longitud determinada
<code>v1.reflect(v2)</code>	devuelve un vector reflejado de una normal dada
<code>v1.reflect_ip(v2)</code>	reflejar el vector de una normal dada en su lugar
<code>v1.distance_to(v2)</code>	calcula la distancia euclidiana a un vector dado
<code>v1.lerp(v2, float)</code>	devuelve una interpolación lineal al vector dado
<code>v1.rotate(angle)</code>	gira un vector en un ángulo dado en grados
<code>v1.rotate_rad(angle)</code>	gira un vector en un ángulo dado en radianes
<code>v1.rotate_ip(angle)</code>	gira el vector en un ángulo determinado en grados en su lugar
<code>v1.angle_to(v2)</code>	Calcula el ángulo entre dos vectores dado en grados

El siguiente código crear un vector entre dos puntos A y B dados, mediante las clases vectoriales de pygame:

```
A = (10.0, 20.0, 0)      # Puntos A
B = (30.0, 35.0, 0)      # Punto B
A = pygame.Vector3(A)    # Vector PyGame A
B = pygame.Vector3(B)    # Vector PyGame B

AB = B-A                 # Crea un vector de A a B
A.distance_to(B)          # Distancia de A a B

print("Vector AB =", B-A)
print("Distancia de A a B =", A.distance_to(B), " unidades")
print("Coordenadas \n x, y = ", AB.x, ",", AB.y)
```

La ejecución de este código produce el siguiente resultado:

```
Vector AB = [20, 15, 0]
Distancia de A a B = 25.0 unidades
Coordenadas
x, y = -1.0 , 1.0
```

3.4.1. Vectores y el movimiento

Ahora que tenemos el conocimiento básico sobre vectores pygame, podemos usarlos para mover objetos o personajes del juego de diversas maneras, e implementar magnitudes físicas simples basadas en la fuerza que simulen más realidad el juego.

3.4.1.1. Movimiento diagonal

Para mover un sprite de una posición a otra en la pantalla, a una velocidad constante. El primer paso es crear un vector a partir de la posición actual al destino ($AB = B - A$), necesitamos información de la dirección de este vector pero no la magnitud, por lo que lo normalizamos el vector (`AB.normalize()`), para obtener el rumbo (sentido) del sprite.

```
import pygame
pygame.init()

tiempo = pygame.time.Clock()    # Objeto tiempo, velocidad de FPS
velocidad = 250 #píxeles
A = (60, 50, 0)    # Punto A
B = (100, 100, 0)  # Punto B
A = pygame.Vector3(A) # vector A
B = pygame.Vector3(B) # Vector B
AB = B - A           # Vector de A a B. Posición
rumbo = AB.normalize() # dirección
```

Calculamos también `velocidad*tiempo_pasado_segundos` que indica cuanto se ha movido el sprite del anterior cuadro, y luego lo multiplicamos por el vector de rumbo. El vector resultante nos da el cambio desde la posición anterior en x e y, luego lo agregamos a la posición actual (recorrido) del sprite.

```
tiempo_pasado = tiempo.tick()
tiempo_pasado_segundos = tiempo_pasado/1000.0
velocidad = velocidad_I*tiempo_pasado_segundos
cambio = rumbo*velocidad # cambio en x e y
```

Ahora dentro del bucle principal del juego calculamos qué tanto se ha movido el sprite con `velocidad * tiempo_pasado_segundos`, luego multiplicamos por el vector de rumbo. El vector resultante nos da el cambio en x e y desde el cuadro anterior. Este cambio lo agregamos a la posición (recorrido) del sprite, pero en el bucle principal del juego.

```
# Rebotar en los bordes
if recorrido.x <= 20 or recorrido.x + OBJ_WIDTH >= WIDTH:
    cambio.x = -cambio.x
if recorrido.y <= 20 or recorrido.y + OBJ_HEIGHT >= HEIGHT:
    cambio.y = -cambio.y
cambio = pygame.Vector3(cambio.x, cambio.y, 0)
```

En el bucle principal agregamos el cambio a la posición del sprite, y para seguir la idea anterior colocamos una “detección de colisión”, para que el sprite rebote en la ventana PyGame.

Ahora presentamos todo el trabajo en el siguiente script `movimiento_vectorial.py`

```

archivo_imagen_fondo = 'pygame.jpg'
import pygame, sys
from pygame.locals import *

tamaño = (640, 480)
CELESTE = (170,238,187)      # Color azul Celeste
WHITE =(255,255,225)
velocidad_I = 250  # Configurando fotograma por segundos

pygame.init()
tiempo = pygame.time.Clock() # Objeto tiempo, velocidad de FPS

# Inicializar la pantalla
Obj_ancho, Obj_alto = 20, 20
ancho, alto = 640, 480
pantalla = pygame.display.set_mode((ancho, alto), 0, 32)
pygame.display.set_caption("Movimiento vectorial")
fondo = pygame.image.load(archivo_imagen_fondo).convert()

velocidad = 250          # píxeles
A = (60, 50,0)          # Punto A
B = (100, 100,0)        # Punto B
A = pygame.Vector3(A)   # vector A
B = pygame.Vector3(B)   # Vector B
recorrido = B-A          # AB = B-A, Vector de A a B. Posición
rumbo = recorrido.normalize() # dirección o rumbo

tiempo_pasado = tiempo.tick()
tiempo_pasado_seconds = tiempo_pasado/1000.0
velocidad = tiempo_pasado_seconds *velocidad_I
cambio = rumbo*velocidad

while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
        pantalla.fill(CELESTE )
        pantalla.blit(fondo, (0,0))
        pygame.draw.circle(pantalla,WHITE,(recorrido.x,recorrido.y),20)

    # Rebotar en los bordes
    if recorrido.x <= 20 or recorrido.x + Obj_ancho >= ancho:
        cambio.x = -cambio.x
    if recorrido.y <= 20 or recorrido.y + Obj_alto >= alto:
        cambio.y = -cambio.y
    cambio = pygame.Vector3(cambio.x,cambio.y,0)
    recorrido += cambio
    pygame.display.update()
    tiempo.tick(velocidad_I)

```

El script implementa un movimiento basado en el tiempo utilizando vectores. Cuando lo ejecutes, verás un sprite moviéndose en la ventana, pero una vez que haces clic en la pantalla, el sprite inicia el movimiento en la posición indicada por el mouse, con la dirección anterior, el código calculará un vector

[illegible]

75

```
tiempo.tick(velocidad_I)
```

El siguiente script `hacia_el_mouse.py` implementa un movimiento basado en el tiempo utilizando vectores. Cuando lo ejecutes, verás un sprite moviéndose en la ventana, pero una vez que haces clic en la pantalla, el código calculará un vector para el nueva posición.

```
hacia_el      archivo_imagen_fondo = 'pygame.jpg'
mouse.py      sprite_image = "alien03.jpg"
              Obj_ancho = 24
              Obj_alto = 20
              ancho = 640
              alto = 480
              CELESTE = (170,238,187)

              import pygame, sys
              from pygame.locals import *
              pygame.init()
              tiempo = pygame.time.Clock() # Objeto tiempo, velocidad de FPS

              pantalla = pygame.display.set_mode((ancho,alto), 0, 32)
              fondo= pygame.image.load(archivo_imagen_fondo).convert()
              pygame.display.set_caption("Hacia la posición del mouse")
              sprite = pygame.image.load(sprite_image).convert_alpha()

              posicion = pygame.Vector2(100.0, 100.0)
              recorrido = posicion
              rumbo = posicion.normalize()
              cambio = rumbo
              destination = posicion
              while True:
                  pantalla.fill(CELESTE)
                  pantalla.blit(sprite, (recorrido.x, recorrido.y))
                  for event in pygame.event.get():
                      if event.type == QUIT:
                          pygame.quit()
                          sys.exit()

                      if event.type == MOUSEBUTTONDOWN:
                          destination = pygame.Vector2(event.pos)
                          posicion = destination - recorrido
                          cambio = posicion.normalize()
                          recorrido += cambio
                  # Rebotar en los bordes
                  if recorrido.x <= 0 or recorrido.x + 2*Obj_ancho >= ancho:
                      cambio.x = -cambio.x
                  if recorrido.y <= 0 or recorrido.y + 2*Obj_alto >= alto:
                      cambio.y = -cambio.y
                  pantalla.blit(fondo, (0,0))
                  pantalla.blit(sprite, (recorrido.x, recorrido.y))

              pygame.display.update()
              tiempo.tick(200)
```

Si vuelves a hacer clic, se calculará un nuevo vector, y el objeto cambiará su dirección hacia la posición del mouse.

3.5. Resumen

Mover un objeto, o cualquier otra cosa en la pantalla, requiere que agregues pequeños cambios o valores a las coordenadas en cada una, pero si desea que el movimiento sea suave y consistente, debe basarse en el actual tiempo, o más específicamente, el tiempo transcurrido desde el último fotograma. Conocer movimiento basado en el tiempo también es importante para ejecutar el juego en una gama tan amplia de computadoras como sea posible; las computadoras pueden variar mucho en el número de fotogramas por segundo que pueden generar.

Se ha tratado vectores con la clases `Vector2` y `Vector3`, que son una parte esencial de la caja de herramientas de cualquier desarrollador de juegos. Los vectores simplifican gran parte de los cálculos los tendrás que hacer al escribir tu juego y los encontrarás notablemente versátiles.

Las técnicas para moverse en dos dimensiones se extienden fácilmente a las tres dimensiones. Encontrarás que la clase `Vector3` contiene muchos de los métodos utilizados en la clase `Vector2` pero con la componente adicional en `z`. Ahora sería un buen momento para empezar a experimentar moviendo elementos en pantalla y mezclando varios tipos de movimiento. Puedes divertirte mucho creando gráficos de tus amigos y familiares en pantalla y ¡hacer que se deslicen y reboten!

Pendiente el teclado y el joystick, a sprites, para que el jugador pueda interactuar con el mundo del juego.

En el siguiente capítulo, se estudiara el tema relacionado con gráficos en tres dimensiones, ahora incluimos una conocida como **OpenGL** (Open Graphics Library). Es la API de gráficos más adoptada, una librería multiplataforma, compatible con múltiples sistemas operativos, incluyendo Windows, macOS y Linux; los resultados visuales son consistentes en cualquier sistema operativo compatible. Además, OpenGL se puede utilizar junto con una variedad de lenguajes de alto nivel mediante enlaces: bibliotecas de software que unen dos lenguajes de programación, permitiendo que funciones de un lenguaje se utilicen en otro.

Capítulo 4

Interacción del Jugador con el juego

Hay una variedad de formas en que el jugador puede interactuar con un juego, por ejemplo, el teclado, mouse, joysticks y joypads, etc; el este capítulo se cubren algunos dispositivos de entrada en detalle.

Además de recuperar información de los dispositivos, también exploraremos cómo traducir la actividad del jugador con eventos significativos en el juego. Esto es extremadamente importante para cualquier juego: independientemente de lo bien que se vea y suene un juego, también debe ser fácil interactuar con él.

Controlando el juego

En una computadora doméstica típica, podemos confiar en que habrá un teclado y un mouse. Esta es la configuración preferida por los fanáticos de los juegos de disparos en primera persona a quienes les gusta controlar el movimiento de la cabeza (es decir, mirar a su alrededor) con el mouse y mantener una mano en el teclado para controles direccionales y disparos. Los teclados se pueden utilizar para el movimiento asignando una tecla para las cuatro direcciones básicas: arriba, abajo, izquierda y derecha. Se pueden indicar otras cuatro direcciones presionando estas teclas en combinación (arriba + derecha, abajo + derecha, abajo + izquierda, arriba + izquierda). Pero teniendo ocho direcciones sigue siendo muy limitante para la mayoría de los juegos, por lo que no son ideales para juegos que necesitan un poco de delicadeza

para jugar.

La mayoría de los jugadores prefieren un dispositivo analógico como el mouse. El estándar es ideal para controlar la direccional porque puede detectar con precisión cualquier cosa, desde pequeños ajustes hasta barridos rápidos en cualquier dirección.

Si alguna vez has jugado a un juego de disparos en primera persona con solo el control del teclado, apreciarás la diferencia. Al presionar la tecla rotar a la izquierda", el jugador gira como un robot con una velocidad constante, lo cual no es casi tan útil como poder darse la vuelta rápidamente para disparar a un monstruo que se acerca por un lado.

El teclado y el mouse funcionan bien para juegos, lo cual quizás sea un poco irónico porque ninguno de los dos dispositivos fue diseñado pensando en los juegos. Sin embargo, los joysticks y joypads, fueron diseñados exclusivamente para juegos y han evolucionado junto con los juegos. Los primeros joysticks se inspiraron en los controles utilizados en avión y tenía palancas direccionales simples con un solo botón. Eran populares entre las consolas de juegos, pero a los jugadores les resultaba incómodo tener que agarrar la base del joystick con una mano mientras movía el otro.

Esto llevó al desarrollo de joypads, que podían sostenerse con ambas manos y así daban al jugador un fácil acceso a los controles con los dedos y los pulgares. Los primeros joypads tenían un control direccional encendido por un lado y botones de disparo por el otro. Hoy en día los joypads cuentan con numerosos botones ubicados por todas partes hay un dedo libre para presionarlos, junto con varias palancas. Los pads direccionales clásicos todavía se encuentran en joypads, pero la mayoría también tiene palancas analógicas que pueden detectar ajustes finos.

Muchos también tienen retroalimentación, características que pueden agregar una dimensión adicional a los juegos al hacer que el joystick tiemble o retumbe en respuesta a eventos en la pantalla. Sin duda, en el futuro se agregarán otras funciones a los joypads que mejorarán aún más mejorar la experiencia de juego.

Hay otros dispositivos que se pueden utilizar para juegos, pero la mayoría imitan los dispositivos de entrada estándar. Entonces, si su juego se puede jugar con el mouse, también se podrá jugar con estos dispositivos similares a un mouse.

Por ahora discutiremos el manejo de la entrada con teclados, PyGame tiene el módulo `pygame.key` para detecta los eventos de pulsaciones en el teclado, y lo describimos en la siguiente sección.

4.1. Comprender el teclado para controlar el juego

La mayoría de los teclados que se utilizan hoy en día son teclados QWERTY, llamados así porque las seis letras iniciales de la primera fila que deletrea QWERTY. Existen variaciones entre marcas; las teclas pueden tener formas y tamaños ligeramente diferentes, pero tienden a estar aproximadamente en la misma posición en el teclado. Esto es bueno, ya que en la computadora los usuarios no tienen que volver a aprender a escribir cada vez que compran un teclado nuevo, los teclados que he utilizado han tenido cinco filas para teclas de escritura estándar: una fila para teclas de función F1–F12 y cuatro teclas para moviendo un cursor por la pantalla. También tienen un "teclado numérico", que es un bloque de teclas para ingresar números y hacer algunas operaciones. El teclado numérico suele omitirse en los teclados de portátiles para ahorrar espacio, porque las teclas numéricas están duplicadas en la parte principal del teclado.

Aunque es el más común, QWERTY no es la única distribución de teclado; hay otros teclados como AZERTY y Dvorak; que se pueden utilizar las mismas constantes del teclado, pero las teclas pueden estar en diferentes ubicaciones en el teclado.

Para detectar o identificar la teclas se usa el módulo `pygame.key`. En la próxima sección se estudia las pulsaciones de teclas como evento PyGame.

4.1.1. Detección de pulsaciones de teclas

Hay dos formas para detectar la pulsación de una tecla en PyGame. Una forma es manejar eventos `KEYDOWN`, que se emiten cuando se presiona una tecla, y la otra son eventos `KEYUP`, que se emiten cuando se suelta la tecla.

Esto es genial para escribir texto porque siempre obtendremos eventos de teclado incluso si se ha presionado y soltado una tecla en el tiempo transcurrido desde el último fotograma.

Los eventos también capturarán toques muy rápidos de la tecla para los botones de disparo. Pero cuando usamos el teclado como entrada para movimiento, simplemente necesitamos saber si la tecla está presionada o no antes de dibujar el siguiente cuadro. En este caso, podemos utilizar el módulo `pygame.key` de forma más directa. Cada tecla tiene una constante asociada,

que es un valor que podemos usar para identificar la tecla en el código. Cada constante comienza con K_. Hay letras (K_a a K_z), números (K_0 a K_9) y muchas otras constantes como K_f1, K_LEFT y K_RETURN. Puede consultar en la documentación de Pygame para obtener una lista completa <https://www.pygame.org/docs/ref/key.html>.

Debido a que existen constantes para cada tecla, desde K_a a K_z, tal vez Usted puede suponer que también hay versiones equivalentes para el caso de mayúsculas, pero no es así. La razón de esto es que las letras mayúsculas son el resultado de combinar teclas (Shift + tecla). Si necesita detectar letras mayúsculas u otras teclas desplazadas, debe utilizar el parámetro Unicode en eventos que contienen el resultado de dichas combinaciones de teclas.

La función `pygame.key.get_pressed` devuelve una lista de booleanos (valores Verdaderos o Falso) si se presiona una tecla. Para buscar una clave en particular, debe usar su constante como índice en la lista presionada. Por ejemplo, si usáramos la barra espaciadora como botón de disparo, podría escribir el código de activación de esta manera:

```
pressed_keys = pygame.key.get_pressed()
if pressed_keys[K_SPACE]:
    # Tecla espaciadora ha sido presionada
    fire()
```

Es importante saber que algunos teclados tienen varias reglas con respecto a cuántos pulsaciones de teclas simultáneas admiten. Si observa los teclados para juegos, encontrará que casi siempre enumeran, como punto de venta, cuántas pulsaciones simultáneas de teclas admiten, muchos teclados baratos admiten tan solo de 3 a 5 pulsaciones de teclas simultáneas, mientras que los teclados para juegos pueden soportar hasta más de veinte. En muchos juegos, es posible que tengas una clave para agacharte, avanzar, ametrallar un poco y moverte lentamente. Tal vez mientras haces este ataque lento y agachado en diagonal, debes hacer otra acción.

En el siguiente script se utiliza la función `get_pressed` para detectar cualquier tecla presionada y muestra una lista de ellas en la pantalla. La función genera una lista de las teclas presionadas como con valores booleanos, mediante un bucle `for` que itera a través de cada una de ellas valor.

```
demo
_tecla.py import pygame
from pygame.locals import *
from sys import exit
pygame.init()
screen = pygame.display.set_mode((640, 480), 0, 32)
font = pygame.font.SysFont("arial", 32);
```

```

font_height = font.get_linesize()
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            exit()

    screen.fill((255, 255, 255))
    pressed_key_text = []
    pressed_keys = pygame.key.get_pressed()
    y = font_height
    for key_constant, pressed in enumerate(pressed_keys):
        if pressed:
            key_name = pygame.key.name(key_constant)
            print( key_name)
            text_surface = font.render("pressed" + key_name, True, (0,0,0))
            screen.blit(text_surface, (8, y))
            y+= font_height
    pygame.display.update()

```

El bucle itera sobre las teclas presionadas sobre `enumerate`, que es una función incorporada que devuelve una tupla del índice (el primer valor es 0, el segundo es 1, etc.) y el valor de la lista iterada. Si el valor iterado (presionado) es Verdadero, significa que esa tecla ha sido presionada, luego con un bloque de código se muestra en la pantalla. Este bloque de código utiliza otra función en el módulo de teclado, `pygame.key.name`, que toma una clave constante y devuelve una cadena con una descripción de la clave (es decir, se convierte `K_SPACE` en "espacio").

Aquí puedes verificar cuántas teclas admite su teclado. Los escenarios pueden variar desde admitir hasta 8 o más, pero algunas veces tan solo 4, o algo así. Para más información a cerca del funcionamiento de los teclados en http://www.sjbaker.org/wiki/index.php?title=Keyboards_Are_Evil.

Ahora describimos el módulo `pygame.key` con más detalle:

- `key.get_focused`: una ventana de Pygame solo recibe eventos clave cuando la ventana está enfocado, generalmente haciendo clic en la barra de título de la ventana. Esto es cierto para todos las ventanas de alto nivel. La función `get_focused` devuelve `True` si la ventana tiene foco y puede recibir eventos clave; de lo contrario, devuelve `False`. Cuando Pygame se está ejecutando en modo de pantalla completa, siempre tendrá foco porque no tiene forma para compartir la pantalla con otras aplicaciones.
- `key.get_pressed`: devuelve una lista de valores booleanos para cada

clave. Si alguno de los valores están configurados en `True`, se presiona la tecla para ese índice.

- `key.get_mods`: devuelve un valor único que indica cuál de las teclas modificadoras están presionados. Las teclas modificadoras son teclas como Shift, Alt y Ctrl que se utilizan en combinación con otras teclas. Para verificar si se presiona una tecla modificadora, debe usar el Operador AND bitwise (`&`) con `KMOD_constant`. Por ejemplo, para comprobar si se presiona la tecla Shift izquierda, usaría `pygame.key.get_mods() & KMOD_LSHIFT`.

- `pygame.key.set_mods`: también puedes configurar una de las teclas modificadoras para imitar el efecto de presionar una tecla. Para configurar una o más teclas modificadoras, se combina `KMOD_constant` con el operador bitwise OR (`|`). Por ejemplo, para configurar las teclas Shift y Alt podrías usar `pygame.key.set_mods(KMOD_SHIFT | KMOD_ALT)`.

- `pygame.key.set_repeat`: si abres tu editor de texto favorito y mantienes presionada una letra, verá que después de un breve retraso la tecla comienza a repetirse, lo cual es útil si desea ingresar un carácter varias veces sin presionar y soltar. Puedes pedirle a Pygame que te envíe eventos `KEY_DOWN` repetidos con la función `set_repeat`, que toma un valor para el retraso inicial antes de que se repita una tecla y un valor para el retraso entre teclas repetidas. Ambos valores están en milisegundos (1000 milisegundos en un segundo). Puede desactivar la repetición de teclas llamando a `set_repeat` sin parámetros.

- `pygame.key.name`: esta función toma una `KEY_constant` y devuelve una cadena descriptiva para ese valor. Es útil para depurar cuando el código se está ejecutando y sólo podemos ver el valor de una `KEY_constant` y no su nombre. Por ejemplo, si obtengo un evento `KEY_DOWN` para una tecla con un valor de 103, puedo usar `key.name` para imprimir el nombre de la clave (que en este caso es “g”).

4.2. Movimiento direccional con teclas

Se puedes mover un objeto en la pantalla con el teclado asignando una tecla para arriba, abajo, izquierda y derecha. Cualquier tecla pueden ser usada para movimiento direccional, pero las teclas más obvias para usar son las teclas de cursor, porque están diseñadas para movimiento direccional y están colocados en el lugar correcto para operar con una mano. Los aficionados a los shooters en primera persona también están acostumbrados a utilizar las

teclas W, A, S y D para desplazarse. Para convertimos las pulsaciones de teclas en movimientos direccionales, se necesita crear un vector de rumbo que apunte en la dirección que queremos ir. Si sólo una de las cuatro teclas de dirección está presionado, el vector de rumbo es bastante simple. En la siguiente tabla 4.2 se muestran los cuatro vectores de dirección básicos.

Table 4.2. Cuatro vectores de dirección básicos:

Dirección	Vector
Left	-1, 0
Right	+1, 0
Up	0, -1
Down	0, 1

Además del movimiento horizontal y vertical, se necesita que el usuario pueda moverse en dirección diagonal, presionando dos teclas al mismo tiempo. Por ejemplo, si se presionan las teclas Arriba y Derecha, el sprite debería moverse en diagonal hacia la parte superior derecha de la pantalla. Podemos crear este vector diagonal sumando dos de los vectores simples. Si sumamos Arriba (0.0, -1.0) y Derecha (1.0, 0.0), obtenemos (1.0, -1.0), que apunta hacia arriba, pero no podemos usar esto como vector de rumbo porque ya no es un vector unitario (longitud 1). Si tuviéramos que usarlo como vector de rumbo, encontraríamos que nuestro sprite se mueve más rápido en diagonal que en vertical u horizontalmente, lo cual no es tan útil.

Antes de usar nuestro rumbo calculado, debemos convertirlo nuevamente en un vector unitario normalizándolo, lo cual nos da un rumbo de aproximadamente (0.707, -0.707) (Ver Figura 4.1 para obtener una representación visual de la diagonal).

El siguiente código implementa este movimiento direccional. Al ejecutarlo ves un objeto que se mueve horizontal o verticalmente presionando cualquiera de las teclas del cursor, o en diagonal presionando dos teclas del cursor en combinación. Si ejecuta el código sin modificar, encontrará un error generado por intentar dividir algo por cero. Puedes manejarlo dentro con un método de normalización:

```

movimiento_ import pygame
teclas.py   from pygame.locals import *
            from sys import exit

fondo_image_filename = 'pygame.jpg'
sprite_image_filename = 'nave.png'

```

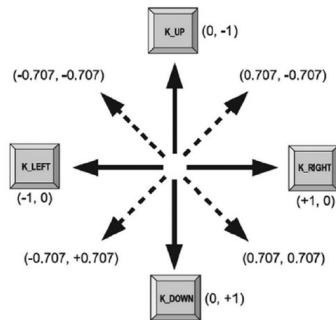


Figura 4.1: Vectores diagonales combinando vectores simples.

```
pygame.init()
screen = pygame.display.set_mode((640, 480), 0, 32)
fondo = pygame.image.load(fondo_image_filename).convert()
sprite = pygame.image.load(sprite_image_filename).convert_alpha()
clock = pygame.time.Clock()
sprite_pos = Vector2(200, 150)
sprite_speed = 300
pygame.display.set_caption('Movimiento con teclas')

while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            exit()

    pressed_keys = pygame.key.get_pressed()
    key_direction = pygame.Vector2(1, 1)
    if pressed_keys[K_LEFT]:
        key_direction.x = -1
    elif pressed_keys[K_RIGHT]:
        key_direction.x = +1
    if pressed_keys[K_UP]:
        key_direction.y = -1
    elif pressed_keys[K_DOWN]:
        key_direction.y = +1

    # Evitar error al dividir por cero
    if key_direction == (0,0):
        key_direction = (0,0)
    elif key_direction != (0,0):
        key_direction = key_direction.normalize()

    screen.blit(fondo, (0,0))
    screen.blit(sprite, (sprite_pos.x, sprite_pos.y))
    time_passed = clock.tick(30)
    time_passed_seconds = time_passed / 1000.0
    sprite_pos += key_direction * sprite_speed * time_passed_seconds
    pygame.display.update()
```

En el script se evita dividir por cero al normalizar, lo tratamos con el

condicional `if ... elif`, poco de habilidad al calcular el vector de dirección. Establece el componente `x` en `-1` o `+1` si se presiona `K_LEFT` o `K_RIGHT` y la componente `y` a `-1` o `+1` si se presiona `K_UP` o `K_DOWN`. Esto nos da el mismo resultado que sumar dos de los vectores con dirección horizontal y vertical simples. Si alguna vez observa un atajo matemático que te permite hacer algo con menos código, no dudes en probarlo.

Es posible que hayas notado que solo se utilizan ocho vectores para este movimiento vectorial. Si tuviéramos que precalcular estos vectores e insertarlos directamente en el código, podríamos reducir la cantidad de trabajo hecho al ejecutar el script.

Vale la pena hacerlo como ejercicio si te gustan los desafíos, pero dado que calcular el vector de dirección solo se realiza una vez por cuadro, acelerarlo no hará ninguna diferencia notable en la velocidad de fotogramas. Esté atento a situaciones como esta; reduciendo la cantidad de trabajo que el juego debe hacer para crear un marco que se llama optimización y se vuelve más importante cuando hay mucha acción en marcha.

4.2.1. Movimiento rotacional con teclas

Hasta ahora tenemos ocho direcciones para el movimiento, sin embargo es un poco artificial porque no se ven muchas cosas moviéndose así en la vida real. La mayoría de los objetos móviles pueden girar libremente, pero moverse en la dirección a la que apuntan, o quizás hacia atrás, pero definitivamente en más de ocho direcciones de la brújula. Todavía podemos usar las mismas teclas Arriba, Abajo, Izquierda y Derecha para simule esto, pero tenemos que cambiar lo que controlan las teclas. Lo que queremos es tener teclas para izquierda y derecha que controlan la rotación y teclas para el movimiento hacia adelante y hacia atrás, lo que le daría a nuestro sprite la capacidad de moverse en cualquier dirección.

En el siguiente script se usa exactamente el mismo conjunto de teclas para el movimiento de un sprite alrededor de la pantalla, pero con movimiento de rotación libre.

```
movimiento import pygame
_rotacion.py from pygame.locals import *
from sys import exit
from math import *

fondo_image_filename = 'pygame.jpg'
sprite_image_filename = 'nave.png'
pygame.init()
```

```

screen = pygame.display.set_mode((640, 480), 0, 32)
fondo = pygame.image.load(fondo_image_filename).convert()
sprite = pygame.image.load(sprite_image_filename).convert_alpha()
clock = pygame.time.Clock()
sprite_pos = pygame.Vector2(200, 150)
sprite_speed = 300
sprite_rotation = 0
sprite_rotation_speed = 360 # Degrees per second
pygame.display.set_caption('Rotación con teclas')
while True:
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            exit()

    pressed_keys = pygame.key.get_pressed()
    rotation_direction = 0.
    movement_direction = 0.
    if pressed_keys[K_LEFT]:
        rotation_direction = +1.0
    if pressed_keys[K_RIGHT]:
        rotation_direction = -1.0
    if pressed_keys[K_UP]:
        movement_direction = -1.0
    if pressed_keys[K_DOWN]:
        movement_direction = +1.0

    screen.blit(fondo, (0,0))
    rotated_sprite = pygame.transform.rotate(sprite, sprite_rotation)
    w, h = rotated_sprite.get_size()
    sprite_draw_pos = pygame.Vector2(sprite_pos.x-w/2, sprite_pos.y-h/2)
    screen.blit(rotated_sprite, (sprite_draw_pos.x, sprite_draw_pos.y))
    time_passed = clock.tick()
    time_passed_seconds = time_passed / 1000.0
    sprite_rotation += rotation_direction * sprite_rotation_speed * time_passed_seconds
    # Cálculos
    heading_x = sin(sprite_rotation*pi/180.0)
    heading_y = cos(sprite_rotation*pi/180.0)
    heading = pygame.Vector2(heading_x, heading_y)
    heading *= movement_direction
    sprite_pos += heading * sprite_speed * time_passed_seconds
    pygame.display.update()

```

El código funciona de manera similar al ejemplo anterior, pero calcula el encabezado de manera diferente. En lugar de crear un vector de dirección a partir de la tecla que se presione, creamos la dirección a partir de la rotación del sprite (almacenado en la variable `sprite_rotation`). Es este valor el que modificamos al presionar la tecla; cuando se presiona la tecla Derecha, agregamos rotación al sprite, girándolo en esa dirección. Y cuando se pulsa la tecla izquierda, restamos rotación al sprite para girarla en sentido contrario.

Para calcular el vector de dirección a partir de la rotación, debemos calcular el seno y el coseno del ángulo, lo cual podemos hacer con las funciones `sin()` y `cos()` que se encuentran en el módulo de matemáticas. La función

`sin()` calcula la componente x, y la función `cos()` calcula el componente y. Ambas funciones toman un ángulo en radianes (hay $2 * \pi$ radianes en un círculo, 360 grados), debido a que en el script `movimiento_rotacion.py` se usan grados, deben convertirse la rotación a radianes multiplicando por π y dividiendo por 180. Después de ensamblar el vector de dirección de los dos componentes, para usarse como antes para dar el movimiento basado en el tiempo.

Antes de que el código muestre el sprite se gira visualmente para que podamos ver en qué dirección está orientado. Esto se hace con la función del módulo `pygame.transform` llamada `rotate` (`pygame.transform.rotate(sprite, sprite_rotation)`), que toma una superficie más un ángulo y devuelve la nueva superficie que contiene el objeto girado.

Un problema con esta función es que el sprite devuelto puede no tener las mismas dimensiones que el original (consulte la Figura 4.2, por lo que no podemos convertirlo en el original misma coordenada que el objeto sin rotar o se dibujará en la posición incorrecta en la pantalla; una manera de trabajar esto es dibujar el sprite en una posición que coloque el centro de la imagen del sprite debajo del Posición del sprite en la pantalla. De esa manera, el sprite estará en la misma posición en la pantalla independientemente del tamaño de la superficie rotada.



Figura 4.2: Al girar una superficie cambia su tamaño.

4.3. Implementación del control con el mouse

El mouse existe desde hace casi tanto tiempo como el teclado. El diseño del mouse no ha cambiado mucho por días, los dispositivos se volvieron un poco más ergonómicos (con forma de mano), pero el diseño siguió siendo el

mismo. Los ratones clásicos tienen una bola recubierta de goma debajo que rueda sobre el escritorio o la alfombrilla del ratón. El movimiento de la pelota es captado por dos rodillos dentro del mouse que están en contacto con la pelota.

Hoy en día, casi todos los mouse son láser, lo que ofrece mayor precisión. Atrás quedaron los días en los amigos podían quitarnos la bola del mouse como una broma, pero ahora podemos pegar papel sobre el orificio del láser para hacer la broma divertida.

A medida que pasa el tiempo, cada vez más mouse tienen también varios botones. También hay muchos mouse con muchos botones diferentes. Los juegos suelen utilizar estas innovaciones de mouses. Los botones adicionales siempre son útiles para un acceso rápido a los controles del juego, y la rueda del mouse podría haberse diseñado para cambiar de arma en primera persona ¡tirador! Y, por supuesto, la precisión adicional de los mouses láser siempre resulta útil a la hora de eliminar enemigos con un rifle de francotirador.

4.3.1. Movimiento rotacional con el mouse

Ya has visto que dibujar el cursor del ratón en la pantalla es bastante sencillo, sólo necesitas obtener las coordenadas del mouse de un evento `MOUSEMOTION` o directamente con la función `pygame.mouse.get_pos`. Cualquiera de los métodos está bien si sólo desea mostrar el cursor del mouse, pero el movimiento del mouse también puede usarse para controlar algo que no sea una posición absoluta, como girar o mirar hacia arriba y hacia abajo en un juego 3D. En este caso, no podemos usar la posición del mouse directamente porque las coordenadas serían restringido a los bordes de la pantalla, y no queremos que el jugador esté restringido en cuántas veces gira a la izquierda o a la derecha. En estas situaciones, necesitamos obtener el movimiento relativo del mouse, a menudo llamado `mouse mickeys`, que simplemente significa qué tan lejos se ha movido el mouse desde el cuadro anterior.

En el script `rotacion_mouse.py` se agrega movimiento de rotación a mouse al movimiento del sprite. Además de las teclas del cursor, el objeto rotará cuando el ratón se mueve hacia la izquierda o hacia la derecha.

```
rotacion      import pygame
_mouse.py     from pygame.locals import *
              from sys import exit
              from math import *

fondo_image_filename = 'pygame.jpg'
sprite_image_filename = 'nave.png'
```

```

pygame.init()
screen = pygame.display.set_mode((640, 480), 0, 32)
fondo = pygame.image.load(fondo_image_filename).convert()
pygame.display.set_caption('Movimiento y rotación con mouse')
sprite = pygame.image.load(sprite_image_filename).convert_alpha()
clock = pygame.time.Clock()

# Desactivar el cursor del mouse
pygame.mouse.set_visible(False)
# PyGame tiene el control total el mouse
pygame.event.set_grab(True)
sprite_pos = pygame.Vector2(200, 150)

sprite_speed = 300.
sprite_rotation = 0.
sprite_rotation_speed = 360. # Degrees per second

while True:
    for evento in pygame.event.get():
        if evento.type == KEYDOWN:
            if evento.key==K_q:
                pygame.quit()
                exit()

    pressed_keys = pygame.key.get_pressed()
    pressed_mouse = pygame.mouse.get_pressed()
    rotation_direction = 0.
    movement_direction = 0.
    rotation_direction = pygame.mouse.get_rel()[0] / 3.
    if pressed_keys[K_LEFT]:
        rotation_direction = +1.
    if pressed_keys[K_RIGHT]:
        rotation_direction = -1.
    if pressed_keys[K_UP] or pressed_mouse[0]:
        movement_direction = +1.
    if pressed_keys[K_DOWN] or pressed_mouse[2]:
        movement_direction = -1.
    screen.blit(fondo, (0,0))
    rotated_sprite = pygame.transform.rotate(sprite, sprite_rotation)
    w, h = rotated_sprite.get_size()
    sprite_draw_pos = pygame.Vector2(sprite_pos.x-w/2, sprite_pos.y-h/2)
    screen.blit(rotated_sprite, (sprite_draw_pos.x, sprite_draw_pos.y))
    time_passed = clock.tick()
    time_passed_seconds = time_passed / 1000.0
    sprite_rotation += rotation_direction * sprite_rotation_speed * time_passed_seconds
    heading_x = sin(sprite_rotation*pi/180.)
    heading_y = cos(sprite_rotation*pi/180.)
    heading = pygame.Vector2(heading_x, heading_y)
    heading *= movement_direction
    sprite_pos+= heading * sprite_speed * time_passed_seconds
    pygame.display.update()

```

Observe que para salir debe usar la tecla `q`, debido a que el mouse se está usando para el movimiento del sprite; y el área de acción del mouse se restringe al área física de la ventana (pantalla). Esto se hace con las siguientes

dos líneas:

```
pygame.mouse.set_visible(False)
pygame.event.set_grab(True)
```

La llamada a `set_visible(False)` desactiva el cursor del mouse y la llamada a `set_grab(True)` le da todo el control sobre mouse a PyGame. Un efecto secundario de esto es que no puedes usar el mouse para cerrar la ventana, por lo que debe proporcionar una forma alternativa de cerrar el script.

Usar el mouse de esta manera también dificulta el uso de otras aplicaciones, por lo que es normalmente se utiliza mejor en modo de pantalla completa.

La siguiente línea obtiene los `mickeys` del mouse para el eje x (en el índice [0]). Los divide por 5, porque descubrí que usar el valor directamente rotaba el sprite demasiado rápido. Puede ajustar este valor a cambie la sensibilidad del mouse (cuanto mayor sea el número, menos sensible será el control del mouse). Juegos a menudo permiten que el jugador ajuste la sensibilidad del mouse en un menú de preferencias.

```
rotation_direction = pygame.mouse.get_rel()[0] / 5.
```

Además de rotar con el mouse, los botones se usan para mover el sprite hacia adelante (izquierdo del mouse). botón) y hacia atrás (botón derecho del mouse). La detección de botones presionados del mouse es similar a las teclas. La función `pygame.mouse.get_pressed()` devuelve una tupla de tres valores booleanos para el botón izquierdo del ratón, el botón central del ratón y el botón derecho del ratón. Si alguno es Verdadero, entonces el botón correspondiente del mouse se mantuvo presionado en el momento de la llamada.

Una forma alternativa de utilizar esta función es la siguiente, que descomprime la tupla en tres valores:

```
lmb, mmb, rmb = pygame.mouse.get_pressed()
```

Veamos `pygame.mouse` con más detalle. Es otro módulo sencillo, y tiene sólo ocho funciones:

- `pygame.mouse.get_pressed`: devuelve los botones presionados del mouse como una tupla de tres. booleanos, uno para los botones izquierdo, central y derecho del mouse.
- `pygame.mouse.get_rel`: devuelve el movimiento relativo del mouse (o mickeys) como un tupla con el movimiento relativo x e y.

- `pygame.mouse.get_pos`: devuelve las coordenadas del mouse como una tupla de x e y valores.
- `pygame.mouse.set_visible`: cambia la visibilidad del cursor del mouse estándar. Si es falso, el cursor será invisible.
- `pygame.mouse.get_focused`: devuelve True si la ventana de Pygame está recibiendo entrada del mouse. Cuando Pygame se ejecuta en una ventana, solo recibirá la entrada del mouse cuando la ventana está seleccionada y en el frente de la pantalla.
- `pygame.mouse.set_cursor`: configura la imagen estándar del cursor. Esto rara vez es necesario porque se pueden lograr mejores resultados modificando una imagen a la coordenada del mouse.
- `pygame.mouse.get_cursor`: obtiene la imagen estándar del cursor. Ver el punto anterior.

4.3.1.1. Jugabilidad del ratón

El humilde mouse puede permitirte ser bastante creativo en un juego. Incluso un simple cursor puede ser una gran herramienta de juego para juegos de rompecabezas y estrategia, donde el mouse se usa a menudo para seleccionar y soltar elementos del juego en el campo de juego. Estos elementos podrían ser cualquier cosa, desde espejos que reflejan láser hasta ogros sedientos de sangre, dependiendo del juego. No es necesario que el puntero del mouse sea una flecha. Para un juego de DIOS, sería probablemente la mano omnipotente de una deidad o quizás una misteriosa sonda alienígena. El clásico point 'n' click, los juegos de aventuras también hicieron un excelente uso del mouse. En estos juegos, el personaje del juego caminaba a lo que sea en lo que el jugador hizo clic y échale un vistazo. En estos juegos, el personaje del juego caminaba a lo que el jugador hizo clic y écha un vistazo. Si fuera un objeto útil, podría almacenarse para su uso más tarde en el juego.

Para juegos con un control más directo del personaje del jugador, el mouse se usa para rotar o moverlo. Un simulador de vuelo puede usar los ejes x e y del mouse para ajustar el cabeceo y la guiñada del avión de modo que el jugador puede hacer ajustes finos a la trayectoria.

Una forma sencilla de hacer esto en Pygame sería agregar al mouse el ángulo del avión. También me gusta usar el mouse para tiradores (shooters) en primera persona y usar el eje x para girar hacia la izquierda y hacia la derecha, y el eje y para mirar hacia arriba y hacia abajo. Esto parece bastante natural porque las cabezas tienden moverse de esa manera; puedo girar la

cabeza hacia cualquier lado y mirar hacia arriba y hacia abajo. También puedo inclinar la cabeza para de cualquier lado, pero no lo hago muy a menudo.

También puede combinar controles de mouse y teclado. Me gustan especialmente los juegos que usan el teclado para movimiento y el ratón para apuntar. Un tanque, por ejemplo, podría moverse con las teclas del cursor, pero usa el mouse para girar la torreta¹ y poder dispararle a un enemigo sin mirar en la misma dirección. Por supuesto, no estás limitado a utilizar el mouse del mismo modo que otros juegos. Sea tan creativo como usted desee, ¡solo asegúrate de probarlo primero con algunos de tus amigos!

4.4. Implementación del control por palanca de mando

Pendiente...

Los joysticks no se han visto limitados por la necesidad de utilizarlos en otros lugares además de los juegos, y han sido completamente libertad para innovar, por lo que los joysticks modernos tienen un diseño moldeado suave y aprovechan todos los dígitos sobrantes. tienen a su disposición. Aunque la palabra joystick se utiliza a menudo para referirse a un controlador de juego de cualquier tipo, describe más técnicamente una palanca de vuelo utilizada en simulaciones de vuelo. Hoy en día, el joypad es el más popular entre los jugadores y, a menudo, viene con las consolas. Encajan cómodamente en dos manos y tienen muchos botones, además de un pad direccional y dos operados con el pulgar palos analógicos. Piensa en tus controladores de Xbox o PlayStation. Mucha gente conecta su controlador Xbox para su computadora.

El módulo de joystick de Pygame admite todo tipo de controladores con la misma interfaz y no distinguir entre los distintos dispositivos disponibles. Esto es bueno porque no queremos tener que hacerlo. ¡Escribe código para cada controlador de juego que existe!

Conceptos básicos del mando

¹Una torreta es usualmente una plataforma rotativa que puede ser montada en un edificio fortificado o estructura tales como baterías de costa, sobre un tanque, buque de guerra, o avión.

Comencemos mirando el módulo `pygame.joystick`.

4.5. Resumen

Hay tres módulos de controles en PyGame: `pygame.keyboard`, `pygame.mouse` y `pygame.joystick`, entre ellos, puedes admitir cualquier dispositivo que el jugador quiera usar en un juego. Puede darle al jugador la opción para seleccionar el método de control.

A veces, los controles influyen instantáneamente en el personaje del jugador, de modo que obedecerá exactamente lo que el jugador le dice que haga. Los juegos de disparos clásicos eran así; presionaste hacia la izquierda, ella nave instantáneamente giró hacia la izquierda y viceversa. Sin embargo, para un poco más de realismo, los controles deberían afectar al personaje del jugador indirectamente al aplicando fuerzas como empuje y rotura.

Ahora que sabes cómo leer información de los controladores de juegos, los siguientes capítulos le enseñarán cómo utilizar esta información para manipular objetos de forma más realista los mundos de juego.

Los controles de un juego son la forma en que el jugador interactúa con el mundo del juego, porque no podemos ingresar al juego, estilo Matrix, los controles deben sentirse lo más naturales posible. Al considerar el control como métodos para un juego, es una buena idea observar cómo se juegan juegos similares. Los controles no varían mucho para juegos de un género similar. Esto no se debe a que a los diseñadores de juegos les falte imaginación; es solo que los jugadores esperan que los juegos funcionen de manera similar. Si utilizas un control más creativo métodos, ¡prepárate para justificárselo a los jugadores u ofrecerles una alternativa más estándar!

En el próximo capítulo exploraremos el tema de la inteligencia artificial y aprenderá cómo agregar personajes no jugadores a un juego.

Dos proyectos de juegos los alienígenas y fútbol...

SEGUNDA PARTE: Mis Proyectos

Ahora conocemos lo suficiente sobre Python y PyGame para comenzar a crear proyectos interactivos y significativos. Al crear sus propios proyectos le enseñará nuevas habilidades y solidificará su comprensión de los conceptos introducidos en la primera parte.

En la segunda parte presentamos tres proyectos, que usted puede elegir cualquiera o todos estos proyectos en el orden que desee.

A continuación se incluye una breve descripción de cada proyecto para ayudarle a decidir cuál iniciara primero.

Proyecto 1: Un juego con Python y PyGame

Invasión alienígena:

En el proyecto Invasión alienígena (capítulos 12, 13 y 14), utilizarás el paquete PyGame para desarrollar un juego en 2D. El objetivo del juego es derribar una flota de alienígenas a medida que descienden por la pantalla, en niveles que aumentan en velocidad y dificultad. Al final del proyecto, habrás aprendido habilidades que te permitirán desarrollar tus propios juegos 2D en Pygame.

Usaremos Pygame, una colección de divertidos y poderosos Módulos de Python que gestionan gráficos, animación e incluso sonido, lo que facilita para que puedas crear juegos sofisticados. Con Pygame al manejar tareas como dibujar imágenes en la pantalla, usted Puede centrarse en la lógica de nivel superior de la dinámica del juego.

Mientras creas este juego, también aprenderás a gestionar grandes proyectos que abarcan varios archivos. Refactorizaremos una gran cantidad de

código y administraremos archivos contenidos para organizar el proyecto y hacer que el código sea eficiente.

Proyecto 2: Visualización de datos

Los proyectos de visualización de datos comienzan en el Capítulo ??, donde aprenderá a generar datos y a crear una serie de visualizaciones y funcionales de esos datos usando la librerías `Matplotlib` y `Plotly`.

El Capítulo ?? le enseña a acceder a datos de fuentes en línea e introducirlos en un paquete de visualización para crear gráficos de datos meteorológicos y un mapa de la actividad sísmica global.

Finalmente, el Capítulo 17 le muestra cómo escribir un programa para descargar y visualizar datos automáticamente. Aprender a hacer visualizaciones le permite explorar el campo de la ciencia de datos, que es una de las áreas de programación de mayor demanda en la actualidad.

Proyecto 3: Aplicaciones web

En el proyecto de aplicación web (capítulos 18, 19 y 20), utilizará el paquete Django para crear una aplicación web sencilla que permita a los usuarios llevar un diario sobre diferentes temas que han estado aprendiendo. Los usuarios crearán una cuenta con un nombre de usuario y contraseña, ingresarán un tema y luego harán entradas sobre lo que están aprendiendo. También implementará su aplicación en un servidor remoto para que cualquier persona en el mundo pueda acceder a ella. Después de completar este proyecto, podrá comenzar a crear sus propias aplicaciones web simples y estará listo para profundizar en recursos más completos sobre la creación de aplicaciones con Django.



Cuando se va a construir un proyecto grande o pequeño, es recomendable e importante hacer un plan antes de comenzar a escribir el código, en el plan debe describir la idea en forma clara y las etapas posibles a seguir. Un plan te mantendrá enfocado en el objetivo planteado y le permitirá ir cumpliendo metas para completar el proyecto. Es importante hacer una descripción general del juego; aunque al principio tal vez la descripción no cubra todos los detalles que el juego requiere, pero proporciona una idea clara para empezar a construir el juego.

En el próximo capítulo 5 se desarrolla el juego Invasión Alienígena, un juego donde el jugador controla una nave que inicialmente aparece en la parte inferior central de la pantalla. Para la primera fase de desarrollo, construimos una nave que pueda moverse correctamente hacia la izquierda y derecha cuando el jugador presiona las teclas de flecha, y que dispare balas cuando el jugador presiona la barra espaciadora. Después de configurar este comportamiento, podemos crear los alienígenas o extraterrestres y perfeccionar el juego.

Borrador

Capítulo 5

Invasión Alienígena



En este capítulo, se configura su ambiente de trabajo, y se inicia escribiendo el código para crear una nave espacial que se mueve hacia la derecha y hacia la izquierda, y que dispara balas en respuesta a la entrada del jugador. Luego, en los próximos dos capítulos, se creará una flota de alienígenas que debe destruir el jugador; luego se continúa

refinando el juego, estableciendo límites en la cantidad de naves que puedes usar y se añade un marcador.



Alienígena Invasión abarca varios archivos diferentes, así que es recomendable crear una carpeta digamos `invasion_alien` en su sistema. Asegúrese de guardar todos los archivos del proyecto en esta carpeta para que las declaraciones de importación funcionen correctamente. Además, si se siente cómodo usando el control de versiones, es posible que desee usarlo para este proyecto. Si no ha utilizado el control de versiones antes, consulte el Apéndice D para obtener más información.

InvasionAlien

5.1. Una Clase base para el juego

Iniciamos con la creación de una clase vacía para representar el juego `class InvasionAlien`. Para esto debe crear en su editor de texto un script y guárdelo en la carpeta principal para el juego, digamos `invasión_alien.py`.

En este archivo escriba el siguiente código:

```
invasión
_alien.py

import sys
import pygame

class InvasionAlien:
    """Clase general para gestionar comportamiento del juego."""
    def __init__(self):
        """Inicializa el juego y crea recursos para el juego."""
        pygame.init()

        self.pantalla = pygame.display.set_mode((640, 480))
        pygame.display.set_caption("Invasión Alienígena")

    def run_jugar(self):
        """Inicia el bucle principal del juego."""
        while True:
            # Recuperar los Eventos del teclado y el mouse.
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()

            # Hacer visible la pantalla dibujada más recientemente.
            pygame.display.flip()

if __name__ == '__main__':
    # Crea una instancia de juego y ejecuta el juego.
    ia = InvasionAlien()
    ia.run_jugar()
```

Observe, primero importamos los módulos `sys` y `pygame`. El módulo `pygame` que contiene la funcionalidad que necesitamos para hacer un juego. Y Usamos la herramienta del módulo `sys` para salir del juego cuando el jugador lo desee, haciendo clic en el botón de cerrar la ventana del juego. PyGame cierra y desaparece la ventana.

Invasión Alienígena comienza como una clase llamada `InvasionAlien`. En el método `__init__()`, se incluye la función `pygame.init()`, que inicializa la configuración de fondo que PyGame necesita funcionar correctamente. Luego llamamos a `pygame.display.set_mode()` para crear una ventana de visualización, en la que dibujaremos todos los elementos gráficos del juego. El argumento `(640, 480)` es una tupla que define las dimensiones de la ventana del juego, en este caso el tamaño es `(640, 480)`, que tendrá 640 píxeles de ancho por 480 píxeles de alto. Y le damos nombre a la ventana PyGame con el método `pygame.display.set_caption("Invasión Alienígena")`

Asignamos esta ventana de visualización al atributo `self.pantalla`, por lo que estará disponible en todos los métodos de la clase. El objeto que

asignamos a `self.pantalla` se llama superficie.

La superficie devuelta por `display.set_mode()` representa toda la ventana del juego. Cuando activamos el bucle principal (`while`) de animación del juego, esta superficie se volverá a dibujar en cada paso por el bucle, por lo que se puede actualizar con cualquier cambio provocado por la entrada del usuario.

El juego está controlado por el método `run_jugar()`. Este método contiene un bucle `while` que se ejecuta continuamente. El bucle `while` contiene un bucle `for` (anidado) para detectar y responder a eventos específicos, que por ahora solo interesa el evento `pygame.QUIT`. Luego el código `pygame.display.flip()` gestiona las actualizaciones de la pantalla.

La llamada a `pygame.display.flip()` le dice a `pygame` que haga visible la pantalla dibujada más recientemente. En este caso, simplemente dibuja una pantalla vacía en cada paso por el bucle `while`, borrando la pantalla anterior para que solo sea visible la nueva. Cuando movemos los elementos del juego, `pygame.display.flip()` actualiza continuamente la pantalla para mostrar las nuevas posiciones de los elementos del juego y ocultar las antiguas, creando la ilusión de un movimiento suave.

Al final del archivo, creamos una instancia del juego y luego llamamos a `run_jugar()`. Colocamos `run_jugar()` en un bloque `if` que solo se ejecuta si el archivo se llama directamente. Al ejecutar el archivo `invasion_alien.py`, debería ver una ventana de `pygame` vacía.

Es importante tener en cuenta que si utiliza una pantalla con doble buffer, debe llamar a

```
pygame.display.flip()
```

en lugar de `pygame.display.update()`. Esto hace que la visualización instantánea cambie en lugar de copiar los datos de la pantalla.

En la tabla 2.8.4 se presenta un resumen de la información con las diferencias entre `pygame.display.update()` y `pygame.display.flip()`:

Table 2.8.4. Diferencias entre los métodos `.update()` y `.flip()`

Método	<code>pygame.display.update()</code>	<code>pygame.display.flip()</code>
Funcionalidad	Actualiza solo las partes de la pantalla que han cambiado	Actualiza toda la pantalla, incluyendo áreas que no han cambiado
Rendimiento	Puede ser más eficiente para aplicaciones que requieren una actualización rápida y precisa de solo ciertas áreas de la pantalla	Puede ser más eficiente para aplicaciones que requieren una actualización completa y sincronizada de toda la pantalla
Uso	Se utiliza cuando se necesita actualizar solo ciertas partes de la pantalla, como en juegos que requieren una actualización rápida de solo ciertas áreas	Se utiliza cuando se necesita actualizar toda la pantalla, como en aplicaciones que requieren una actualización completa y sincronizada de toda la pantalla
Efectos visuales	Puede causar artefactos visuales si no se utiliza correctamente, como por ejemplo, si se actualizan áreas de la pantalla que no han cambiado	No causa artefactos visuales, ya que actualiza toda la pantalla de manera sincronizada

5.1.1. Agregamos la velocidad de fotogramas

Idealmente, los juegos deberían ejecutarse a la misma velocidad o velocidad de fotogramas en todos los sistemas. Para controlar la velocidad de fotogramas de un juego que puede ejecutarse en múltiples sistemas es una cuestión compleja, pero Pygame ofrece una forma relativamente sencilla de lograr este objetivo, el método `pygame.time.Clock()`. Lo agregamos en el método `__init__()` después de `pygame.init()`, y aseguraremos de que el reloj marque una vez en cada paso por el bucle principal. Cada vez que el bucle se procese más rápido que la velocidad que definimos, Pygame calculará la cantidad de tiempo correcta para pausar el juego y se ejecute a una velocidad constante:

```
class InvasionAlien:
    """Clase general para gestionar comportamiento del juego."""
    def __init__(self):
        """Inicializa el juego y crea recursos para el juego."""
        pygame.init()
        self.clock = pygame.time.Clock()
        --recorte--
```

Después de inicializar pygame, creamos un objeto `self.clock`, con el módulo `pygame.time.Clock()`. Luego dejamos que el reloj avance al final del bucle while en `run_jugar()`:

```
def run_jugar(self):
    """Inicia el bucle principal del juego."""
    while True:
        # Recuperar los Eventos del teclado y el mouse.
        --recorte--
        pygame.display.flip()
        self.clock.tick(60)
```

El método `tick()` toma la velocidad de fotogramas del juego como el argumento; en este caso usamos un valor de 60, por lo que Pygame ejecutara el bucle exactamente 60 veces por segundo.



El reloj de Pygame debería ayudar a que el juego se ejecute de manera consistente en la mayoría de los sistemas. Si hace el juego se ejecuta de manera menos consistente en tu sistema, puedes probar diferentes valores de velocidad. Si no puede encontrar una buena velocidad de fotogramas en su sistema, puede probar con velocidad de fotograma reloj redactar por completo y ajusta la configuración del juego para que funcione bien en tu sistema.

5.1.2. Configurar el color de fondo

Pygame crea una pantalla negra de forma predeterminada, pero eso es muy alegre. Podemos establecer un color de fondo agradable. El valor del color (230, 230, 230) mezcla cantidades iguales de rojo, azul y verde, lo que produce un color de fondo gris claro. Este procedimiento lo haremos al final del método `__init__()`, donde asignamos este color a la variable `self.color_fondo`:

```
class InvasionAlien:
    """Clase general para gestionar comportamiento del juego."""
    def __init__(self):
        """Inicializa el juego y crea recursos para el juego."""
        pygame.init()
        --recorte--
        pygame.display.set_caption("Invasión Alienígena")
        # Configurando color de fondo
        self.color_fondo = (170,238,187)
    def run_game(self):
        --recorte--

        # Colorea la pantalla en cada paso del bucle
        self.pantalla.fill(self.color_fondo)
        # Actualiza. Hacer visible la pantalla dibujada más recientemente.
        pygame.display.flip()
        self.clock.tick(60)
```

Y rellenamos la pantalla con el color de fondo usando el método `fill()` de la forma siguiente `self.pantalla.fill(self.color_fondo)`, que actúa sobre una superficie y toma un solo argumento el color.

5.2. Crear una clase de configuración

Cada vez que introducimos nuevas funciones en el juego, normalmente también creamos algunas configuraciones. Podemos escribir un módulo llamado configuración que contenga una clase llamada Configuración para almacenar todos esos valores en un solo lugar, en lugar de agregar configuraciones a lo largo del código. Este enfoque nos permite trabajar con el objeto configuración cada vez que necesitemos acceder a una configuración individual; además facilita la modificación, apariencia y el comportamiento del juego a medida que nuestro proyecto crece. Ahora para modificar el juego cambiaremos los valores relevantes en `configuración.py`, en lugar de buscar diferentes configuraciones a lo largo del proyecto.

Debemos crear el nuevo archivo llamado `configuración.py` y guardarlo en la carpeta principal `invasión_alienígena`, con el siguiente código:

```
configurar.py class Configuracion:
    """Almacena todas las configuraciones para Invasión Alienígena."""
    def __init__(self):
        """Inicializa las configuraciones del juego"""
        # Configuración de pantalla
        self.pantalla_width = 800
        self.pantalla_height = 600
        self.Color_fondo = (170,238,187)
```

Para crear una instancia de configuración en el proyecto y usarla para acceder a nuestra configuración, necesitamos modificar el módulo `invasión_alien.py` de la siguiente manera:

```
invasión    import sys
_alien.py   import pygame
            from configurar import Configuracion
            class InvasionAlien:
                """Clase general para gestionar comportamiento del juego."""
                def __init__(self):
                    """Inicializa el juego y crea recursos para el juego."""
                    pygame.init()
                    self.clock = pygame.time.Clock()
                    self.configuracion = Configuracion()
                    self.pantalla= pygame.display.set_mode(
                        (self.configurar.pantalla_width,
                         self.configurar.pantalla_height))
                    pygame.display.set_caption("Invasión Alienígena")
```



```

def run_jugar(self):
    """Inicia el bucle principal del juego."""
    while True:
        --recorte--
        # Dibuja la pantalla durante cada paso del bucle
        self.pantalla.fill(self.color_fondo)
        #self.pantalla.blit(self.fondo, (0,0))

        self.pantalla.fill(self.settings.Color_fondo)
        # Hacer visible la pantalla dibujada más recientemente.
        pygame.display.flip()
        --recorte--

```

Del módulo `configurar.py` se importa la clase `Configuracion()`; y después de la llamada de `self.clock = pygame.time.Clock()` en método `__init__()`, creamos una instancia y se la asignamos a la variable `self.configuracion = Configuaracion()`.

Al ejecutar el script `invasion_alien.py` no verás ningún cambio, porque lo único que hemos hecho es mover las configuraciones que ya estábamos usando al archivo `configuar.py`.

Ahora estamos listos para comenzar a agregar nuevos elementos a la pantalla.

5.3. Agregar la imagen de la nave

Para dibujar la nave del jugador en la pantalla, cargaremos una imagen y luego usaremos el método `blit()` de Pygame para dibujar la imagen.

Puedes usar casi cualquier tipo de archivo de imagen en tu juego, pero es el más fácil cuando usas un archivo de mapa de bits (`.bmp`) porque Pygame carga mapas de bits de forma predeterminada. Sin embargo, puedes configurar PyGame para usar otros tipos de archivos, algunos tipos de archivos dependen de ciertas bibliotecas de imágenes que deben estar instaladas en su computadora. La mayoría de las imágenes que encontrarás están en formatos `.jpg` o `.png`, pero puedes convertirlas a mapas de bits usando herramientas como Photoshop, GIMP o Paint.



Cuando elijas ilustraciones para tus juegos, asegúrate de prestar atención Se re- a la concesión de licencias. La forma más segura y económica de empezar es utilizar gráficos con licencia libre que puede usar y modificar desde un sitio web como <https://opengameart.org>.

comienda un archivo con un fondo transparente o sólido que se puede reemplazar con cualquier color de fondo, usando un editor de imágenes. Los juegos se verá mejor si el color de fondo de la imagen coincide con el color de fondo de tu juego. Alternativamente, puedes hacer coincidir color de fondo de tu juego con el fondo de la imagen.



Figura 5.1: La nave para Invasión Alienígena.

En este caso, el color de fondo del archivo coincide con la configuración que estamos usando en este proyecto; y usaremos el archivo `nave.bmp` (Figura 5.1), que es un archivo manipulado para ilustrar el juego.

Ahora debemos crea una carpeta llamada imágenes dentro de la carpeta principal, y guardar el archivo `nave.bmp` en esa carpeta.

5.3.1. Una clase `Nave()`

Después de seleccionar la imagen para la nava, necesitamos mostrarla a en pantalla. Para ello creamos un módulo llamado `nave.py` que contiene una clase que llamaremos `Nave()`; en esta clase se maneja el comportamiento de la nave en el juego.

Pygame es eficiente porque te permite tratar todos los elementos del juego como rectángulos (`rect`), incluso si no tienen exactamente la forma de rectángulos. Tratar un elemento como un rectángulo, es eficaz porque los rectángulos son formas geométricas simples. Cuando PyGame necesita determinar si dos elementos del juego han chocado, por ejemplo, puede hacerlo más rápidamente si trata cada objeto como un rectángulo. Este enfoque suele funcionar lo suficientemente bien. En este juego notará que no estamos

trabajando con la forma exacta de cada elemento del juego. En esta clase trataremos la nave y la pantalla como rectángulos.

```
nave.py import pygame
class Nave:
    """Una clase que maneja el comportamiento de la nave"""
    def __init__(self, ia_jugar):
        """Inicia la nave y configura la posicion inicial."""
        self.pantalla = ia_jugar.pantalla
        self.pantalla_rectang = ia_jugar.pantalla.get_rect()
        # Carga la imagen y obtiene su rectangulo
        self.imagen = pygame.image.load('imagenes/nave.bmp')
        self.rectang = self.imagen.get_rect()

        # comienza cada nave nueva en centro de la pantalla
        self.rectang.mi_botom = self.pantalla_rectang.midbottom

    def blitme(self):
        """Dibuja la nave en la posicion actual"""
        self.pantalla.blit(self.imagen, self.rect)
```

En el script importamos el módulo `pygame` antes de definir la clase. Luego se define la clase `Nave()`, y en el método `__init__()` se definen dos parámetros: `self` y una referencia a la instancia `ia_jugar` actual de la clase `InvasionAlien`. Esto le dará la clase `Nave()` acceso a todos los recursos del juego definidos en `InvasionAlien`. Cuando asignamos la pantalla a un atributo de `Nave()` (`self.pantalla = ia_jugar.pantalla`), podemos acceder fácilmente a todos los métodos de esta clase. Accedemos al atributo `rect` de la pantalla usando el método `get_rect()` y se asigna a `self.pantalla_rectang`. Con esto nos permite colocar la nave en la ubicación correcta en la pantalla.

Para cargar la imagen, llamamos a `pygame.image.load()` y le damos la ubicación de la imagen de nuestra nave. Esta función devuelve una superficie que representa la nave, que asignamos a la propia imagen. Cuando se carga la imagen, llamamos a `get_rect()` para acceder al atributo `rect` de la superficie de la nave, para que luego podamos usarlo para colocar la nave.

Cuando se trabaja con objetos `rect`, puedes usar coordenadas x y y de los bordes superior, inferior, izquierdo y derecho del rectángulo, como así como el centro, para colocar el objeto (ver en sección 2.4). Puede establecer cualquiera de estos valores para ubicar la posición actual del objeto `rect`.

Cuando estás centrando un elemento del juego, es recomendable trabajar con los atributos `center`, `centerx` o `centery` de un objeto `rect`. Cuando está trabajando en un borde de la pantalla, puede trabajar con la parte superior, inferior, izquierda o atributos `rect`. También hay atributos que combinan estas propiedades, como medio inferior, medio superior, medio izquierdo y

medio derecho. Cuando estás ajustando para colocar horizontal o vertical objeto `rect`, puedes usar los atributos `x` e `y`, que son las coordenadas `x` e `y` de su esquina superior izquierda.

Estos atributos te evitan tener que hacer cálculos que los desarrolladores de juegos antes tenías que hacerlo manualmente y los usarás con frecuencia.



En Pygame, el origen (0, 0) está en la esquina superior izquierda de la pantalla y las coordenadas aumentan a medida que bajas y va hacia la derecha. En una pantalla de 1200×800, el origen está en la esquina superior izquierda y la esquina inferior derecha tiene las coordenadas (1200, 800). Estas coordenadas se refieren a la ventana del juego, no a la pantalla física.

La nave la ubicaremos en la parte inferior central de la pantalla. Para ello hacemos que el valor de `self.rectang.midbottom` coincida con el atributo `midbottom` del objeto `rect` en la pantalla (ver en sección 2.4):

```
self.rectang.midbottom = self.pantalla_rectang.midbottom
```

Pygame usa los atributos del objeto `rect` para posicionar la imagen de la nave para que esté centrado horizontalmente y alineado con la parte inferior de la pantalla. La otra forma sería hacer los cálculos de forma manual.

Finalmente, definimos el método `blitme()`, que dibuja la imagen en la pantalla en la posición especificada por `self.rectang`.

5.3.1.1. Dibujando la nave en la pantalla

Ahora actualicemos en módulo `invasión_alien.py` para que cree la nave y la dibuje en la pantalla.

```
invasión    import sys
_alien.py   import pygame
           fondo_image_filename = 'pygame.jpg'

           from configurar import Configuracion
           from nave import Nave

           class InvasionAlien:
               """Clase general para gestionar comportamiento del juego."""
               def __init__(self):
                   class InvasionAlien:
                       --recorte--
                       pygame.display.set_caption("Invasión Alienígena")
                       self.fondo = pygame.image.load(
                           self.configuracion.fondo_image_filename).convert()
                       self.nave = Nave(self)

               def run_game(self):
                   --recorte--
                   self.pantalla.blit(self.fondo,(
```

```

        (self.configuracion.pantalla_width-640)/2,
        (self.configuracion.pantalla_height-480)/2))
self.nave.blitme()
# Actualiza. Hace visible la pantalla dibujada más recientemente
pygame.display.flip()
--recorte--

```

Puede ver que importamos la clase `Nave()` de `nave.py`, y luego creamos una instancia de `Nave()` después de que se haya creado la pantalla PyGame. La llamada a `Nave()` requiere un argumento: una instancia de la clase `InvasionAlien`. En este caso el argumento `self`, se refiere a la instancia actual de `Invasión alienígena`. Este parámetro le da (`Nave(self)`) acceso a los recursos del juego, como el objeto de pantalla.

Asignamos esta instancia de `Nave()` a la variable `self.nave = Nave(self)`. Después de llenar el fondo, dibujamos la nave en la pantalla llamando a `self.nave.blitme()`, para que la nave aparezca encima del fondo.

Al ejecutar `invasión_alien.py`, deberías ver una pantalla de juego con la nave en la parte inferior central, como se muestra en la Figura 5.2.



Figura 5.2: La nave en la parte inferior central de la pantalla.

5.4. Refactorización

Métodos auxiliares `_chequear_eventos()` y `_actualizar_pantalla()`

En proyectos grandes, a menudo se refactorizara el código que hayas escrito antes de agregar más código. La refactorización simplifica la estructura del código que ya tienes escrito, lo que facilita la construcción.

En esta sección, dividimos el método `run_jugar()` que se está haciendo largo, en dos métodos auxiliares. Un método ayuda o auxiliar funciona dentro de una clase pero no debe ser utilizado por código fuera de la clase.

En Python, un único guión bajo al inicio del nombre de un método indica que es un método auxiliar (helper).

5.4.1. El método `_chequear_eventos()`

Moveremos el código que administra los eventos a un método auxiliar separado llamado `_chequear_eventos()`. Esto simplificará `run_jugar()` y aislará el bucle que gestiona de los eventos. Aislar el bucle de eventos le permite administrar eventos por separado de otros aspectos del juego, como la actualización de la pantalla.

En la clase `InvasionAlien` se crea el nuevo `_chequear_eventos()`, que sólo afecta al código del método `run_jugar()`:

```
invasión  --recorte--
_alien.py  def run_jugar(self):
            """Inicia el bucle principal del juego."""
            while True:
                self._check_events()
            --recorte--

            def _chequear_eventos(self):
                """Responde a las teclas presionada y a eventos del mouse"""
                for event in pygame.event.get():
                    if event.type == pygame.QUIT:
                        pygame.quit()
                        sys.exit()
```

Creamos un nuevo método `_chequear_eventos()` y movemos las líneas de código que verifican si el jugador ha hecho clic para cerrar la ventana a este nuevo método.

Para llamar a un método dentro de una clase, se usa la notación de puntos con la variable `self` y el nombre del método (`self._chequear_eventos()`). Llamamos al método dentro del bucle `while` en el método `run_jugar()`.

5.4.2. El método `_actualizar_pantalla()`

Para simplificar aún más `run_jugar()`, moveremos el código para actualizar la pantalla a un método separado llamado `_actualizar_pantalla()`:

```
--recorte--
def run_game(self):
    """Inicia el bucle principal del juego."""
```

```

while True:
    self._chequear_eventos()
    self._actualizar_pantalla()
    self.clock.tick(60)
def _chequear_eventos(self):
    --recorte--
def _actualizar_pantalla(self):
    """Actualiza images sobre la pantalla, y flip actualiza la nuave pantalla."""
    self.pantalla.fill(self.configuracion.color_fondo)
    self.nave.blitme()
    pygame.display.flip()

```

Hemos movimos el código que dibuja el fondo de pantalla y la nave al método `_actualizar_pantalla()`.

Ahora el cuerpo del bucle principal en `run_jugar()` es mucho más simple. Es fácil ver que estamos buscando nuevos eventos, actualizando la pantalla y marcando el reloj en cada paso por el bucle.

Si ya has creado varios juegos, probablemente comenzarás dividiendo tu código en métodos como estos. Pero si nunca ha abordado un proyecto como este, probablemente al principio no sabrá exactamente cómo estructurar su código. Este enfoque le da una idea de un proceso de desarrollo realista: comienza escribiendo su código de la manera más simple posible y luego lo refactoriza a medida que su proyecto se vuelve más complejo.

Ahora que hemos reestructurado el código para que sea más fácil agregarlo, ¡podemos trabajar en los aspectos dinámicos del juego!

5.5. Ejercicios

Cielo azul: crea una ventana de Pygame con un fondo azul.

Personaje del juego: busque una imagen de mapa de bits de un personaje del juego que le guste o convierta una imagen a un mapa de bits.

Cree una clase que dibuje el personaje en el centro de la pantalla, luego haga coincidir el color de fondo de la imagen con el color de fondo de la pantalla o viceversa.

5.6. Pilotando la nave

A continuación, le daremos al jugador la posibilidad de mover la nave hacia la derecha y hacia la izquierda. Escribiremos un código que responda

cuando el jugador presione la tecla de flecha derecha o izquierda. Nos centraremos primero en el movimiento hacia la derecha y luego aplicaremos los mismos principios para controlar el movimiento hacia la izquierda. A medida que agreguemos este código, aprenderá cómo controlar el movimiento de las imágenes en la pantalla y responder a las entradas del usuario.

5.6.0.1. Responder a una pulsación de tecla

Cada vez que el jugador presiona una tecla, esa pulsación se registra en Pygame como un evento. Cada evento es recogido por el método `event.get()`. Necesitamos especificar en nuestro método `_chequear_eventos()` qué tipo de eventos queremos que el juego verifique.

Cada pulsación de tecla se registra como un evento `KEYDOWN`. Cuando Pygame detecta un evento `KEYDOWN`, debemos verificar si la tecla que se presionó es la que desencadena determinada acción. Por ejemplo, si el jugador presiona la tecla de flecha derecha, se debe aumentar el valor `rect.x` de la nave para moverla hacia la derecha:

curso.py

```
def _chequear_eventos(self):
    """Responde a las teclas presionadas y a eventos del mouse"""
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RIGHT:
                # Mueve la nave a la derecha
                self.nave.rect.x += 1
```

Dentro de `_chequear_eventos()` agregamos un bloque `elif` al bucle de eventos, para responder cuando PyGame detecta un evento `KEYDOWN`. Comprobamos si la tecla presionada es la tecla de flecha derecha. La tecla de flecha derecha está representada por `pygame.K_RIGHT`. Si se presionó la tecla de flecha derecha, movemos la nave a la derecha aumentando el valor de `self.nave.rect.x` en una unidad.

Al ejecutar `invasion_alien.py`, la nave debe moverse hacia la derecha una unidad de píxel cada vez que presiona la tecla de flecha derecha. Eso es un comienzo, pero no lo es una manera eficiente de controlar la nave. Mejoremos este control permitiendo movimiento continuo.

5.6.0.2. Permitir el movimiento continuo

Si el jugador mantiene presionada la tecla de flecha derecha la nave debe moverse continuamente hacia la derecha, hasta que el jugador suelte la tecla. Para esto necesitamos que el juego detecte los eventos `pygame.KEYUP` hasta que se suelta la tecla; usando los eventos `KEYDOWN` y `KEYUP` con un identificador llamado `movimiento_derecho` podemos el movimiento continuo. Cuando el identificador `movimiento_derecho` sea `False`, la nave estará inmóvil, y cuando el jugador presiona la tecla de flecha derecha, estableceremos el identificador como `True`, cuando el jugador suelta la tecla, volveremos a establecer la identificador como `False`.

La clase `Nave()` controla todos los atributos la nave, así que le daremos un atributo llamado `self.movimiento_derecho` y un método `actualizar()` para verificar el estado del identificador `self.movimiento_derecho`. El método `actualizar()` cambiará la posición la nave si el identificador es `True`.

Llamaremos a este método una vez en cada paso por el bucle `while` para actualizar la posición de la nave. Veamos los cambios en `nave.py`:

```
nave.py import pygame
class Nave:
    """Una clase que maneja el comportamiento de la nave"""
    def __init__(self, ia_jugar):
        --recorte--
        # Cada nave nueva empieza al centro inferior de la pantalla
        self.rectang.midbottom = self.pantalla_rectang.midbottom
        # Señal de identificador; comienza la nave sin movimiento
        self.movimiento_derecho = False

    def actualizar(self):
        """Actualiza posición de la nave basado en la señal del indicador."""
        if self.movimiento_derecho:
            self.rectang.x += 1

    def blitme(self):
        --recorte--
```

Se agrego el atributo `self.movimiento_derecho` en el método `__init__()`, y lo configuramos inicialmente como `False`. Luego agregamos en el método `actualizar()` el código:

```
if self.movimiento_derecho:
    self.rectang.x += 1
```

que permite mover la nave hacia la derecha si el indicador es `True`. El método `actualizar()` será llamado desde afuera de la clase, por lo que no se considera un método auxiliar.

Ahora necesitamos modificar el método `_chequear_eventos(self)` para

establecer el indicador `self.movimiento_derecho` en `True` cuando se presiona la tecla de flecha derecha y en `False` cuando se suelta:

```
def _chequear_eventos(self):
    """Responde a las teclas presionada y a eventos del mouse"""
    --recorte--
    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RIGHT:
            self.nave.movimiento_derecho = True
    elif event.type == pygame.KEYUP:
        if event.key == pygame.K_RIGHT:
            self.nave.movimiento_derecho = False
```

Y eliminamos el código del módulo `_chequear_eventos(self)`:

```
elif event.type == pygame.KEYDOWN:
    if event.key == pygame.K_RIGHT:
        # Mueve la nave a la derecha
        self.nave07.rectang.x += 1
```

Aquí, se modifico la forma cómo responde el juego cuando el jugador presiona la tecla de flecha derecha: en lugar de cambiar la posición de la nave directamente, simplemente configuramos `self.nave.movimiento_derecho = True`. Luego agregamos un nuevo bloque `elif`, que responde a los eventos `KEYUP` cuando el jugador suelta la tecla de flecha derecha (`K_RIGHT`), y se configura inicialmente en `False` (`self.nave.movimiento_derecho = False`).

A continuación, modificamos el bucle `while` en `run_jugar()` para que llame al método `actualizar()` de la nave en cada paso por el bucle:

```
invasión
_alien.py
def run_jugar(self):
    """Inicia el bucle principal del juego."""
    while True:
        self._chequear_eventos()
        self.nave.actualiza()
        self._actualiza_pantalla()
        self.clock.tick(60)
```

La posición de la nave se actualizará después de que hayamos verificado los eventos del teclado y antes de actualizar la pantalla. Esto permite que la posición de la nave se actualice en respuesta a la entrada del jugador y garantiza que la posición actualizada se utilizará al dibujar la nave en la pantalla.

Al ejecutar `invasion_alien.py` y mantiene presionada la tecla de flecha derecha, la nave debe moverse continuamente hacia la derecha hasta que suelte la tecla.

5.6.0.3. Moviéndose tanto hacia la izquierda como hacia la derecha

Ahora que la nave puede moverse continuamente hacia la derecha, agregando el movimiento hacia la izquierda. Nuevamente, modificaremos la clase `Nave()` y el método `_chequear_eventos()`:

Los cambios en `nave.py`, en los métodos `__init__()` y `actualizar()` se muestran a continuación:

```
nave.py  --recorte--
def __init__(self, ai_game):
    --recorte--
    # Señal de identificador; comienza la nave sin movimiento
    self.movimiento_derecho = False
    self.movimiento_izquierdo = False

def actualizar(self):
    """Actualiza posición de la nave basado en la señal del indicador."""
    if self.movimiento_derecho:
        self.rectang.x += 1
    if self.movimiento_izquierdo:
        self.rect.x -= 1
```

En el método `__init__()`, se agregó el identificador `self.movimiento_izquierdo`. Y en el método `actualizar()`, usamos dos bloques `if` separados, en lugar de un `elif`, para permitir que el valor `rect.x` de la nave aumente y disminuya cuando se mantienen presionadas ambas teclas de flecha, provoca que la nave se detenga.

Si usamos `elif` para el movimiento hacia la izquierda, la tecla de flecha derecha siempre tendría prioridad. Usar dos bloques `if` hacen los movimientos precisos cuando el jugador presione momentáneamente ambas teclas al cambiar de dirección.

Finalizamos estos ajustes haciendo dos cambios adicionales en la clase `class InvasionAlien()`, específicamente en el método `_chequear_eventos()`:

```
invasión  --recorte--
_alien.py  def _chequear_eventos(self):
    """Responde a las teclas presionada y a eventos del mouse"""
    --recorte--

    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RIGHT:
            self.nave07.movimiento_derecho = True
        elif event.key == pygame.K_LEFT:
            self.nave07.movimiento_izquierdo = True

    elif event.type == pygame.KEYUP:
        if event.key == pygame.K_RIGHT:
            self.nave07.movimiento_derecho = False
        elif event.key == pygame.K_LEFT:
```

```

self.nave07.movimiento_izquierdo = False
--recorte--

```

Si ocurre un evento KEYDOWN generado por la tecla K_LEFT, el indicador movimiento_izquierdoTrue; si ocurre un evento KEYUP generado por la tecla K_LEFT, el indicador movimiento_izquierdo es configurado como False. No podemos usar un bloquea elif, porque cada evento está conectado a una sola tecla, y al jugador presionas ambas teclas a la vez, se detectarán dos eventos separados.

Al ejecutar invasion_alien.py, debe poder mover la nave continuamente a la derecha y a la izquierda. Además, si mantienes presionadas ambas teclas, la nave debería dejar de moverse.

A continuación, perfeccionaremos aún más el movimiento la nave. Ajustemos la velocidad de la nave y limitemos qué tan lejos puede moverse para que no desaparezca de los lados de la pantalla.

5.6.0.4. Ajustar la velocidad de la nave

Actualmente, la nave se mueve un píxel por ciclo a través del bucle while, pero podemos tomar un control más preciso de la velocidad de la nave agregando un atributo la velocidad de la nave velocidad_nave, en la clase Configuracion().

Usaremos este atributo para determinar qué tan lejos mover ella nave en cada paso por el circuito.

Agregamos el nuevo atributo en configurar.py:

```

configurar.py class Configuracion:
    """Una clase que almacena todas las confuguarciones
    para Invasion Alienigena."""
    def __init__(self):
        """Inicializa las configuraciones del juego"""
        # Configuración de pantalla
        self.fondo_image_filename = 'pygame.jpg'
        self.pantalla_width = 800
        self.pantalla_height = 600
        self.color_fondo = (170,238,187)#(230, 230, 230)
        self.velocidad_nave = 1.5

```

Establecemos el valor inicial de velocidad_nave en 1,5. Cuando ella nave se mueva ahora, su posición se ajusta en 1,5 píxeles (en lugar de 1 píxel) en cada paso por el bucle. Y usamos un flotador para configurar la velocidad para tener un control más preciso de la velocidad de la nave cuando aumentamos el ritmo del juego más adelante. Sin embargo, los atributos rect como

x se almacenan solo valores enteros, por lo que debemos realizar algunas modificaciones en la clase `Nave()`:

```
nave.py import pygame
class Nave:
    """Una clase que maneja el comportamiento de la nave"""
    def __init__(self, ia_jugar):
        """Inicia la nave y configura la posicion inicial."""
        self.pantalla = ia_jugar.pantalla
        self.configuracion = ia_jugar.configuracion

        self.pantalla_rectang = ia_jugar.pantalla.get_rect()

        # Carga la imagen y obtiene su rectangulo
        #self.imagen = pygame.image.load('../A Logos de Python/nave.bmp')
        self.imagen = pygame.image.load('imagenes/nave05.bmp') #("nave.bmp")
        self.rectang = self.imagen.get_rect()

        # Cada nave nueva empieza al centro inferior de la pantalla
        self.rectang.midbottom = self.pantalla_rectang.midbottom
        # Almacena un float para la posicion exacta horizontal de la nave.
        self.x = float(self.rectang.x)
        # Señal de identificador; comienza la nave sin movimiento
        self.movimiento_derecho = False
        self.movimiento_izquierdo = False

    def actualizar(self):
        """Actualiza posición de la nave basado en la señal del indicador."""
        # Actualiza el valor x de la nave, no en el objeto rect.
        if self.movimiento_derecho:
            self.x += self.configuracion.velocidad_nave
            #self.rectang.x += 1
        if self.movimiento_izquierdo:
            self.x -= self.configuracion.velocidad_nave
            #self.rectang.x -= 1
        # Actualiza el objeto rectang con self.x.
        self.rectang.x = self.x
    def blitme(self):
        """Dibuja la nave en la posicion actual"""
        self.pantalla.blit(self.imagen, self.rectang)
```

Creamos un atributo `self.configuracion = ia_jugar.configuracion` de configuración en clase `Nave()`, para usarlo en `actualizar()`. Debido a que estamos ajustando la posición de la nave en fracciones de píxel, es necesario asignar la posición a una variable a la que se le pueda asignar un flotante. Se puede usar un flotante para establecer un atributo del objeto `rectang`, pero el `rect` solo mantendrá la porción entera de ese valor. Para realizar un seguimiento preciso de la posición de la nave, definimos un nuevo atributo `self.x`; y usamos la función `float()` para convertir el valor de `self.rect.x` en un flotante y luego asignar este valor a `self.x`.

Ahora, cambiamos la posición de la nave en `actualizar()`, el valor de

`self.x` se ajusta según la cantidad almacenada en `self.configuracion.velocidad_nave`.

Después que `self.x` se ha actualizado, utilizamos el nuevo valor para actualizar `self.rect.x`, que controla la posición de la nave.

Sólo la porción entera de `self.x` será asignado a `self.rect.x`, pero eso está bien.

Ahora podemos cambiar el valor de `velocidad_nave` a cualquier valor mayor que 1, y hará que la nave se mueva más rápido. Esto ayudará a que la nave responda lo suficientemente rápido como para derribar alienígenas, y nos permitirá cambiar el ritmo del juego a medida que el jugador avanza en el juego.

5.6.0.5. Limitar el alcance de la nave

La nave desaparecerá en cualquier borde de la pantalla si mantiene presionada una tecla de flecha el tiempo suficiente. Para evitar esta situación haremos que nave se detenga al llega al borde de la pantalla. Esto lo hacemos modificando el método `actualizar()` en `Nave()`:

curso.py

```
--recorte--
def _chequear_eventos(self):
    """Responde a las teclas presionada y a eventos del mouse"""
--recorte--

def actualizar(self):
    """Actualiza posición de la nave basado en la señal del indicador."""
    # Actualiza el valor x de la nave, no en el objeto rect
    if self.movimiento_derecho and self.rectang.right < self.pantalla_rectang.right:
        self.x += self.configuracion.velocidad_nave
    if self.movimiento_izquierdo and self.rectang.left > 0:
        self.x -= self.configuracion.velocidad_nave
    # Actualiza el objeto rectang con self.x.
    self.rectang.x = self.x

def blitme(self):
    --recorte--
```

El código `self.rectang.right` devuelve la coordenada x del borde derecho del rectángulo de la nave. Si este valor es menor que el valor devuelto por `self.pantalla_rect.right`, ella nave no ha llegado al borde derecho de la pantalla. De igual forma con el borde izquierdo: si el valor del lado izquierdo del rectángulo es mayor que 0, la nave no ha llegado al borde izquierdo de la pantalla. Esto asegura que la nave está dentro de estos límites antes de ajustar el valor de `self.x`.

Al ejecutar `invasion_alien.py`, la nave debería dejar de moverse en cualquier borde de la pantalla. Esto está muy bien; todo lo que hemos hecho es agregar una prueba condicional en una declaración `if`, pero se siente como si la nave golpeará una pared o un campo de fuerza en los borde de la pantalla.

5.7. Refactorización `_chequear_eventos()`

El método `_chequear_eventos()` en `invasion_alien.py` aumentará en longitud a medida que avancemos en el desarrollo del juego, así que dividamos en dos métodos separados: uno que maneja eventos `KEYDOWN` y otro que maneja eventos `KEYUP`:

```
invasión
_alien.py  --recorte--
def _chequear_eventos(self):
    """Respuesta a teclas presionads y eventos del muuse"""
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

        elif event.type == pygame.KEYDOWN:
            self._chequear_keydown_eventos(event)
        elif event.type == pygame.KEYUP:
            self._chequear_keyup_eventos(event)

def _chequear_keydown_eventos(self, event):
    """Respuesta a teclas presionadas"""
    if event.key == pygame.K_RIGHT:
        self.nave.movimiento_derecho = True
    elif event.key == pygame.K_LEFT:
        self.nave.movimiento_izquierda = True
    if event.key == pygame.K_q:
        pygame.quit()
        sys.exit()

def _chequear_keyup_eventos(self, event):
    """respuesta a tecla liberada"""
    if event.key == pygame.K_RIGHT:
        self.nave.movimiento_izquierda = False
    elif event.key == pygame.K_LEFT:
        self.nave.movimiento_izquierda = False
--recorte--
```

Creemos dos métodos auxiliares para reemplazamos el código antiguo del método `_chequear_eventos()`, el método `_chequear_keydown_eventos()` y el método `_chequear_keyup_eventos`. Cada uno con un parámetro `self` y un parámetro para los eventos. Esta estructura de código es más limpia y facilitará el desarrollo de respuestas adicionales a las entradas del jugador.

5.7.0.1. Presionando Q para salir

Ahora respondemos a las pulsaciones de teclas de manera eficiente, y podemos agregar otra forma adicional para salir del juego. Muy útil cuando se trabaja con ventanas sin borde (ver sección 2.8.3). Hace clic en el botón X en la esquina superior derecha de la ventana del juego para finalizar el juego cada vez que pruebas una nueva función algunas veces es tedioso, por lo que agregaremos un atajo de teclado para finalizar el juego cuando el jugador presiona q:

En el archivo `invasion_alien.py` agregamos la segunda opción para salir del juego, específicamente al final del método auxiliar `_chequear_keydown_eventos()`, agregamos un nuevo bloque `elif` que finaliza el juego cuando el jugador presiona q o Q.

```
invasión    --recorte--
_alien.py   def _chequear_eventos(self):
            """Respuesta a teclas presionadas y eventos del mouse"""
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()

            def _chequear_keydown_eventos(self, event):
                """Respuesta a teclas presionadas"""
                if event.key == pygame.K_RIGHT:
                    self.nave07.movimiento_derecho = True
                elif event.key == pygame.K_LEFT:
                    self.nave07.movimiento_izquierdo = True
                elif event.key == pygame.K_q:
                    pygame.quit()
                    sys.exit()
            --recorte--
```

Ahora, al ejecutar `invasion_alien.py`, puedes presionar Q para cerrar el juego, como una opción más.

5.7.1. Ejecutar el juego en modo de pantalla completa

Para ejecutar el juego en modo de pantalla debemos hacer unos cambios sencillos en configuración de pantalla en el archivo `invasion_alien.py`, dentro del método `init`:

```
invasión    --recorte--
_alien.py   class AlienInvasion:
            """Clase general para gestionar comportamiento del juego."""
            def __init__(self):
                """Inicializa el juego y crea recursos para el juego."""
                pygame.init()
                self.clock = pygame.time.Clock()
```



```

self.configuracion = Configuracion()
self.pantalla = pygame.display.set_mode((0, 0), pygame.FULLSCREEN)
self.configuracion.pantalla_width = self.pantalla.get_rect().width
self.configuracion.pantalla_height = self.pantalla.get_rect().height

pygame.display.set_caption("Invasión Alienígena")
self.pantalla= pygame.display.set_mode(
    (self.configuracion.pantalla_width,
     self.configuracion.pantalla_height))
pygame.display.set_caption("Invasión Alienígena")
self.fondo = pygame.image.load(
    self.configuracion.fondo_image_filename).convert()
self.nave07 = Nave(self)
--recorte--

```

Se usaron los atributos de ancho y alto del objeto `rect()` de la pantalla (`self.pantalla.get_rect().width`) para actualizar la configuración de pantalla. Al crear la superficie de la pantalla, le pasamos un tamaño de (0, 0) y el parámetro `pygame.FULLSCREEN`. Esto le dice a PyGame que busque una tamaño de ventana que llenará la pantalla. Porque no conocemos el ancho y el alto de la pantalla con anticipación, actualizamos esta configuración después de crear la pantalla.



Asegúrate de poder salir presionando Q antes de ejecutar el juego en modo de pantalla completa; PyGame no ofrece una forma predeterminada de salir de un juego en modo de pantalla completa.

En este caso al ejecutar y desplegar el juego en modo de pantalla completa no tenemos la opción de hacer clic en la esquina superior derecha de la pantalla por que no se ve; pero con la opción Q salimos del juego.

Algunos juegos se ven mejor en modo de pantalla completa, y en algunos sistemas, el juego puede funcionar mejor en general en modo de pantalla completa. Si te gusta cómo se ve o como se comporta el juego en modo de pantalla completa, puede seguir con estos ajustes, y si no puedes volver al enfoque original donde establecimos un tamaño de pantalla específico para el juego.

5.8. Resumiendo el trabajo

En la siguiente sección, agregaremos la capacidad de disparar balas, lo que implica agregar un nuevo archivo llamado `balas.py` y realizando algunas

modificaciones en algunos de los archivos que ya estamos usando.

Hasta ahora tenemos tres archivos que contienen un número de clases y métodos. Para tener claro cómo está organizado el proyecto, revisemos cada uno de estos archivos antes de agregar más funciones.

5.8.1. `invasión_alien.py`

El archivo principal `invasion_alien.py`, contiene la clase `InvasionAlien`. Esta clase crea una serie de atributos importantes que se utilizan a lo largo del juego: como las configuraciones se asignan de la superficie principal de visualización de la pantalla, también se crea una instancia de la nave en este archivo. El bucle principal del juego, un bucle `while`, también se almacena en este módulo. El bucle `while` llama a `_chequear_eventos()`, y `_actualizar_pantalla()`. También marca el reloj en cada paso por el bucle.

El método `_chequear_eventos()` detecta eventos relevantes, como pulsaciones y liberaciones de teclas, procesa cada uno de estos tipos de eventos a través de los métodos `_chequear_keydown_eventos()` y `_chequear_keyup_eventos()`. Por ahora, estos métodos gestionan el movimiento de la nave.

La clase `InvasionAlien()` también contiene `_actualizar_pantalla()`, que dibuja la pantalla en cada paso por el bucle principal.

El archivo `invasion_alien.py` es el único archivo que necesitas ejecutar cuando quieras jugar *Invasión Alienígena*.

Los otros archivos, `configurar.py` y `nave.py`, contienen código que se importa a este archivo.

5.8.2. `configurar.py`

El archivo `configurar.py` contiene la clase `Configuracion()`. Esta clase sólo tiene un método `__init__()`, que inicializa los atributos que controlan la apariencia del juego y la velocidad de la nave.

5.8.3. `nave.py`

El archivo `nave.py` contiene la clase `Nave()`, y esta contiene un método `__init__()`, el método `actualizar()` para gestionar la posición de la nave, y el método `blitme()` para dibujar la nave en la pantalla. La imagen de la nave es almacenada en `nave.bmp`, que se encuentra en la carpeta de imágenes.

5.9. INTÉNTALO TÚ MISMO

12-3. Documentación de Pygame: Ya hemos avanzado lo suficiente en el juego como para que desees consultar parte de la documentación de Pygame. La página de inicio de Pygame está en <https://pygame.org> y la página de inicio de la documentación está en <https://pygame.org/docs>. Simplemente lea la documentación por ahora. No lo necesitarás para completar este proyecto, pero te ayudará si luego quieres modificar Alien Invasion o crear tu propio juego.

12-4. Cohete: Crea un juego que comience con un cohete en el centro de la pantalla. Permita que el jugador mueva el cohete hacia arriba, abajo, izquierda o derecha usando las cuatro teclas de flecha. Asegúrate de que el cohete nunca se mueva más allá de ningún borde de la pantalla.

12-5. Teclas: Cree un archivo Pygame que crea una pantalla vacía. En el bucle de eventos, imprima el atributo `event.key` cada vez que se detecte un evento `pygame.KEYDOWN`. Ejecute el programa y presione varias teclas para ver cómo responde Pygame.

5.10. Disparar balas

Ahora le agregaremos la capacidad de disparar balas a la nave. Para esta tarea crearemos un módulo `balas.py`, donde se construye las balas, tamaño, posición, y disparo de balas.

Se usará el módulo `pygame.Rect()` para crear un rectángulo que representara las balas; las balas serán disparadas desde la nave hacia arriba; es decir, las balas viajarán desde la nave hacia arriba hasta que desaparezcan la parte superior de la pantalla.

Y en el módulo `configurar.py` se agregan las configuración para las balas al final del método `__init__()`, aquí incluimos los valores que necesitaremos para la nueva clase `Balas`, velocidad, tamaño y color:

```
configurar.py --recorte--
    def __init__(self):
        --recorte--
        # configuraciones para las balas
        self.velocidad_balas = 6.0
        self.ancho_balas = 10 # 2
        self.alto_balas = 16
        self.color_balas = (0, 0, 0)
```

Estas configuraciones crean balas de color negro con un ancho de 3 píxeles y un altura de 16 píxeles. Las balas viajarán un poco más rápido que ella nave.

Ahora debemos crear la clase `Balas()`.

Clase Balas

Debe crear un módulo `balas.py` para almacenar la clase `Balas()`, y guardarlo en la carpeta principal. En este módulo escribimos el siguiente código:

```
balas.py
import pygame
from pygame.sprite import Sprite

class Balas(Sprite):
    """Una clase para manejar las balas disparadas desde la nave"""
    def __init__(self, ia_jugar):
        """crea un objeto bala desde la posicion actual de la nave"""
        super().__init__()
        self.pantalla = ia_jugar.pantalla
        self.configuraciones = ia_jugar.configuraciones
        self.color = self.configuraciones.color_balas
        # Crea un rectangulo en (0, 0) y se configura la posicion correcta
        self.rectang = pygame.Rect(0, 0, self.configuraciones.ancho_balas,
                                    self.configuraciones.alto_balas)
        self.rectang.midtop = ia_jugar.nave.rectang.midtop
        # Almacena la posicion de las balas como un flotante
        self.y = float(self.rectang.y)
```

Aquí estamos usando una clase `Sprite` del módulo `pygame.sprite` como argumento de la clase `Balas()`; de esta forma nuestra clase es una clase hija, heredera de la clase `Sprite`.

Se importa del módulo `pygame.sprite` la clase `Sprite`. Cuando usas el módulo `pygame.sprite`, puedes agrupar elementos relacionados en tu juego y actúa sobre todos los elementos agrupados a la vez.

Para crear una instancia de `Balas()`, se necesita la instancia actual de la clase `InvasionAlien`, y llamamos a `super().__init__()` para heredar correctamente de la clase `Sprite`. También se configuran los atributos para la pantalla, objetos de configuración, y el color de la balas.

Ahora creamos el atributo `self.rectang` para la balas. Las balas no se basa en una imagen, por lo que tenemos que construir un rectángulo, para ello se usara la clase `pygame.Rect()`. Esta clase requiere de las coordenadas *x* e *y* de la esquina superior izquierda del rectángulo, el ancho y alto del rectángulo.

Inicializamos el rectángulo en (0, 0), pero lo moveremos a la ubicación correcta en la siguiente línea, porque la posición de la bala depende de la posición de la nave. Obtenemos el ancho y el alto de la bala a partir de los valores almacenados en `self.configuraciones`. Configuramos el atributo `.midtop` de la bala para que coincida con el atributo `.midtop` de la nave. Esto hará que la bala emerja desde la parte superior de la nave, haciendo que parezca que la bala se dispara desde la nave. Usamos un flotador para la coordenada *y* de la bala para poder hacer ajustes a la velocidad de la bala.

Agregamos una segunda parte al archivo `balas.py` que consiste en dos métodos, `update()` y `dibuja_balas()`:

```
balas.py --recorte--
def update(self):
    """Movimiento de bala hacia arriba por la pantalla"""
    # Actualiza exacta de la posicion de las balas.
    self.y -= self.configuraciones.velocidad_balas
    # Actualiza la posicion del rectangulo
    self.rectang.y = self.y

def dibuja\_balas(self):
    """Dibuja las balas en la pantalla"""
    pygame.draw.rect(self.pantalla, self.color, self.rectang)
```

El método `actualiza()` gestiona la posición de la bala. Cuando se dispara una bala, sube por la pantalla, lo que corresponde a un valor decreciente de la coordenada *y*.

Para actualizar la posición, restamos la cantidad almacenada en la configuración `.velocidad_balas` de `self.y`. Luego usamos el valor de `self.y` para establecer el valor de `self.rectang.y = self.y`.

La configuración `.velocidad_balas` nos permite aumentar la velocidad de las balas a medida que avanza el juego o según sea necesario para refinar el comportamiento del juego.

Una vez que se dispara una bala, nunca cambiamos el valor de su coordenada *x*, por lo que viajará verticalmente en línea recta incluso si la nave se mueve.

Cuando queremos dibujar una bala, llamamos a `dibuja_balas()`. La función `pygame.draw.rect()` llena la parte de la pantalla definida por el rectángulo de la bala con el color almacenado en `self.color`.

5.10.0.1. Almacenamiento de balas en un grupo

Ahora que tenemos la clase `Balas()` y las configuraciones necesarias definidas, podemos escribir código para disparar una bala cada vez que el jugador

presiona la barra espaciadora.

Creamos un grupo en la clase `InvasionAlien` para almacenar todas las balas activas que se han disparado. Este grupo será una instancia de la clase `pygame.sprite.Group`, que se comporta como una lista con algunas funciones adicionales que son útiles al crear juegos. Usaremos este grupo para dibujar bala en la pantalla en cada paso por el bucle principal y para actualizar la posición de cada bala.

Actualizamos el archivo `invasion_alien.py`.

Se importa la clase `Balas()`:

```
invasión
_alien.py  --recorte--
           from nave import Nave
           from balas import Balas
```

Creamos el grupo que contiene las balas en método `__init__()`:

```
invasión
_alien.py  --recorte--
           def __init__(self):
               --recorte--
               self.nave = Nave(self)
               self.balas = pygame.sprite.Group()
```

Luego necesitamos actualizar la posición de las balas en cada paso el bucle `while`:

```
invasión
_alien.py  def run_jugar(self):
           """Inicia el bucle principal del juego"""
           while True:
               self._check_events()
               self.nave.actualizar()
               self.balas.update()
               self._actualiza_pantalla()
               self.clock.tick(60)
```

Cuando se llama a `update()` en un grupo, el grupo actualiza automáticamente cada sprite del grupo. En este caso, la línea `self.balas.update()`, actualiza automáticamente para cada bala que colocamos en el grupo.

5.10.1. Disparar balas

En la clase `InvasionAlien`, es necesario modificar el método `_chequear_keydown_eventos()` para disparar una bala cuando el jugador presiona la barra espaciadora. Sin embargo, en el método `_chequear_keyup_eventos()` no es necesario modificaciones, porque no pasa nada cuando al liberar la barra espaciadora.

Y también necesitamos modificar el método `_actualizar_pantalla()` para asegurarse de que cada bala se dibuje en la pantalla antes de llamar a `flip()`.

Y para disparar una bala se escribe un nuevo método, `_disparar_balas()`, que hace el trabajo de disparar balas:

```
invasión    --recorte--
_alien.py   def _chequear_keydown_eventos(self, event):
            --recorte--
            elif event.key == pygame.K_q:
                pygame.quit()
                sys.exit()
            elif event.key == pygame.K_SPACE:
                self._dispara_bala()
        def _check_keyup_events(self, event):
            --recorte---
        def _dispara_bala(self):
            """Crea una nueva bala y la añade al grupo de balas"""
            nueva_bala = Balas(self)
            self.balas.add(nueva_bala)

        def _actualiza_pantalla(self):
            """Actualiza imagen sobre pantalla, y cambia la nueva pantalla"""
            self.pantalla.fill(self.configuracion.color_fondo)
            for bala in self.balas.sprites():
                bala.dibuja_bala()
            self.nave07.blitme()
            pygame.display.flip()
            --recorte--
```

Al presionar la barra espaciadora se activa el llamado a `_dispara_bala()`. Se crea un objeto de la clase `Balas()`, llamado `nueva_bala`. Luego lo agregamos al grupo de balas usando el método `self.balas.add(nueva_bala)`. El método `add()` es similar al método `append()`, pero está escrito específicamente para grupos de PyGame (`self.balas = pygame.sprite.Group()`).

El método `balas.sprites()` devuelve una lista de todos los sprites del grupo de balas. Para dibujar todas las balas disparadas en la pantalla, recorreremos los sprites en `balas` y llamamos a `draw_bullet()` en cada una 4. Colocamos este bucle antes de la línea que dibuja ella nave, para que las balas no comiencen encima de la nave.

Cuando ejecute `invasion_alien.py` ahora, deberías poder mover la nave hacia la derecha y hacia la izquierda y disparar tantas balas como quieras. Las balas suben por la pantalla y desaparecen cuando llegan a la cima, como se muestra en la Figura 5.3.

Como tarea Usted puede modificar el tamaño, el color y la velocidad de las balas en `configurar.py`.



Figura 5.3: La nave después de disparar una serie de balas.

5.10.1.1. Eliminar balas fuera del alcance

Por el momento, las balas desaparecen cuando llegan a la parte superior, pero sólo porque PyGame no puede dibujarlas por encima de la parte superior de la pantalla. En realidad, las balas siguen existiendo; sus valores de coordenadas *y* se vuelven cada vez más grandes pero negativas. Esto es un problema porque continúan consumiendo memoria y potencia de procesamiento.

Necesitamos deshacernos de estas viejas balas o el juego se ralentizará haciendo tanto trabajo innecesario. Para resolver este inconveniente, necesitamos detectar el valor inferior del objeto rectángulo de la bala cuando alcance un valor igual a 0, lo que indica que la bala pasó parte superior de la pantalla:

```
invasión  --recorte--
_alien.py  def run_jugar(self):
            """Inicia el principal bucle del juego"""
            while True:
                self._chequear_eventos()
                self.nave.actualiza()
                self.balas.update()
                # Obtener rid de balas que han disparado.
                for balas in self.balas.copy():
                    if balas.rectang.bottom <= 0:
                        self.balas.remove(balas)
                        print("Una", len(self.balas))
                self._actualiza_pantalla()
                self.clock.tick(60)
```


Cuando se usa un bucle `for` con una lista o un grupo en PyGame, Python espera que la lista mantenga la misma longitud mientras se ejecute el bucle. Eso significa que no puedes eliminar elementos de una lista o grupo dentro de un bucle `for`, entonces tenemos que optar por una copia del grupo. Para ello usamos el método `copy()` para configurar el bucle `for`, lo que nos deja libres para modificar el grupo de balas original dentro del bucle. Comprobamos cada bala para ver si ha desaparecido en la parte superior de la pantalla. Si lo tiene, lo eliminamos de las balas. Insertamos un llamada `print()` para mostrar cuántas balas existen actualmente en el juego y verificar se eliminan cuando llegan a la parte superior de la pantalla.

Si este código funciona correctamente, podemos observar la salida en la terminal mientras disparamos balas y ver que el número de balas disminuye a cero después de cada serie de balas se ha despejado en la parte superior de la pantalla.

Después de ejecutar el juego y verificar que las bala se eliminen correctamente, elimine el `print()`. Si lo dejas, el juego se ralentizará significativamente porque requiere más tiempo para escribir la salida en la terminal que para dibujar gráficos en el ventana de juego.

5.10.2. Limitar el número de balas

En muchos juegos de disparos limitan la cantidad de balas que un jugador puede tener en el pantalla al mismo tiempo; esta limitación hace que el jugado dispare con precisión. Incorporaremos este restricción en Invasión Alienígena.

Primero, almacenamos la cantidad de balas permitidas en pantalla en la variable `self.balas_permitidas = 10` dentro del archivo `configurar.py`:

```
configurar.py --recorte--
                # Configurando Balas
                --recorte--
                self.balas_color = (0, 0, 0)
                self.balas_permitidas = 10
```

Esta restricción limita al jugador a diez balas a la vez en la pantalla. Usaremos esta configuración en `InvasionAlien` para verificar cuántas balas existen antes de crear una nueva bala en `_disparar_balas()`:

```
invasión      --recorte--
  _alien.py    def _dispara_bala(self):
                """Crea una nueva bala y la añade al guupo de balas"""
                if len(self.balas) < self.configuracion.balas_permitidas:
                    nueva_bala = Balas(self)
                    self.balas.add(nueva_bala)
```

Cuando el jugador presiona la barra espaciadora, verificamos la longitud del grupo de balas (número de balas). Si `len(self.balas)` es menor que diez, creamos una nueva bala. Pero si ya hay diez balas activas, no pasa nada cuando se presiona la barra espaciadora.

Al ejecutar el juego, solo deberías poder disparar balas en grupos de diez.

5.10.3. Organizando la clase `InvasionAlien()`

Es importante mantener la clase principal `InvasionAlien` razonablemente bien organizada, por lo que ahora que hemos escrito y verificado el código para la administración de balas, podemos moverlo a un método específico para actualizar balas.

Crearemos un nuevo método llamado `_actualizar_balas()` y lo agregaremos justo antes de `_actualizar_pantalla()`:

```
invasión  --recorte--
_alien.py  def _actualizar_balas(self):
            """Actualizar la posición de balas y obtener rid
            de las balas fuera del alcance."""
            # Actualizar posición de balas.
            self.balas.update()
            # Obtener rid de balas que han disparado
            for balas in self.balas.copy():
                if balas.rectang.bottom <= 0:
                    self.balas.remove(bala)
            --recorte--
```

El código para `_actualizar_balas()` se corta y pega desde `run_jugar()`; todo lo que hemos hecho aquí es aclarar los comentarios.

Ahora el bucle `while` en `run_jugar()` vuelve a quedar simple y claro:

```
invasión  def run_jugar(self):
_alien.py  """Inicia el bucle principal del juego."""
            while True:
                self._chequear_eventos()
                self.nave07.actualizar()

                self.balas.update()
                self._actualizar_balas()
                self._actualizar_pantalla()

                self.nave07.blitme()
                # Actualiza la pantalla, dibujada la más recientemente
                pygame.display.flip()
                self.clock.tick(60)
```

Ahora nuestro bucle principal contiene solo un código mínimo, por lo que podemos leer rápidamente los nombres de los métodos y comprender lo que

sucede en el juego. El bucle principal comprueba la entrada del jugador y luego actualiza la posición de la nave y cualquier bala que haya sido disparada. Luego usamos las posiciones actualizadas para dibujar una nueva pantalla y marcar el reloj al final de cada paso por el bucle.

Al ejecutar `invasion_alien.py` una vez más y asegúrese de que aún pueda disparar balas sin errores.

5.11. INTÉNTALO TÚ MISMO

12-6. Tirador lateral: escribe un juego que coloque una nave en el lado izquierdo del pantalla y permite al jugador mover ella nave hacia arriba y hacia abajo. Haz que la nave se dispare bala que viaja a través de la pantalla cuando el jugador presiona la barra espaciadora. Asegúrese de que las balas se eliminen una vez que desaparezcan de la pantalla.

5.12. Resumen

En este capítulo, aprendiste a hacer un plan para un juego y aprendiste la estructura básica de un juego escrito en Pygame. Aprendiste a establecer un fondo. Configuraciones de color y organizar en una clase separada donde puedes ajustar más fácilmente. Aprendiste cómo dibujar una imagen en la pantalla y darle control al jugador sobre el movimiento de los elementos del juego. Creaste elementos que se mueven solos, como balas volando por una pantalla, y eliminaste objetos que ya no son necesarios. También aprendiste a refactorizar código en un proyecto de forma regular para facilitar el desarrollo continuo.

En el Capítulo 6, agregaremos las naves alienígenas en el juego Invasión Alienígena. Por el final de la capítulo, podrás derribar extraterrestres, con suerte antes de que lleguen a la nave.

Capítulo 6

Alienígenas en el cielo



En este capítulo, agregaremos los alienígena o extraterrestres al juego *Invasión Alienígena*. Agregaremos los alienígenas cerca de la parte superior de la pantalla y luego generamos una flota completa de alienígenas. Haremos que la flota avance de lado y hacia abajo, y nos desha-

remos de cualquier Alienígena que sea alcanzado por una bala. Finalmente, limitaremos la cantidad de la naves que tiene un jugador y finalizaremos el juego cuando el jugador se quede sin naves.

A medida que avance en este capítulo, aprenderá más sobre Pygame y sobre la gestión de un gran proyecto. También aprenderás a detectar colisiones entre los objetos del juego, como balas y extraterrestres. Detectar colisiones te ayuda a definir interacciones entre elementos en los juegos.

Por ejemplo, puedes confinar un personaje dentro de las paredes de un laberinto o pasar una pelota entre dos paredes. Continuaremos trabajando a partir de un plan que revisamos ocasionalmente para mantener el enfoque de nuestras sesiones de escritura de códigos.

Antes de comenzar a escribir código nuevo para agregar una flota de alienígenas a la pantalla, echemos un vistazo al proyecto y actualicemos nuestro plan.

6.1. Revisar el proyecto

Cuando estás comenzando una nueva fase de desarrollo en un proyecto grande, siempre es buena idea revisar su plan y aclarar lo que quiere lograr con el código que estás a punto de escribir.

En este capítulo haremos la siguiente:

- Agregamos un único alienígena en la esquina superior izquierda de la pantalla, con espacio suficiente a su alrededor.
- Llena la parte superior de la pantalla con tantos alienígenas como podamos colocar horizontalmente. Luego crearemos filas adicionales de alienígenas hasta que tengamos una flota completa.
- Haremos que la flota se mueva hacia los lados y hacia abajo hasta que toda la flota llegue abajo. Si toda la flota es derribada, crearemos una nueva flota. Si un extraterrestre golpea la nave o llega al suelo destruiremos su nave terrestre y finaliza esta fase del juego.
- Se limita el número de la naves que el jugador puede usar y finaliza el juego cuando el jugador haya agotado el número de naves asignadas.

Perfeccionaremos este plan a medida que implementemos funciones, pero esto es lo específico y suficiente para empezar a escribir código. También debe revisar su código existente cuando comience a trabajar en un nueva serie de características para proyecto. Porque cada nueva fase normalmente hace un proyecto más complejo, es mejor limpiar cualquier desordenado o código ineficiente. Hemos estado refactorizando a medida que avanzamos, por lo que no necesitamos ningún código para refactorizar en este punto.

6.2. El primer alienígena

Dibujar un alienígena en la pantalla es un proceso semejante a de dibujar la nave en la pantalla. El comportamiento de los alienígenas será controlado por una clase llamada `Alien()`, que estructuraremos como la clase `Nave()`.

Continuaremos usando imágenes de mapa de bits para simplificar (.bmp). Puedes encontrar tu propia imagen para un extraterrestre o usar la que se muestra en la Figura 6.1.

que está disponible en los recursos del libro en la web linis. Esta imagen tiene un fondo gris, que coincide con el color de fondo de pantalla. Asegúrese



Figura 6.1: El extraterrestre que usaremos.

de guardar el archivo de imagen que elija en la carpeta de imágenes.

6.2.1. La clase alienígena

Ahora escribiremos la clase `Alien` y la guardaremos como `alien.py`:

```
alien.py
import pygame
from pygame.sprite import Sprite
class Alien(Sprite):
    """Una clase para representar un alienígena del afluente."""
    def __init__(self, ia_jugar):
        """Inicializa el alienígena y se configura su posición inicial."""
        super().__init__()
        self.pantalla = ia_jugar.pantalla
        # carga la imagen y configura el atributo rectang
        self.imagen = pygame.image.load('imagenes/alien.bmp')
        self.rectang = self.imagen.get_rect()
        # Cada alienígena aparece cerca de la esquina superior izquierda
        # de la pantalla
        self.rectang.x = self.rectang.width/3
        self.rectang.y = self.rectang.height/3
        # Almacena el alienígena en la posición exacta
        self.x = float(self.rectang.x)
```

La mayor parte de esta clase es como la clase de la nave, excepto por la ubicación del alienígena en la pantalla. Inicialmente dibujamos a cada alienígena cerca de la esquina superior izquierda de la pantalla; agregamos un espacio a la izquierda que es igual al ancho del alienígena y un espacio por encima es igual a su altura, por lo que es fácil de ver. Nos preocupamos principalmente la velocidad horizontal de los extraterrestres, por lo que rastreamos la posición horizontal de cada uno extraterrestre precisamente.

Esta clase `Alien` no necesita un método para dibujarla en la pantalla; en su lugar, usaremos un método de grupo de Pygame que dibuja automáticamente todos los elementos de un grupo a la pantalla.

6.2.2. Una instancia de la clase Alien

Debemos crear una instancia de Alien para que podamos ver el primer alienígena en la pantalla. Debido a que es parte de nuestro trabajo de configuración, agregaremos el código para la instancia al final del método `__init__()` en la clase `InvasionAlien`.

Luego debemos crear una flota completa de alienígenas, lo que supondrá bastante trabajo, así que para ello creamos un nuevo método auxiliar llamado `_crear_flota()`.

El orden de los métodos en una clase no importa, siempre que haya algo de coherencia en la forma en que se colocan. Colocaré `_crear_flota()` justo antes del método `_actualizar_pantalla()`, pero en cualquier lugar funcionará.

Actualizamos los cambios en `invasión_alien.py`:

Importamos la clase `Alien()`:

```
invasión    from balas import Balas
_alien.py   from alien import Alien
```

Y actualizamos el método `__init__()`:

```
invasión    --recorte--
_alien.py   def __init__(self):
            --recorte--
            self.nave = Nave(self)
            self.balas = pygame.sprite.Group()
            self.aliens = pygame.sprite.Group()
            self._crear_flota()
```

Creamos un grupo para mantener la flota de extraterrestres y llamamos al método nuevo `_crear_flota()`:

```
invasión    --recorte--
_alien.py   def _crear_flota(self):
            """Crear la flota de alienígena."""
            # Hacer un alienígena
            alien = Alien(self)
            self.aliens.add(alien)
```

En este método, creamos una instancia de la clase `Alien` y luego lo agregamos al grupo que contendrá la flota. El extraterrestre se colocará en el área superior izquierda predeterminada de la pantalla. Para que aparezca el alienígena, necesitamos llamar al método `draw()` del grupo en `_actualizar_pantalla()`:

```
invasión    def _actualizar_pantalla(self):
_alien.py   --recorte--
            self.nave.blitme()
            self.aliens.draw(self.pantalla)
            pygame.display.flip()
```

Al llamar al método `draw()` en un grupo, Pygame dibuja cada elemento del grupo en la posición definida por su atributo `rect`. El método `draw()` requiere un argumento: una superficie sobre la cual dibujar los elementos del grupo. `self.aliens.draw(self.pantalla)`.

La Figura 6.2 muestra el primer alienígena en la pantalla.



Figura 6.2: Aparece el primer alienígena en la pantalla.

Ahora que el primer alienígena aparece correctamente, escribiremos el código para dibujar toda una flota.

6.2.3. Construyendo la flota alienígena

Para dibujar una flota, necesitamos descubrir cómo llenar la parte superior de la pantalla con extraterrestres, sin saturar la ventana del juego. Hay un número de maneras para lograr este objetivo. Lo abordaremos agregando extraterrestres a través la parte superior de la pantalla, hasta que no quede espacio para un nuevo extraterrestre. Repitiendo el proceso, siempre y cuando tengamos suficiente espacio vertical para agregar una nueva fila.

6.2.3.1. Una fila de extraterrestres

Ahora estamos listos para generar una fila completa de extraterrestres. Para crear una fila completa, primero haremos un solo alienígena que tengamos acceso al ancho del alienígena, colocaremos el extraterrestre en el lado izquierdo de la pantalla y luego seguir agregando extraterrestres hasta quedarse sin espacio:

Obtenemos el ancho del alienígena del primer alienígena que creamos y luego definimos una variable llamada `actual_x`. Esto se refiere a la posición horizontal del próximo extraterrestre que pretendemos colocar en la pantalla. Inicialmente configuramos esto para el ancho de un alienígena, para luego desplazar del primer alienígena de la flota desde el borde izquierdo de la pantalla.

```
invasión      def _crear_flota(self):
_alien.py      """Crear la flota de alienígena."""
                # Hacer un alienígena
                # Espacio entre alienígenas igual un tercio del ancho del alienígena
                alien = Alien(self)
                ancho_alien = alien.rect.width
                actual_x = alien_width
                while actual_x < (self.configuracion.pantalla_width -
                                1.3 * ancho_alien):
                    nuevo_alien = Alien(self)
                    nuevo_alien.x = actual_x
                    nuevo_alien.rect.x = actual_x
                    self.aliens.add(nuevo_alien)
                    actual_x += 1.5 * ancho_alien
```

A continuación, comenzamos el ciclo `while`; vamos a seguir agregando extraterrestres mientras haya suficiente espacio para colocar uno. Para determinar si hay espacio para colocar otro alienígena, compararemos `while actual_x < (self.configuracion.ancho_pantalla` con algún valor máximo. El primer intento de definir este bucle podría verse así:

```
while actual_x < self.configuracion.ancho_pantalla:
```

Esto parece que podría funcionar, pero colocaría al alienígena final de la fila en el extremo derecho de la pantalla. Entonces agregamos un pequeño margen en el lado derecho de la pantalla. Siempre y cuando haya al menos dos anchos alienígenas de espacio en el borde derecho de la pantalla, ingresamos al bucle y agregamos otro a la flota.

Siempre que haya suficiente espacio horizontal para continuar el bucle, queremos hacer dos cosas: crear un extraterrestre en la posición correcta y definir la posición horizontal del siguiente extraterrestre en la fila. Creamos un alienígena y lo asignamos a `nuevo_alien`.

Luego establecemos la posición horizontal precisa en el valor actual de `actual_x`. También posicionamos el `rect` del alienígena en este mismo valor `x` y agregamos el nuevo alienígena al grupo `self.aliens`.

Finalmente, incrementamos el valor de `actual_x`. Agregamos dos anchos de alienígenas a la posición horizontal, para pasar el alienígena que acabamos de agregar y dejar algo de espacio entre los alienígenas también.

Python reevaluará la condición al comienzo del ciclo `while` y decidirá si hay espacio para otro extraterrestre. Cuando no quede espacio, el bucle terminará y deberíamos tener una fila completa de alienígenas.

Al ejecutar `invasión_alien.py`, deberías ver aparecer la primera fila de alienígenas, como en la Figura 6.3.



Figura 6.3: Aparece la primera fila de extraterrestres en pantalla.

NOTA



No debe ser motivo de para desanimarse al construir un bucle como el que se muestra en esta sección, y colocar demasiado alienígenas a la derecha. Una cosa buena de la programación es que sus ideas iniciales sobre cómo abordar un problema como este no tienen por qué ser correctas. Es perfectamente razonable iniciar un bucle como este con los alienígenas colocados demasiado a la derecha y luego modificar el bucle hasta que tengas una cantidad adecuada de espacio en la pantalla.

6.2.3.2. Refactorización del método `_crear_flota()`

Si el código que hemos escrito hasta ahora fuera todo lo que necesitábamos para crear una flota, probablemente dejaríamos `_crear_flota()` como está. Pero tenemos más trabajo por hacer, así que limpiemos un poco el método. Agregaremos un nuevo método auxiliar, `_crear_alien()`, y lo llamaremos desde `_crear_flota()`:

```
invasión      def _create_flota(self):
_alien.py    --recorte--
              while actual_x < (self.configuracion. ancho_pantalla -
```

```

        1.3 * ancho_alien):
    self._crear_alien(actual_x)
    actual_x += 2 * ancho_alien

def _crear_alien(self, x_posicion):
    """Crear un alienígena y colocar en la fila."""
    nuevo_alien = Alien(self)
    nuevo_alien.x = x_posicion
    nuevo_alien.rect.x = x_posicion
    self.aliens.add(nuevo_alien)

```

El método `_crear_alien()` requiere otro parámetro además de `self`: el valor `x` que especifica dónde se debe colocar el extraterrestre. El código en el método `_crear_alien()` es el mismo código que estaba en `_crear_flota()`: excepto que usamos el parámetro `x_posicion` en lugar de `actual_x`. La refactorización facilitará la adición de nuevas filas y la creación de una flota completa.

6.2.3.3. Agregar filas de alienígenas

Para terminar la flota, seguiremos agregando más filas hasta que nos quedemos sin espacio.

Usaremos un bucle anidado; envolveremos otro bucle `while` alrededor del bucle actual. El bucle interior colocará a los extraterrestres horizontalmente en una fila enfocándose en los valores `x` de los extraterrestres.

El bucle exterior colocará a los extraterrestres verticalmente enfocándose en los valores de `y`. Dejaremos de agregar filas cuando nos acerquemos al final del pantalla, dejando suficiente espacio para la nave y algo de espacio para comenzar a disparar los alienígenas.

Aquí se explica cómo anidar los dos bucles `while` en `_crear_flota()`:

```

def _crear_flota(self):
    """Create the fleet of aliens."""
    # Crear, guardar y anadir alienigenas hasta llenar la pantalla
    # Espacio entre alienigenas es igual al ancho y alto del alienigena.
    alien = Alien(self)
    ancho_alien, alto_alien = alien.rect.size
    actual_x, actual_y = ancho_alien, alto_alien

    while actual_y < (self.configuracion.pantalla_height -
                     6 * alto_alien):
        while actual_x < (self.configuracion.pantalla_width -
                         1.3 * ancho_alien):
            self._crear_alien(actual_x, actual_y)
            actual_x += 1.3 * ancho_alien

        # finaliza una fila; resetea el valor de $x$,

```

```
# e incrementa el valor de $y$.
actual_x = ancho_alien
actual_y += 1.5 * alto_alien
```

Necesitaremos saber la altura del alienígena para poder colocar filas, así que tomamos el ancho y alto del extraterrestre usando el atributo de tamaño de un extraterrestre recto 1. A

El atributo de tamaño de `rect` es una tupla que contiene su ancho y alto.

A continuación, establecemos los valores iniciales x e y para la ubicación del primer alienígena. en la flota. Lo colocamos a un ancho alienígena desde la izquierda y un alienígena altura hacia abajo desde la parte superior.

Luego definimos el bucle `while` que controla cómo Se colocan muchas filas en la pantalla. Siempre que el valor de y para el siguiente fila es menor que la altura de la pantalla, menos tres alturas alienígenas, mantendremos añadiendo filas. (Si esto no deja la cantidad adecuada de espacio, podemos ajustarlo más tarde.)

Llamamos a `_crear_alien()` y le pasamos el valor y así como su posición x 4. Modificaremos `_crear_alien()` en un momento. Observe la sangría de las dos últimas líneas del código 5. Están dentro del bucle `while` externo, pero fuera del bucle `while` interno. Este bloque se ejecuta una vez finalizado el bucle interno; se ejecuta una vez después de crear cada fila. Después de agregar cada fila, restablecemos el valor de `actual_x` para que el primer alienígena de la siguiente fila se coloque en la misma posición que el primer alienígena de las filas anteriores. Luego agregamos dos alturas alienígenas al valor actual de `actual_y`, por lo que la siguiente fila se colocará más abajo en la pantalla. La sangría es realmente importante aquí; si no ves la flota correcta cuando ejecutas `invasion_alien.py`

Al final de esta sección, verifique la sangría de todas las líneas en estos bucles anidados.

Necesitamos modificar `_crear_alien()` para establecer correctamente la posición vertical del extraterrestre:

curso.py

```
def _crear_alien(self, x_posicion, y_posicion):
    """Crear un alienígena y colocar en la flota."""
    nuevo_alien = Alien(self)
    nuevo_alien.x = x_posicion
    nuevo_alien.rect.x = x_posicion
    nuevo_alien.rect.y = y_posicion
    self.aliens.add(nuevo_alien)
```

Modificamos la definición del método para aceptar el valor y para el nuevo alienígena y configuramos la posición vertical de `rect` en el cuerpo del método.

Cuando ejecutes el juego ahora, deberías ver una flota completa de alienígenas, como se muestra en la Figura 6.4.

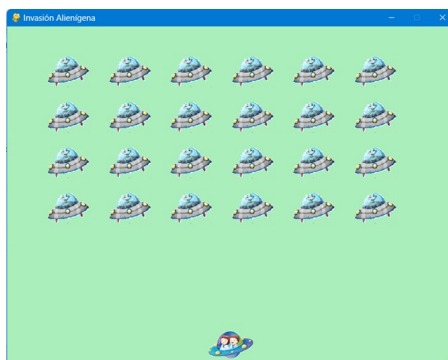


Figura 6.4: Aparece la flota completa.

En la siguiente sección, haremos que la flota se mueva.

6.3. INTÉNTALO TÚ MISMO

13-1. Estrellas: Encuentra una imagen de una estrella. Haz que aparezca una cuadrícula de estrellas en la pantalla.

13-2. Mejores estrellas: puedes crear un patrón de estrellas más realista introduciendo aleatoriedad al colocar cada estrella. Recuerde del Capítulo 9 que puede obtener un número aleatorio como este:

6.4. La flota se mueve

Ahora hagamos que la flota de alienígenas se mueva hacia la derecha a través de la pantalla hasta golpea el borde, y luego del choque se hace caer una cantidad determinada y se mueve en el otra dirección. Continuaremos este movimiento hasta que todos los alienígenas hayan sido fusilados, uno choca con la nave o llegan al final de la pantalla.

Comencemos haciendo que la flota se mueva hacia la derecha.

6.4.0.1. Los extraterrestres se mueven hacia la derecha

Para mover a los alienígenas, usaremos un método `actualizar()` en `alien.py`, que llama a cada extraterrestre del grupo de extraterrestres. Primero, agregue una configuración para controlar la velocidad de cada alienígena:

```
configurar.py    def __init__(self):
                  --recorte--
                  # Configuración Alienígena
                  self.velocidad_alien = 1.0
```

Luego usaremos esta configuración en el método `actualizar()` del módulo `alien.py`:

```
alien.py         def __init__(self, ia_jugar):
                  """Inicializa el alienígena y configuramos su posición inicial."""
                  super().__init__()
                  --recorte--

                  def actualizar(self):
                      """Mover el alienígena hacia la derecha."""
                      self.x += self.configurar.velocidad_alien
                      self.rect.x = self.x
```

Creamos un parámetro de configuración en `__init__()` para que podamos acceder a la velocidad de alienígena en el método `actualizar()`. Cada vez que actualizamos la posición de un alienígena, lo movemos la cantidad almacenada en `velocidad_alien`. Realizamos un seguimiento de la posición exacta del alienígena con el atributo `self.x`, que puede contener valores flotantes. Luego usamos el valor de `self.x` para actualizar la posición del `rect` del alienígena.

En el ciclo `while` principal, ya tenemos llamadas para actualizar las posiciones de la nave y las balas. Ahora también agregaremos una llamada para actualizar la posición de cada alienígena:

```
invasión
_alien.py        while True:
                  self._chequear_eventos()
                  self.nave.actualizar()

                  self.balas.update()
                  self._actualizar_balas()
                  self._actualizar_alien()
                  self._actualizar_pantalla()

                  self.nave.blitme()
                  # Actualiza. Hace visible la pantalla dibujada más recientemente
                  pygame.display.flip()
                  self.clock.tick(60)
```

Estamos a punto de escribir un código para gestionar el movimiento de la flota, por lo que creamos un nuevo método llamado `actualizar_alien()`. Actualizamos las posiciones de los alienígenas después de que se hayan actualizado las balas, porque pronto comprobaremos si alguna bala alcanzó a algún alienígena.

El lugar donde se coloque este método en el módulo no es crítico. Pero para mantener el código organizado y claro, lo colocaré justo después de `actualizar_balas()` para que coincida con el orden de las llamadas a métodos en el bucle `while`. Aquí está la primera versión de `actualizar_alien()`:

```
invasión
_alien.py    def _actualizar_alien(self):
              """Actualiza la posiciones de todos las alienígena de la flota."""
              self.aliens.update()
```

Usamos el método `update()` en el grupo de extraterrestres, que llama al método `update()` de cada extraterrestre.

Al ejecutar `Invasión_Alien`, deberías ver que la flota se mueve hacia la derecha y desaparece del costado de la pantalla.

6.4.0.2. Crear configuraciones para la dirección de la flota

Ahora crearemos la configuración que hará que la flota se mueva hacia abajo y hacia la izquierda cuando llegue al borde derecho de la pantalla.

Agregamos la configuración `flota_cae_velocidad` que controla la rapidez con la que la flota desciende de la pantalla cada vez que un extraterrestre llega a cualquiera de los bordes.

```
alien.py    # Configuración Alienígena
            self.velocidad_alien = 1.0
            self.flota_cae_velocidad = 10    # velocidad_flota_cae
            # Dirección de flota: 1 representa derecha;
            # -1 representa izquierda.
            self.direccion_flota = 1
```

Es útil separar esta velocidad de la velocidad horizontal de los alienígenas para que puedas ajustar las dos velocidades de forma independiente.

Para implementar la configuración de `direccion_flota`, podríamos usar un valor tipo texto como `'izquierda'` o `'derecha'`, pero terminaríamos con declaraciones `if - elif` que prueban la dirección de la flota. En cambio, debido a que solo tenemos dos direcciones con las que lidiar, usemos los valores `1` y `-1`, cambiemos entre ellos cada vez que la flota cambia de dirección. (Usar números también tiene sentido porque moverse hacia la derecha implica agregar al valor de la coordenada x de cada alienígena, y moverse hacia la izquierda implica restando del valor de la coordenada x de cada extraterrestre.)

6.4.0.3. Alienígena ha llegado al borde

Necesitamos un método para comprobar si hay un extraterrestre en alguno de los bordes, y necesitamos modificar `update()` para permitir que cada alienígena se mueva en la dirección apropiada.

Este código es parte de la clase `Alien`:

```
alien.py def chequear_bordes(self):
    """Retorna True si el alienígena esta en el borde de la pantalla (ventana)."""
    pantalla_rect = self.pantalla.get_rect()
    return (self.rect.right >= pantalla_rect.right) or (self.rect.left <= 0)

def update(self):
    """Mueve el alienígena a la izquierda o derecha."""
    self.x += (self.configuracion.velocidad_alien *
               self.configuracion.direccion_flota)
    self.rect.x = self.x
```

Podemos llamar al nuevo método `chequear_bordes()` en cualquier extraterrestre para ver si está en el borde izquierdo o derecho. El alienígena está en el borde derecho si el atributo del rectángulo de su `rect` es mayor o igual que el atributo derecho `rect` de la pantalla. Está en el borde izquierdo si su valor izquierdo es menor o igual a 0.

Más bien, en lugar de poner esta prueba condicional en un bloque `if`, ponemos la prueba directamente en el declaración de devolución.

Este método devolverá `True` si el alienígena está a la derecha o borde izquierdo y `False` si no está en ninguno de los bordes.

Modificamos el método `actualizar()` para permitir movimiento hacia la izquierda o hacia la derecha multiplicando la velocidad del alienígena por el valor de `direccion_flota`. Si `direccion_flota` es 1, el valor de `velocidad_alien` se agregará al valor posición actual del alienígena, moviendo al extraterrestre hacia la derecha; si `direccion_flota` es -1, el valor se le restará de la posición del alienígena, moviéndolo hacia la izquierda.

6.4.0.4. Abandonar la flota y cambiar de dirección

Cuando un extraterrestre llega al borde, toda la flota debe descender y cambia la dirección. Por lo tanto, necesitamos agregar algo de código a la clase `InvasionAlien` porque ahí es donde comprobaremos si hay extraterrestres a la izquierda o a la derecha del borde.

Haremos que esto suceda escribiendo los métodos `_chequear_borde_flota()` y `_cambiar_direccion_flota()`, y luego modificando `_actualizar_alien()`.

Estos nuevos métodos lo ubicare después de `_crear_alien()`, pero nuevamente, la ubicación de estos dos métodos en la clase no son críticos.

```
invasión
_alien.py
def _chequear_borde_flota(self):
    """Responde apropiadamente si cualquier
    alienígena alcanza el borde."""
    for alien in self.aliens.sprites():
        if alien.chequear_borde():
            self._cambiar_direccion_flota()
            break
def _cambiar_direccion_flota(self):
    """Cae la flota y cambia de direccion."""
    for alien in self.aliens.sprites():
        alien.rect.y += self.configuracion.flota_cae_velocidad
        self.configuracion.flota_cae_velocidad *= -1
```

En `_chequear_borde_flota()`, recorreremos la flota y llamamos a `chequear_borde()` en cada alienígena. Si `chequear_borde()` devuelve `True`, sabemos que un alienígena está en un borde y toda la flota necesita cambiar de dirección; entonces llamamos a `_cambiar_direccion_flota()` y salimos del bucle. En `_cambiar_direccion_flota()`, recorreremos todos los alienígenas y soltamos a cada uno usando la configuración `flota_cae_velocidad`; luego cambiamos el valor de `_cambiar_direccion_flota()` multiplicando su valor actual por `-1`. La línea que cambia la dirección de la flota no forma parte del bucle `for`. Queremos cambiar la posición vertical de cada alienígena, pero sólo queremos cambiar la dirección de la flota una vez.

Veamos los cambios en el método `_actualizar_aliens()`:

```
invasión
_alien.py
def _actualizar_aliens(self):
    """Actualiza la posiciones de todos los alienígenas de la flota."""
    self._chequear_borde_flota
    self.aliens.update()
```

Modificamos el método llamando a `_chequear_borde_flota()` antes de actualizar la posición de cada alienígena.

Cuando ejecutas el juego ahora, la flota debería moverse hacia adelante y hacia atrás entre los bordes de la pantalla y descender cada vez que llegue a un borde. Ahora podemos empezar a derribar alienígenas y estar atentos a cualquier alienígena que golpee la nave o llegue al final de la pantalla.

6.5. INTÉNTALO TÚ MISMO

13-3. Gotas de lluvia: busque una imagen de una gota de lluvia y cree una cuadrícula de gotas de lluvia. Haz que las gotas de lluvia caigan hacia la parte inferior de la pantalla hasta que desaparezcan.

13-4. Lluvia constante: Modifique su código en el Ejercicio 13-3 para que cuando una fila de gotas de lluvia desaparezca de la parte inferior de la pantalla, aparezca una nueva fila en la parte superior de la pantalla y comience a caer.

6.5.0.1. Disparar extraterrestres

Hemos construido nuestra nave y una flota de extraterrestres, pero cuando las balas alcanzan a los extraterrestres, simplemente pasan porque no estamos comprobando si hay colisiones. En la programación de juegos, las colisiones ocurren cuando los elementos del juego se superponen. Para hacer que las balas derriben a los extraterrestres, usaremos la función `sprite.groupcollide()` para buscar colisiones entre miembros de dos grupos.

6.6. Detección de colisiones de alienígenas con las balas

Necesitamos saber cuándo una bala alcanza a un alienígena para poder tomar la decisión de desaparece el alienígena tan pronto como sea golpeado por una bala (alcanzado). Para hacer esto, buscaremos colisiones inmediatamente después de actualizar la posición de todas las balas.

La función `sprite.groupcollide()` compara los rectángulos (objetos `Rect`) de cada elemento en un grupo con los rectángulos de cada elemento del otro grupo. En este caso se compara el rectángulo de cada bala con el rectángulo de cada alienígena y devuelve un diccionario que contiene las balas y los alienígenas que han chocado. Cada clave en el diccionario será una bala, y el valor correspondiente será el alienígena que fue alcanzado. (Bueno, también utilizaremos este diccionario cuando implementemos un sistema de puntuación en el Capítulo ??.)

Agregue el siguiente código al final de `_actualizar_balas()` para verificar si hay colisiones entre balas y extraterrestres:

```
invasión
_alien.py    def _actualizar_balas()(self):
              """Actualizamos posición de balas y o btenemos rid de la balas viejas."""
              --recorte--
              # Chequea para cuaquier bala que tiene hit alienigena.
              # Si es asi, obtiene rid de bala y alienigena.
              colisiones = pygame.sprite.groupcollide(
                          self.balas, self.aliens, True, True)
```

El nuevo código que agregamos compara las posiciones de todas las balas en `self.balas` y todos los alienígenas en `self.aliens`, e identifica aquellos que se superponen. Siempre que los rectángulos de una bala y un alienígena se superponen, `groupcollide()` agrega un valor clave par al diccionario que devuelve. Los dos argumentos verdaderos le indican a PyGame borrar las balas y los alienígenas que han chocado. (Para hacer un potente bala que puede viajar a la parte superior de la pantalla, destruyendo a todos los alienígenas en su ruta, puede establecer el primer argumento booleano en Falso y mantener el segundo Argumento booleano establecido en Verdadero. Los alienígenas impactados desaparecerían, pero todas las balas permanecería activo hasta que desaparecieran de la parte superior de la pantalla).

Cuando ejecutes Alien Invasion ahora, los alienígenas que golpees deberían desaparecer.

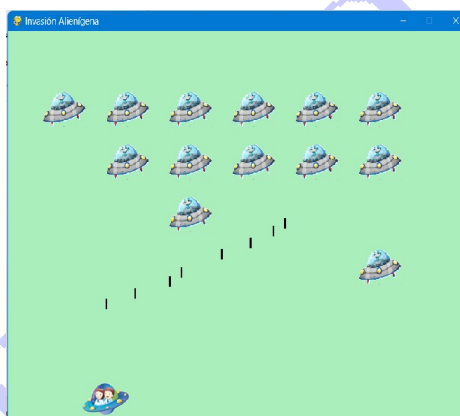


Figura 6.5: La flota ha sido parcialmente derribada.

La Figura 6.5 muestra un flota que ha sido parcialmente derribada .

6.7. Hacer balas más poderosas para realizar pruebas

Puedes probar muchas funciones de Alien Invasion simplemente ejecutando el juego, pero algunas funciones son tediosas de probar en la versión normal del juego. Por ejemplo, es mucho trabajo derribar a cada alienígena en la pantalla varias veces para probar si su código responde correctamente a una flota vacía.

Para probar funciones particulares, puedes cambiar ciertas configuraciones del juego para enfocarte en un área particular. Por ejemplo, puedes reducir la pantalla para que haya menos alienígenas a los que derribar o aumentar la velocidad de las balas y recibir muchas balas a la vez.

Mi cambio favorito para probar Alien Invasion es usar balas muy anchas que permanecen activas incluso después de haber golpeado a un alienígena (ver Figura 6.6). Intenta configurar `ancho_Balas` en 100, o incluso en 300, para ver qué tan rápido puedes derribar la flota



Figura 6.6: Balas más poderosas.

Las balas más poderosas hacen algunos aspectos del juego más fácil de ganar (Figura 6.6).

Cambios como estos te ayudarán a probar el juego de manera más eficiente y posiblemente generarán ideas para otorgar poderes adicionales a los jugadores. Sólo recuerde restaurar la configuración a la normalidad cuando haya terminado de probar una función.

6.7.1. Reposición de la flota

Una característica clave en el juego Invasión Alienígena, es que los alienígenas son implacables, cada vez que se destruye una flota, aparecer una nueva flota.

Para hacer que aparezca una nueva flota de alienígenas después de que una flota haya sido destruida, primero comprobamos si el grupo de alienígenas

está vacío. Si es así, hacemos una llamada a `_crear_flota()`.

Realizaremos esta verificación al final de `_actualizar_balas()`, porque ahí es donde se destruyen los extraterrestres individuales.

```
invasión      def _actualizar_balas(self):
_alien.py      --recorte--
                if not self.aliens:
                    # Destruye la existencia de las balas y crea una nueva flota.
                    self.balas.empty()
                    self._crear_flota()
```

Comprobamos si el grupo de extraterrestres está vacío. Se evalúa un grupo vacío a `False`, por lo que esta es una forma sencilla de comprobar si el grupo está vacío. Si es así, nosotros deshazte de las balas existentes utilizando el método `vacío()`, que elimina todos los sprites restantes de un grupo. También llamamos `_crear_flota()`, que vuelve a llenar la pantalla con extraterrestres.

Ahora aparece una nueva flota tan pronto como destruyes la flota actual.

Acelerando las balas

Si has intentado disparar a los alienígenas en el estado actual del juego, es posible que encuentres que las balas no viajan a la mejor velocidad para el juego. Podrían ser un poco demasiado lento o un poco demasiado rápido. En este punto, puede modificar la configuración, para hacer el juego más interesante.

Tenga en cuenta que el juego se volverá progresivamente más rápido, así que no haga que el juego sea demasiado rápido al comienzo.

Modificamos la velocidad de las balas ajustando el valor de `velocidad_balas` en `configurar.py`. En mi sistema, ajustaré el valor de `bullet_speed` a 2,5, para que el las balas viajan un poco más rápido:

```
configurar.py # Configuración de balas
                self.bullet_speed = 2.5
                self.bullet_width = 3
                --recorte--
```

El mejor valor para esta configuración depende de tu experiencia en el juego, así que encuentre un valor que funcione para usted. También puedes ajustar otras configuraciones.

6.7.2. Refactorización `_actualizar_balas()`

Refactoricemos `_actualizar_balas()` para que no realice tantas tareas diferentes.

Moveremos el código para lidiar con colisiones entre balas y extraterrestres a una sección separada. método:

```

invasión
_alien.py

def _actualizar_balas(self):
    --recorte--
    # obtener rid de las balas que se han disparado.
    for balas in self.balas.copy():
        if balas.rect.bottom <= 0:
            self.balas.remove(balas)
    self._chequear_colisiones_balas_alien()

def _chequear_colisiones_balas_alien(self):
    """Responder a las colisiones balas-alienígena."""
    # Remover cualquier bala y alienígena que han colisionado.
    colisiones = pygame.sprite.groupcollide( self.balas, self.aliens, True, True)
    if not self.aliens:
        # Destruye la existencia de las balas y crea una nueva flota.
        self.balas.empty()
        self._crear_flota()

```

Hemos creado un nuevo método, `_chequear_colisiones_balas_alien`, para buscar colisiones entre balas y extraterrestres y responder adecuadamente si toda la flota ha sido destruida. Hacerlo evita que `_actualizar_balas()` crezca demasiado y simplifica el desarrollo posterior.

6.8. INTÉNTALO TÚ MISMO

13-5. Tirador lateral Parte 2: Hemos recorrido un largo camino desde el Ejercicio 12-6, Tirador lateral. Para este ejercicio, intenta desarrollar Sideways Shooter hasta el mismo punto al que llevamos Alien Invasion.

Añade una flota de alienígenas y haz que se muevan de lado hacia la nave o escribe un código que coloque extraterrestres en posiciones aleatorias a lo largo del lado derecho de la pantalla y luego los envíe hacia la nave. Además, escribe código que haga que los alienígenas desaparezcan cuando sean golpeados.

6.9. Terminando el juego

¿Cuál es la diversión y el desafío de jugar un juego que no puedes perder?

Si el jugador no derriba la flota lo suficientemente rápido, haremos que los alienígenas destruyan la nave cuando hagan contacto. Al mismo tiempo, limitaremos la cantidad de naves que un jugador puede usar y destruiremos la nave cuando un extraterrestre llegue al final de la pantalla. El juego terminará cuando el jugador haya agotado todos sus la naves.

6.10. Detección de colisiones

Comenzaremos verificando colisiones entre alienígenas y la nave, para que podamos responder adecuadamente cuando un alienígena golpee la nave. Comprobaremos si hay colisiones entre naves y alienígenas inmediatamente después de actualizar la posición de cada alienígena en `InvasionAlien`:

```
invasión \begin{lstlisting}
_alien.py def _actualiar_alien(self):
            --recorte--
            self.aliens.update()
            # Ver colisiones entre alienígena y nave.
            if pygame.sprite.spritecollideany(self.nave07, self.aliens):
                print("!!!Nave golpeada!!!")
```

La función `spritecollideany()` toma dos argumentos: un sprite (nave) y un grupo (alien). La función busca cualquier miembro del grupo que haya chocado con el sprite y deja de recorrer el grupo tan pronto como encuentra un miembro que ha chocado con el sprite.

Aquí, recorre el grupo de alienígenas y devuelve al primer alienígena que encuentra que ha chocado con la nave.

Si no ocurren colisiones, `spritecollideany()` devuelve Ninguno y el bloque `if` no ejecutará. Si encuentra un alienígena que ha chocado con la nave, regresa ese alienígena y el bloque `if` se ejecuta: imprime `!!!Nave golpeada!!!`.

Cuando un extraterrestre golpea la nave, tendremos que realizar una serie de tareas: eliminar a todos los alienígenas restantes y balas, volver a centrar la nave y crear una nueva flota.

Para escribir código ese código debemos saber si nuestro enfoque para detectar colisiones de naves alienígenas funciona correctamente. Escribir un `print()` como un indicador, es una forma sencilla y muy práctica de asegurarnos de que estamos detectando adecuadamente estas colisiones.

Ahora, cuando ejecutas `Invasión Alienígena`, debe aparecer el mensaje `!!!Nave golpeada!!!` en la terminal cada vez que un extraterrestre golpea la nave.

Cuando estás probando esta característica, establezca `cae_velocidad_flota` en un valor más alto, como 50 o 100, para que los extraterrestres lleguen a tu nave más rápido.

6.10.0.1. Respondiendo a las colisiones entre naves alienígenas

Ahora necesitamos descubrir exactamente qué sucederá cuando un extraterrestre colisione con la nave. En lugar de destruir la nave y crear una nueva,

contaremos cuántas veces la nave ha sido impactado mediante las estadísticas de seguimiento para el juego. Las estadísticas de seguimiento también serán útiles para las puntuaciones.

Escribamos una nueva clase, `Estadisticas_Juego()`, para realizar un seguimiento de las estadísticas del juego y guárdela. como `stats_juego.py`:

```
stats
_juego.py class Estadisticas_Juego():
            """Track estadísticas para el juego Invasión Alienígena."""
            def __init__(self, ia_jugar):
                """Inicializa estadísticas."""
                self.configuracion = ia_jugar.configuracion
                self.resetear_stats()

            def resetear_stats(self):
                """Inicializa estadísticas que cambian durante el juego."""
                self.nave_permitidas = self.configuracion.limite_nave
```

Creamos una instancia de `Estadisticas_Juego()` durante todo el tiempo que dure `Invasión Alienígena` ejecutándose, pero necesitaremos restablecer algunas estadísticas cada vez que el jugador inicie un nuevo juego. Para hacer esto, inicializaremos la mayoría de las estadísticas en el método `resetear_stats()`, en lugar de escribirlas en `__init__()`.

Llamaremos a este método desde `__init__()` por lo que las estadísticas se configuran correctamente cuando se crea por primera vez la instancia de `Estadisticas_Juego()`. Pero también podremos llamar a `resetear_stats()` cada vez que el jugador comience un nuevo juego.

En este momento solo tenemos una estadística, la `nave_permitidas`, cuyo valor cambiará durante todo el juego. La cantidad de naves que dispone el jugador se almacenan en `configurar.py` como `limite_nave`:

```
configurar.py # Configurar Nave
               self.velocidad_nave = 10
               self.limite_nave = 3
```

También necesitamos realizar algunos cambios en `invasión_alien.py` para crear una instancia de `Estadisticas_Juego()`. Primero, actualizaremos las declaraciones para importar en la parte superior del archivo:

```
invasión
_alien.py import sys
           from time import sleep
           import pygame
           from configurar import Configuracion
           from stats_juego import Estadisticas_Juego
           from Nave import Nave
           --recorte--
```

Importamos la función `sleep()` del módulo de `time` de la biblioteca estándar de Python, para que podamos pausar el juego por un momento cuando

la nave es golpeado. También importamos `Estadisticas_Juego()`. Y crearemos una instancia de `Estadisticas_Juego()` en `__init__()`:

```
invasión
_alien.py
def __init__(self):
    --recorte--
    self.pantalla = pygame.pantalla.set_mode(
        (self.configuracion.pantalla_width,
         self.configuracion.pantalla_height))
    pygame.display.set_caption("Invasión Alienígena")

    # Crea una instancia para almacenar estadísticas del juego.
    self.estadisticas = Estadisticas_Juego(self)
    self.nave = Nave(self)
    --recorte--
```

Creamos la instancia después de crear la ventana del juego, pero antes de definir otros elementos del juego, como por ejemplo la nave. Cuando un extraterrestre golpee la nave, restaremos 1 del número de naves que quedan, y destruimos todos los extraterrestres y las balas existentes, crearemos una nueva flota y reponemos la nave la posición inicial en la pantalla.

También pausaremos el juego por un momento para que el jugador pueda notar la colisión y reagruparse antes de que aparezca una nueva flota.

Crearemos un nuevo método llamado `_nave_golpeada()`, y escribimos la mayor parte de este código en él. En el método `_actualizar_alien()` se hace el llamado al nuevo método, cuando un extraterrestre golpee la nave:

```
invasión
_alien.py
def _nave_golpeada(self):
    """Responde a la nave es golpeada por un alienígena."""
    # Decrementa nave_
    self.estadisticas.nave_permitidas -= 1
    # Obtiene rid de cualquier remaining bala y alienígena
    self.balas.empty()
    self.aliens.empty()
    # Crea una nueva flota y una nave en el centro
    self._crear_flota()
    self.nave.nave_centro()
    # Pause.
    sleep(0.5)
```

El nuevo método `_nave_golpeada()` coordina la respuesta cuando un extraterrestre choca contra la nave. Dentro de `_nave_golpeada()` se reduce en 1, el número de la naves que quedan y se procede a vaciar los grupos de balas y alienígenas.

Se continua creando una nueva flota y colocando la nave en su posición inicial. (Agregaremos el método `nave_centro()` para iniciar la nave). Luego agregamos una pausa después de realizar actualizaciones en todos los elementos del juego, pero antes de cualquier cambio en la pantalla, para permitirle al jugador ver que su nave ha sido golpeada.

La llamada `sleep()` detiene la ejecución del programa durante medio segundo, tiempo suficiente para que el jugador vea que el extraterrestre ha golpeado la nave.

Cuando la función `sleep()` finaliza, la ejecución del código pasa al método `_actualizar_pantalla()`, que dibuja la nueva flota en la pantalla.

En `_actualizar_alien()`, reemplazamos la llamada `print()` con una llamada a `_nave_golpeada()` cuando un extraterrestre choca con otra la nave:

```
invasión      def _actualizar_alien(self):
_alien.py      --recorte--
                if pygame.sprite.spritecollideany(self.nave, self.aliens):
                    self._nave_golpeada()
```

El nuevo método `nave_centro()` que centra la nave en la pantalla, se agrega en el archivo `nave.py`:

```
nave.py      def nave_centro(self):
                """Centra la nave en la pantalla."""
                self.rect.midbottom = self.pantalla_rect.midbottom
                self.x = float(self.rect.x)
```

Centramos la nave de la misma manera que lo hicimos en `__init__()`. Después de centrarlo, restablecemos el atributo `self.x`, que nos permite rastrear la posición exacta de la nave.

Tenga en cuenta que nunca se crea más de una nave; hacemos solo una instancia de la nave para el todo el juego y se vuelve a centrar cada vez que la nave haya sido golpeado. La estadística `se_naves_permitidas` nos avisará cuando el jugador se haya quedado sin naves.

Al ejecutar el juego, dispare a algunos extraterrestres y deje que un extraterrestre golpee la nave.

El juego debería hacer una pausa y debería aparecer una nueva flota con ella nave centrado en el parte inferior de la pantalla nuevamente.

6.10.0.2. Aliens que llegan al final de la pantalla

Si un extraterrestre llega al final de la pantalla, el juego responderá de la misma manera que ocurre cuando un extraterrestre choca contra la nave. Para comprobar cuándo sucede esto, agregue un nuevo método en `invasion_alien.py`:

```
invasión      def _chequear_alien_piso(self):
_alien.py      """Verifica si algún alienígena alcanza el piso de la pantalla."""
                for alien in self.aliens.sprites():
                    if alien.rect.bottom >= self.configuraciones.pantalla_height:
```

```

        # Tratado de la misma forma como si la nave es golpeada.
        self._nave_golpeada()
        break

```

El método `_check_aliens_bottom()` comprueba si algún extraterrestre ha llegado al final de la pantalla. Un extraterrestre llega al fondo cuando su valor `rect.bottom` es mayor o igual a la altura de la pantalla 1. Si un extraterrestre llega al fondo, llamamos `_ship_hit()`. Si un alienígena toca el fondo, no hay necesidad de comprobar el resto, por lo que salimos del bucle después de llamar a `_ship_hit()`. Llamaremos a este método desde `_update_aliens()`:

```

invasión
_alien.py    def _actualizar_aliens(self):
               --recorte--
               # ver colisiones entre alien-nave.
               if pygame.sprite.spritecollideany(self.nave, self.aliens):
                   self._nave_golpeada()
               # Ver alienígena golpea el piso de la pantalla.
               self._chequear_aliens_piso()

```

Llamamos a `self._chequear_aliens_piso()` después de actualizar las posiciones de todos los alienígenas y después de buscar colisiones entre naves alienígenas. Ahora aparecerá una nueva flota cada vez que la nave sea golpeada por un extraterrestre o un extraterrestre llegue al final de la pantalla.

¡Juego terminado!

Invasión se siente más completo ahora, pero el juego nunca termina. El valor de las naves dejados se vuelve cada vez más negativo.

Agreguemos un indicador `juego_activo`, para que podamos finalizar el juego cuando el jugador se quede sin naves. Estableceremos esta indicador al final del método `__init__()` en `InvasionAlien`:

```

invasión
_alien.py    def __init__(self):
               --recorte--
               # Comienza Invasión Alien en un estado activo.
               self.juego_activo = True

```

En el método `_nave_golpeada()` agregamos el indicador `juego_activo = False`, para señala que el jugador ha agotado todos sus naves:

```

invasión
_alien.py    def _nave_golpeada(self):
               """Responde cuando la nave ha sido golpeada por un alienígena."""
               if self.estadisticas.nave_permitidas > 0:
                   # Restamos 1 a nave_permitidas
                   self.estadisticas.nave_permitidas -= 1
               --recorte--
               # Pause.
               sleep(0.5)
               else:
                   self.juego_activo = False

```

La mayor parte del método `_nave_golpeada()` no ha cambiado. Hemos movido todo el código existente en un bloque `if`, que prueba para asegurarse de que el jugador tenga al menos una nave restante. Si es así, creamos una nueva flota, hacemos una pausa y seguimos adelante. Si al jugador no quedan naves, configuramos `juego_activo` en `False`.

6.11. Identificar cuándo deben ejecutarse las partes del juego

Necesitamos identificar las partes del juego que siempre deben ejecutarse y las partes que deberían ejecutarse sólo cuando el juego está activo:

```
invasión
_alien.py
def run_jugar(self):
    """Start the main loop for the game."""
    while True:
        self._check_events()
        if self.juego_activo:
            self.nave.update()
        self._actualizar_balas()
        self._actualizar_alien()
        self._actualizar_pantalla()
        self.clock.tick(60)
```

En el bucle principal, siempre es necesario llamar al método `_chequear_eventos()`, incluso si el juego está inactivo. Por ejemplo, necesitamos saber si el usuario presiona `Q` para salir del juego o hace clic en el botón para cerrar la ventana. También continuamos actualizando la pantalla para que podamos realizar cambios en la pantalla mientras esperamos ver si el jugador elige comenzar un nuevo juego.

El resto de las funciones deben ser llamadas cuando el juego está activo, porque cuando el juego está inactivo, no necesitamos actualizar las posiciones de los elementos del juego. Ahora, cuando juegues Alien Invasion, el juego debería congelarse cuando hayas consumido todas las naves permitidas.

6.12. INTÉNTALO TÚ MISMO

13-6. Fin del juego: en Sideways Shooter (dispara de un lado), lleva la cuenta del número de veces que la nave es golpeada y el número de veces que la nave golpea a un extraterrestre. Decidir sobre una condición apropiada para terminar el juego, y detener el juego cuando esta situación ocurre.

6.13. Resumen

En este capítulo, aprendiste cómo agregar una gran cantidad de elementos idénticos en un juego creando una flota de alienígenas. Se usó bucles anidados para crear un cuadrícula de elementos, y se hizo que un gran conjunto de elementos del juego se movieran llamando con el método `update()` de cada elemento. Aprendiste a controlar la dirección de objetos en la pantalla y para responder a situaciones específicas, como cuando la flota llega al borde de la pantalla. Detectaste y respondiste a colisiones cuando las balas alcanzaron a los extraterrestres y los extraterrestres impactaron en la nave. También aprendió a realizar un seguimiento de las estadísticas de un juego y a utilizar el indicador `juego_activo` para determinar cuándo termina el juego.

En el próximo y último capítulo de este proyecto, agregaremos un botón **Jugar** para que el jugador pueda elegir cuándo comenzar su primer juego y si desea volver a jugar cuando finalice el juego. Aceleraremos el juego cada vez que el jugador derribe a toda la flota y agregaremos un sistema de puntuación.

¡El resultado final será un juego totalmente jugable!

Capítulo 7

PUNTUACIÓN

En este capítulo, terminaremos de construir *Invasión Alienígena*. Agregaremos un botón para iniciar el juego a solicitud del jugador y reiniciar el juego una vez que termine. Le asignaremos un sistema de niveles cuando el jugador elimina una flota; el nivel del juego está asociado con la rapidez de los alienígenas para bajar; también implementar un sistema de puntuación. xxx

Al final del capítulo, habrás aprendido lo suficiente como para empezar a escribir juegos que aumenta en dificultad a medida que el jugador progresa y que cuentan con sistemas completos de puntuación.

7.1. Agregar el botón Play

En esta sección, agregaremos un botón **Play** (Jugar o reproducir) que aparece antes de que comience un juego y reaparece cuando termina el juego, para que el jugador pueda volver a jugar si lo desea.

Hasta este momento, el juego comienza tan pronto como ejecutas el archivo `invasion_alien.py`, ahora vamos a iniciar el juego en un estado inactivo, y solo después de que el jugador haga clic en el botón **Play** comenzará el juego.

Para hacer esto, debemos modificar el método `__init__()` de la clase `InvasionAlien`:

```
invasión      def __init__(self):
_alien.py      """Inicializa el juego y crea recursos para el juego."""
                pygame.init()
```

```
--recorte--
# Inicia Invasión Alienígena en un estado inactivo.
self.juego_activo = Falso
```

Ahora el juego debería comenzar en un estado inactivo, sin posibilidad de iniciarlo (hasta que hagamos clic en un botón de Play, pero por ahora no lo hemos creado).

7.1.0.1. Crear una clase de botón

Debido a que Pygame no tiene un método integrado para crear botones, debemos escribir una clase botón para crear un rectángulo relleno con una etiqueta. Puedes usar este código para hacer cualquier botón en un juego.

Creamos un módulo `boton.py` para desarrollar la de la clase `Boton`, y lo guardamos en la carpeta principal; es escribimos el siguiente código:

```
boton.py
import pygame.font
class Boton:
    """Una clase para contruir botón en el juego."""
    def __init__(self, ia_jugar, msg):
        """Inicializa atributos de Botón."""
        self.pantalla = ia_jugar.pantalla
        self.pantalla_rect = self.pantalla.get_rect()
        # Configuramos las dimensiones y propiedades del botón.
        self.ancho, self.alto = 200, 50
        self.color_boton = (0, 135, 0)
        self.color_texto = (255, 255, 255)
        self.font = pygame.font.SysFont(None, 48)
        # Construcción del objeto rectangulo del botón y cálculo del centro.
        self.rect = pygame.Rect(0, 0, self.ancho, self.alto)
        self.rect.center = self.pantalla_rect.center
        # El mensaje necesario para presionar una sola vez.
        self._prep_msg(mensaje)
```

Primero, importamos el módulo `pygame.font`, que permite a PyGame representar texto a la pantalla. El método `__init__()` toma los parámetros `self` y el objeto `ia_jugar` y el mensaje, que contiene el texto del botón. Configuramos las dimensiones ancho 200 y alto 50, el color verde oscuro del objeto `rect`, y configure `texto_color` para representar el texto en blanco.

A continuación, preparamos un atributo de fuente para representar el texto. El argumento `None`, le dice a PyGame que use la fuente predeterminada, y 48 especifica el tamaño del texto. Para centrar el botón en la pantalla, creamos un rectángulo para el botón y establezca su atributo central para que coincida con el de la pantalla. Pygame trabaja con texto representando la cadena que desea mostrar una imagen. Finalmente, llamamos a `_prep_msg()` para manejar esta representación.

Aquí está el código para `_prep_msg()`:

```
boton.py
def _prep_msg(self, mensaje):
    """Retorna mensaje dentro una imagen renderizada y centra
    el texto en el botón."""
    self.msg_image = self.font.render(mensaje, True,
                                      self.color_texto, self.color_boton)
    self.mensaje_image_rect = self.mensaje_image.get_rect()
    self.mensaje_image_rect.center = self.rect.center
```

El método `_prep_msg()` necesita un parámetro propio y el texto que se va a representar como imagen (mensaje). La llamada a `font.render()` convierte el texto almacenado en una una imagen, que luego almacenamos en `self.mensaje_image`. El `font.render()`

El método también toma un valor booleano para activar o desactivar el antialiasing (antialiasing suaviza los bordes del texto). Los argumentos restantes, el color de fuente y color de fondo especificados. Establecemos antialiasing en `True` y configuramos el fondo del texto al mismo color que el botón. Si no incluye un color de fondo, PyGame intentará representar la fuente con un color de fondo transparente. Centramos la imagen del texto en el botón creando un rectángulo a partir de la imagen y configurando su atributo central para que coincida con el del botón.

Finalmente, creamos un método `dibuja_boton()` al que podemos llamar para mostrar el botón en pantalla:

```
boton.py
def dibuja_boton(self):
    """Dibuja boton y dibuja."""
    self.pantalla.fill(self.color_boton, self.rect)
    self.pantalla.blit(self.mensaje_image, self.mensaje_image_rect)
```

Llamamos a `pantalla.fill()` para dibujar la parte rectangular del botón, y `pantalla.blit()` para dibujar la imagen del texto en la pantalla, pasándole una imagen y el objeto `rectángulo` asociado con la imagen. Esto completa la clase Botón.

7.1.0.2. Dibujar el botón en la pantalla

Usaremos la clase Botón para crear un botón Play en la clase `AlienInvasion`. Primero, vamos a actualice las declaraciones de importación:

```
invasión --recorte--
_alien.py from stats_juego import Estadisticas_Juego
         from boton import Boton
```

Como solo necesitamos un botón para reproducir (Play), para ello creamos el botón dentro del método `__init__()` de la clase `AlienInvasion` pero lo colocamos al final:


```

invasión
_alien.py
def __init__(self):
    --recorte--
    self.juego_activo = False
    # Hace el boton Play
    self.play_boton = Boton(self, "Play")

```

Este código crea una instancia de Botón con la etiqueta Play, pero no dibuja el botón en la pantalla. Para dibujarlo, llamaremos al método `dibujar_boton()` en `_actualizar_pantalla()`:

```

invasión
_alien.py
def _actualizar_pantalla(self):
    --recorte--
    self.aliens.draw(self.pantalla)
    # Dibuja el botón Play si el juego esta inactivo
    if not self.jugar_activo:
        self.play_boton.dibujar_boton()
    pygame.display.flip()

```

Para que el botón Play sea visible sobre todos los demás elementos de la pantalla, lo dibujamos después de que se hayan dibujado todos los demás elementos, pero antes de pasar a una nueva pantalla. Lo incluimos en un bloque `if`, por lo que el botón solo aparece cuando el juego está inactivo.

Ahora, al ejecutar Invasión Alienígena, debe aparecer el botón Play en el centro de la pantalla, como se muestra en la Figura 7.1.

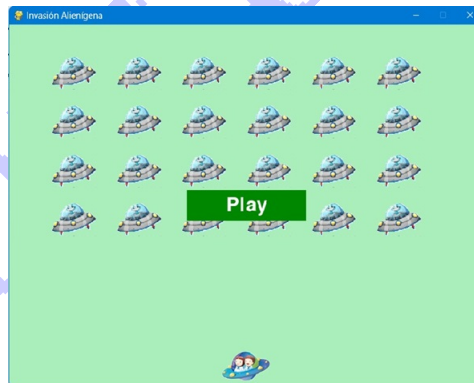


Figura 7.1: Aparecer el botón Play.

7.1.0.3. Comenzando el juego

Para comenzar un juego nuevo cuando el jugador hace clic en Jugar, se agregue el siguiente bloque `elif` al final del método `_chequear_eventos()` para monitorear los eventos del mouse sobre el botón:

```

invasión      def _chequear_eventos(self):
_alien.py      """Respuesta a teclas presionads y eventos del muuse"""
               --recorte--
               elif event.type == pygame.MOUSEBUTTONDOWN:
                   mouse_pos = pygame.mouse.get_pos()
                   self._chequear_play_boton(mouse_pos)

```

Pygame detecta un evento `MOUSEBUTTONDOWN` cuando el jugador hace clic en cualquier lugar en la pantalla, pero queremos restringir el juego para que responda sólo a los clics del mouse en el botón Play. Para lograr esto, usamos `pygame.mouse.get_pos()`, que devuelve una tupla que contiene las coordenadas x e y del cursor del mouse al hace clic. Para escribir estos códigos creamos un nuevo método `_chequear_play_button()`, y lo ubicamos después de `_chequear_eventos()`:

```

invasión      def _chequear_play_boton(self, mouse_pos):
_alien.py      """Comenzar un nuevo juego cuando hace clic en Play."""
               if self.play_boton.rect.collidepoint(mouse_pos):
                   self.juego_activo= True

```

Usamos el método `rect.collidepoint()` se usa para verificar si el punto del clic del mouse se superpone a la región definida por el rectángulo del botón Play. Entonces si configuramos `juego_activo` en `True`, haciendo clic en Play: ¡el juego comienza!

En este punto, deberías poder comenzar y jugar un juego completo.

Cuando el el juego termina, el valor de `juego_activo` debe volver al estado inactivo `False` y el botón Play debería reaparecer.

7.1.0.4. Reiniciar el juego

El código del botón Play que acabamos de escribir funciona la primera vez que el jugador hace clic Jugar. Pero no funciona después de que termina el primer juego, porque las condiciones que provocaron que el juego terminara no se han reiniciado.

Para reiniciar el juego cada vez que el jugador hace clic en Play, debemos resetear las estadísticas del juego, eliminar viejos alienígenas y balas, construir una nueva flota y centrar la nave, como se muestra aquí:

```

invasión      def _chequear_play_boton(self, mouse_pos):
_alien.py      """Comienza un nuevo juego cuando el jugador hace clic en Play."""
               if self.play_boton.rect.collidepoint(mouse_pos):
                   # Restablecer las estadísticas del juego.
                   self.estadisticas.resetear_stats()
                   self.juego_activo = True

                   # Obtiene las rid de los balas y alienigenas remaining.
                   self.balas.empty()

```

```

self.aliens.empty()
# Crea una nueva flota y centra la nave
self._crear_flota()
self.nave.nave_centro()

```

Restablecemos las estadísticas del juego, lo que le da al jugador tres naves nuevos. Luego configuramos `juego_activo` en `True` para que el juego comience tan pronto como termine de ejecutarse el código de esta función. Vaciamos los alienígenas y los grupos de balas, y luego creamos una nueva flota y centramos la nave.

Ahora el juego se reiniciará correctamente cada vez que hagas clic en Jugar, ¡lo que te permitirá jugar tantas veces como quieras!

7.1.0.5. Desactivar el botón Play

Un problema con nuestro botón Play es que la región del botón en la pantalla seguirá respondiendo a los clics incluso cuando el botón Play no esté visible. Si haces clic en el área del botón Play por accidente después de que comience un juego, ¡el juego se reiniciará!. Para solucionar este problema, configura el juego para que se inicie solo cuando `juego_activo` sea `False`:

```

invasión
_alien.py
def _chequear_play_boton(self, mouse_pos):
    """Comienza un nuevo juego cuando el jugador hace clic en Play."""
    boton_clicked = self.play_boton.rect.collidepoint(mouse_pos)
    if boton_clicked and not self.juego_activo:
        # Restablecer las estadísticas del juego.
        self.estadisticas.resetear_stats()
        self.juego_activo = True
    --recorte--

```

El indicador `boton_clicked` almacena un valor `True` o `Falso`, y el juego se reiniciará solo si se hace clic en Play y el juego no está activo actualmente. Para probar este comportamiento, inicie un nuevo juego y haga clic repetidamente donde debería estar el botón Play.

Si todo funciona como se esperaba, al hacer clic en el área del botón Play no debe tener ningún efecto en el juego.

7.1.0.6. Ocultar el cursor del mouse

Queremos que el cursor del mouse esté visible cuando el juego esté inactivo, pero una vez que comienza el juego, simplemente estorba. Para solucionar este problema, lo haremos invisible cuando el juego se active. Podemos hacer esto al final del bloque `if` en `_chequear_play_boton()`:

```

invasión
_alien.py

```

```
def _chequear_play_boton(self, mouse_pos):
    """Comienza un nuevo juego cuando el jugador hace clic en Play."""
    boton_clicked = self.play_boton.rect.collidepoint(mouse_pos)
    if boton_clicked and not self.juego_activo:
        --recorte--
        # Ocultar el cursor del mouse
        pygame.mouse.set_visible(False)
```

Pasar `False` a `set_visible()` le dice a PyGame que oculte el cursor cuando el mouse está sobre la ventana del juego.

Haremos que el cursor reaparezca una vez que termine el juego para que el jugador pueda Haz clic en Jugar nuevamente para comenzar un nuevo juego. Aquí está el código para hacer eso:

```
invasión
_alien.py    def _nave_golpeada(self):
            """Responde a cuando la nave ha sido golpeada por un alienigena."""
            if self.estadisticas.nave_izquierda > 0:
                --recorte--
            else:
                self.juego_activo = False
                pygame.mouse.set_visible(True)
```

Hacemos que el cursor vuelva a ser visible tan pronto como el juego se vuelve inactivo, lo que sucede en `_nave_golpeada()`. La atención a detalles como este hace que su juego tenga un aspecto más profesional y permite al jugador concentrarse en jugar, en lugar de que descubrir la interfaz de usuario.

7.2. INTÉNTALO TÚ MISMO

14-1. Presione P para reproducir: debido a que Alien Invasion usa la entrada del teclado para controlar el nave, sería útil iniciar el juego presionando una tecla. Agregue código que permita al El jugador presiona P para comenzar. Podría ser útil mover algún código de `_chequear_play_boton()` a un método `_start_game()` que se puede llamar desde `_chequear_play_boton()` y `_chequear_keydown_eventos()`.

14-2. Práctica de tiro: cree un rectángulo en el borde derecho de la pantalla que sube y baja a un ritmo constante. Luego, en el lado izquierdo de la pantalla, cree un nave que el jugador puede mover hacia arriba y hacia abajo mientras dispara balas al objetivo rectangular. Añade un botón Jugar que inicia el juego y cuando el jugador falla el objetivo tres veces, finaliza el juego y haz que reaparezca el botón Jugar.

Deje que el jugador reinicie el juego con este botón Jugar.

7.3. Subir de nivel

En nuestro juego actual, una vez que un jugador derriba a toda la flota alienígena, el jugador alcanza un nuevo nivel, pero la dificultad del juego no cambia, debemos agregarle algo de dificultad para darle un poco emoción y hacer que el juego sea más desafiante; si aumentamos la velocidad de los alienígenas cada vez que un jugador elimina una flota, el juego se complica un poco más en cada nivel.

7.3.0.1. Modificar la configuración de velocidad

Primero reorganizaremos la clase Configuración para agrupar la configuración del juego en los estáticos y los dinámicos. También nos aseguraremos de que cualquier configuración cambie durante el reinicio del juego cuando al comenzar un juego nuevo:

```
configurar.py    def __init__(self):
                  """Inicializamos la configuracion de estatica del juego."""
                  # Configuración de pantalla
                  self.fondo_image_filename = 'pygame.jpg'
                  self.pantalla_width = 800
                  self.pantalla_height = 600
                  self.color_fondo = (170,238,187)#(230, 230, 230)

                  # Configuraciones de Nave
                  self.limite_nave = 3

                  # configuraciones para las balas
                  self.velocidad_balas = 1.0
                  self.ancho_balas = 3
                  self.alto_balas = 10
                  self.color_balas = (0, 0, 0)
                  self.balas_permitidas = 10

                  # Configuracion Alienigena
                  self.velocidad_alien = 1.0
                  self.flota_cae_velocidad = 5

                  # Dirección de flota: 1 representa derecha;
                  # -1 representa izquierda.
                  self.direccion_flota = 10

                  # Que tan rapido la sube velocidad del juego
                  self.velocidad_sube_escal = 1.1

                  self.inicialize_configuracion_dinamica()
```

Las configuraciones que permanecen constantes la ubicamos en el método `__init__()`. Agregamos una configuración `self.velocidad_sube_scale = 1.1` para controlar qué tan rápido se acelera el juego: un valor de 2 duplicará

la velocidad del juego cada vez que el jugador alcance un nuevo nivel; un valor de 1 mantendrá la velocidad constante. Un valor como 1,1 debería aumentar la velocidad lo suficiente como para que el juego sea desafiante pero no imposible.

Finalmente, para inicializar los valores de los atributos que cambian a lo largo del juego llamamos `inicialize_configuracion_dinamica()`:

```
configurar.py      # configuracion dinámica
def inicialize_configuracion_dinamica(self):
    """Inicializa la configuración de dinámica del juego."""
    self.velocidad_nave = 6
    self.velocidad_balas = 1.0
    self.velocidad_alien = 1.0
    # Dirección de flota: 1 representa derecha;
    # -1 representa izquierda.
    self.direccion_flota = 1
```

Este método establece los valores iniciales para las velocidades de la nave, la bala y el alienígena. Aumentaremos estas velocidades a medida que el jugador avance en el juego y las restableceremos cada vez que el jugador comience un nuevo juego. Incluimos `direccion_flota` en este método para que los alienígenas siempre se muevan justo al comienzo de un nuevo juego.

No necesitamos aumentar el valor de `self.flota_cae_velocidad = 5`, porque cuando los extraterrestres se muevan más rápido por la pantalla, también vendrán bajar la pantalla más rápido.

Para aumentar la velocidad de la nave, de las balas y de los alienígenas cada vez que el jugador alcanza un nuevo nivel, escribiremos un nuevo método llamado `incrementa_velocidad()`:

```
configurar.py      def incrementar_velocidad(self):
    """Configuración para incremento de velocidad."""
    self.velocidad_nave *= self.velocidad_sube_escalas
    self.velocidad_balas *= self.velocidad_sube_escalas
    self.velocidad_alien *= self.velocidad_sube_escalas
```

Para aumentar la velocidad de estos elementos, multiplicamos cada uno de ellos por el valor de `self.velocidad_sube_escalas`. Aumentamos el tiempo del juego llamando a `incrementa_velocidad()` en `_chequear_colision_balas_alien()` cuando el último alienígena de una flota ha sido derribado:

```
invasión
_alien.py          def _chequear_colisiones_balas_alien(self):
    --recorte--
    if not self.aliens:
        # Se destruye existencia de balas y se crea una nueva flota.
        self.balas.empty()
        self._crear_flota()
        self.configuracion.incrementar_velocidad()
```

cambiando los valores de velocidad de la nave, del alienígena, y de las balas, es suficiente para subir el nivel del juego.

7.3.0.2. Resetear la velocidad

Ahora necesitamos retornar cualquier cambio en la configuración de sus valores iniciales cada vez que comience un nuevo juego; por otro lado en cada tiempo de juego; por otro lado, cada juego nuevo debe comenzar con un incremento de velocidad configurado previamente:

```
invasión
_alien.py    def _chequear_play_boton(self, mouse_pos):
              """Comienza un nuevo juego cuando el jugador hace clic en Play."""
              boton_clicked = self.play_boton.rect.collidepoint(mouse_pos)
              if boton_clicked and not self.juego_activo:
                  # resetea la configuracion del juego
                  self.configuracion.inicialize_configuracion_dinamica()
              --recorte--
```

Y ahora jugar Invasión Alienígena debería ser más divertido y desafiante. Cada vez que inicia un nuevo juego, debe acelerarse y volverse rápido y por supuesto más difícil. Si el juego se vuelve demasiado difícil, muy rápido, disminuya los valores de `self.velocidad_sube_escalas`, o si el juego no es lo suficientemente desafiante, aumente ligeramente el valor.

Encuentra un punto óptimo aumentando la dificultad en una cantidad de tiempo razonable. Los primeros dos juegos deben ser fáciles, los siguientes deberían ser desafiantes pero factibles, y luego increíblemente difícil.

7.4. INTÉNTALO TÚ MISMO

14-3. Práctica de tiro desafiante: comience con su trabajo del ejercicio 14-2 (página 283). Haz que el objetivo se mueva más rápido a medida que avanza el juego y reinicia el objetivo a la velocidad original cuando el jugador hace clic en Reproducir.

14-4. Niveles de dificultad: crea un conjunto de botones para Alien Invasion que permitan al jugador seleccionar un nivel de dificultad inicial apropiado para el juego. Cada botón debe asignar los valores apropiados para los atributos en Configuración necesarios para crear diferentes niveles de dificultad.

7.5. Puntuación

Implementemos un sistema de puntuación para realizar un seguimiento de la puntuación del juego en tiempo real y mostrar la puntuación más alta, el nivel y la cantidad de naves restantes. La puntuación es una estadística del juego, por lo que agregaremos un atributo de puntuación en el módulo `Stats_Juego`:

```
stats
_juego.py class Estadisticas_Juego():
    """Track estadísticas para el juego Invasión Alienígena."""
    def __init__(self, ia_jugar):
        --recorte:

    def resetear_stats(self):
        """Inicializa los estadísticas que cambian durante el juego."""
        self.nave_permitidas = self.configuracion.limite_naves
        self.puntuacion = 0
```

Para restablecer la puntuación cada vez que comienza un nuevo juego, inicializamos la puntuación en `resetear_stats()` en lugar de `__init__()`. Visualización de la puntuación Para mostrar la puntuación en la pantalla, primero creamos una nueva clase, `Marcador`. Por ahora, esta clase solo mostrará la puntuación actual. Con el tiempo, también lo usaremos para informar la puntuación más alta, el nivel y la cantidad de naves restantes. Aquí está la primera parte de la clase; guárdelo como `marcador.py`:

```
puntuacion.py import pygame.font
class Puntuacion:
    """Una clase para reportar el informacion de la puntuacion."""
    def __init__(self, ia_jugar):
        """Inicializa atributo de puntaje."""
        self.pantalla = ia_jugar.pantalla
        self.pantalla_rectang = self.pantalla.get_rect()
        self.configuracion = ia_jugar.configuracion
        self.estadisticas = ia_jugar.stats
        # Configuración de fuente para informar puntaje scoring
        self.color_texto = (30, 30, 30)
        self.fuente = pygame.font.SysFont(None, 48)
        # Prepara la imagen inicial de la puntuacion
        preparacion_puntuacion()
```

Debido a que `puntuacion.py` escribe texto en la pantalla, comenzamos importando el módulo `pygame.font`.

A continuación, le damos a `__init__()` el parámetro `ia_jugar` para que pueda acceder a los objetos de configuración, pantalla y estadísticas, que necesitará para informar los valores que estamos rastreando. Luego configuramos un color de texto y creamos una instancia de un objeto fuente.

Para convertir el texto que se mostrará en una imagen, llamamos a `preparacion_puntuacion()`, que definimos aquí:


```
puntuacion.py
def preparacion_puntuacion(self):
    """retorna la puntuacion dentro de la imagen renderizada."""
    puntuacion_str = str(self.estadisticos.puntuacion)
    self.puntuacion_imagen = self.fuente.render(puntuacion_str, True,
                                                self.color_texto,
                                                self.configuracion.color_fondo)
    # Muestra los puntos en la parte superior izquierda de la pantalla.
    self.rectang_puntuacion = self.puntuacion_imagen.get_rect()
    self.rectang_puntuacion.right = self.pantalla_rectang.right - 20
    self.rectang_puntuacion.top = 20
```

En el método `preparacion_puntuacion()`, convertimos el valor numérico de `stats.puntaje` en una cadena y luego pasa esta cadena a la función `render()`, para que crear la imagen. Para mostrar la puntuación clara en pantalla, pasamos el color de fondo de pantalla y el color del texto a `renderizar()`.

Colocaremos la información en la esquina superior derecha de la pantalla y que se expanda hacia la izquierda a medida que aumenta la puntuación y el crece ancho del número. Para asegurarse de que la puntuación siempre se alinee con el lado derecho de la pantalla, creamos un rectángulo llamado `rectang_puntuacion` y configuramos su borde derecho en 20 píxeles desde el borde derecho de la pantalla 4. Luego colocamos el borde superior a 20 píxeles hacia abajo desde la parte superior de la pantalla 5.

Luego creamos un método `mostrar_puntuacion()` para mostrar la puntuación renderizada. imagen:

```
puntuacion.py
def mostrar_puntuacion(self):
    """Dibujar puntuacion en pantalla."""
    self.pantalla.blit(self.puntuacion_imagen, self.rectang_puntuacion)
```

Este método dibuja la imagen de la puntuación en la pantalla en la ubicación `mostrar_puntuacion()` especifica.

7.5.0.1. El marcador en pantalla

Para mostrar la puntuación, crearemos una instancia de `Marcador` en `AlienInvasion`. Primero, actualicemos las declaraciones de importación:

```
invasión
    _alien.py --recorte--
from stats_juegos import Estadisticas_Juego
from puntuacion import Puntuacion
--snip--
```

Ahora, creamos una instancia de `Puntuacion` en `__init__()` en la clase `AlienInvasion`:

```
invasión
    _alien.py
```

```

def __init__(self):
    --recorte--
    pygame.display.set_caption("Invasión Alienígena")
    # Crea una instancia para almacenar estadísticos del juego
    # y crea un puntuacion en pantalla
    self.estadisticas = Estadisticas_Juego(self)
    self.Pp = Puntuacion(self)
    --recorte--

```

Y el dibujo de puntuación sobre pantalla en `actualizar_pantalla`:

```

invasión
_alien.py
def actualizar_pantalla(self):
    --recorte--
    self.aliens.draw(self.pantalla)
    # Dibuja la puntuacion en pantalla Pp
    self.Pp.mostrar_puntuacion()

```

Llamamos a `mostrar_puntuacion()` justo antes de dibujar el botón Play. Cuando ejecute Invasión Alienígena debe aparecer un 0 en la parte superior derecha de la pantalla (en este punto, solo queremos asegurarnos de que la puntuación aparezca en el lugar correcto antes de continuar desarrollar el sistema de puntuación).

La Figura 7.2 muestra la puntuación tal como aparece antes de que comience el juego. A continuación, asignaremos valores de puntos a cada extraterrestre!

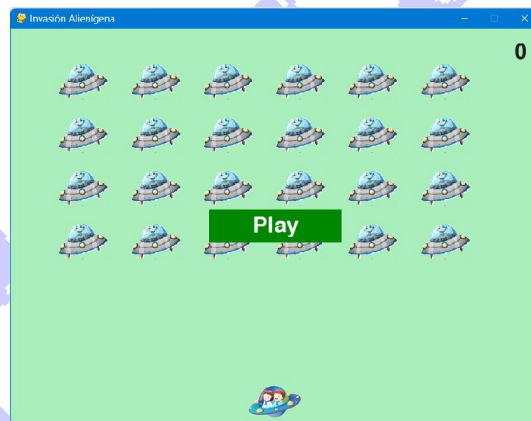


Figura 7.2: La puntuación en la esquina superior derecha.

7.5.0.2. Actualización de la puntuación los alienígenas

Para escribir una puntuación en vivo en pantalla, actualizamos el valor de `stats.score` cada vez que se golpea a un extraterrestre y luego se llama

a `preparacion_puntuacion()` para actualizar la imagen de la puntuación. Pero Primero, determinemos cuántos puntos obtiene un jugador cada vez que dispara. derribar a un extraterrestre:

```
configurar.py    def initialize_configuracion_dinamica(self):
                  """Inicializa la configuración de dinámica del juego."""
                  --recorte--
                  # Configuración de puntuación
                  self.puntos_alien = 50
```

Aumentaremos el valor de puntuación cada alienígena a medida que avance el juego. Para asegurar que éste el valor de puntuación se restablezca cada vez que comienza un nuevo juego, configuramos el valor en el método `initialize_configuracion_dinamica()`.

Actualicemos la puntuación en `_chequear_colisiones_balas_alien()` cada vez que alienígena es derribado:

```
invasión
_alien.py    def _chequear_colisiones_balas_alien(self):
              """Responder a las colisiones balas-alienígena."""
              # Remover cualquier bala y alienígena que han colisionado.
              colisiones = pygame.sprite.groupcollide(
                          self.balas, self.aliens, True, True)

              if colisiones:
                  self.estadisticas.puntuacion += self.configuracion.puntos_alien
                  self.Pp.preparacion_puntuacion()
              --recorte--
```

Cuando una bala alcanza a un extraterrestre, Pygame devuelve un diccionario de colisiones. Comprobamos si el diccionario existe, y si existe, el valor del alienígena es añadido a la puntuación. Luego llamamos al método `preparacion_puntuacion()` para crear una nueva imagen de la puntuación actualizada.

¡Ahora, cuando juegues Invasión Alienígena, debe acumular puntos por cada alienígena que derribe.

7.5.0.3. Restablecer la puntuación

En este momento, solo estamos preparando una nueva puntuación después de que un extraterrestre haya sido derribado que funciona durante la mayor parte del juego. Pero cuando comenzamos un nuevo juego, todavía se ve puntuación del juego anterior hasta que derribes al primer alienígena.

Podemos solucionar este problema preparando la puntuación al iniciar un nuevo juego:

```
invasión
_alien.py    def _chequear_play_boton(self, mouse_pos):
              """Comienza un nuevo juego cuando el jugador hace clic en Play."""
```

```

    boton_clicked = self.play_boton.rect.collidepoint(mouse_pos)
    --recorte--
    # Restablecer las estadísticas del juego.
    self.estadisticas.resetear_stats()
    self.Pp.preparacion_puntuacion()
    --recorte--

```

Llamamos el método `preparacion_puntuacion()` después de resetear las estadísticas del juego al iniciar un juego nuevo. Esto prepara el marcador con una puntuación de 0. Asegurándose de anotar todos los aciertos tal como está escrito actualmente, nuestro código podría no puntuar para algunos extraterrestres. Por ejemplo, si dos balas chocan con alienígenas durante el mismo paso por el bucle o si hacemos una bala extra ancha para impactar a varios alienígenas, el jugador solo recibirá puntos por impactar a uno de los alienígenas. Para solucionar este problema, reorientamos la forma en que se detectan las colisiones entre balas y extraterrestres.

En `_chequear_colisiones_balas_alien(self)`, cualquier bala que colisione con un extraterrestre se convierte en una clave en el diccionario de colisiones. El valor asociado con cada bala es una lista de extraterrestres con los que chocó. Revisamos los valores en el diccionario de colisiones para asegurarnos de otorgar puntos por cada impacto alienígena:

```

invasión      def _check_bullet_alien_collisions(self):
_alien.py    --recorte--
            if colisiones:
                for aliens in colisiones.values():
                    self.estadisticas.puntuacion +=
                        self.configuracion.puntos_alien * len(aliens)
                    self.Pp.preparacion_puntuacion()
            --recorte--

```

Si se ha definido el diccionario de colisiones, recuerda que cada valor del diccionario es una lista de alienígenas alcanzados por una sola bala. Multipliquemos el valor de cada alienígenas por el número de alienígenas en cada lista y sumamos esta cantidad a la puntuación actual. Para probar esto, cambia el ancho de una bala a 300 píxeles y verifica que recibes puntos por cada alienígena que derribes con tus balas extra anchas; luego regresa el ancho de la bala a su valor normal.

Aumento del valor de los puntos debido a que el juego se vuelve más difícil cada vez que un jugador alcanza un nuevo nivel, los alienígenas en niveles posteriores deberían valer más puntos. Para implementar esta funcionalidad, agregaremos código para aumentar el valor de los puntos cuando aumente la velocidad del juego: `settings.py` `class Configuración:`

```

configurar.py class Configuración:

```

```

"""Una clase para almacenar todas las configuraciones de
Invasión Alienígena."""
def __init__(self):
    """Inicializamos la configuracion de estatica del juego."""
    # Configuración de pantalla

    --recorte--
    # Incremento de la velocidad del juego
    self.velocidad_sube_escalas = 1.1

    # Incremento de los valores puntuacion de los alienigenas
    self.puntuacion_escala = 1.5

    self.initialize_dynamic_settings()

    def initialize_dynamic_settings(self): --snip--
        --recorte--

def incrementar_velocidad(self):
    """Configuración para incremento de velocidad, y la puntuación por
    alienígena."""
    self.velocidad_nave *= self.velocidad_sube_escalas
    self.velocidad_balas *= self.velocidad_sube_escalas
    self.velocidad_alien *= self.velocidad_sube_escalas
    self.puntos_alien = int(self.puntos_alien * 50 *
                             self.velocidad_sube_escalas)

```

Definimos una tasa `puntuación_escala`, para aumentar la puntuación. Un pequeño aumento en la velocidad (1.1) hace que el juego sea un poco más rápido. Sin embargo, para ver una diferencia notable usamos el valor de la tasa en 1,5.

Ahora cuando aumentamos la velocidad del juego, también aumentamos el valor en puntos de cada nave derribada. Usamos la función `int()` para aumentar el valor del punto en números enteros. Para ver el valor de cada alienígena, agregue una llamada `print()` al `incremento_speed()` método en Configuración:

```

configurar.py
def increase_speed(self):
    --recorte--
    self.puntos_alien = int(self.puntos_alien * 50 *
                           self.velocidad_sube_escalas)
    print(self.puntos_alien)

```

El nuevo valor de puntos aparece en la terminal cada vez que alcanzar un nuevo nivel.

Después de verificar el valor de puntos está aumentando, debe eliminar la llamada `print()`, que puede afectar el rendimiento del juego y distraer al jugador.

7.5.0.4. Redondeando la puntuación

La mayoría de los juegos del estilo de disparos reportan puntuaciones como múltiplos de 10, así que sigamos con nuestras puntuaciones. Además, formateemos la información para incluir comas separadores en grandes cantidades. Haremos este cambio en el Marcador:

```
puntuacion.py    def preparacion_puntuacion(self):
                  """retorna la puntuacion dentro de la imagen renderizada."""
                  redondeo_puntuacion = round(self.estadisticas.puntuacion, -1)
                  puntuacion_str = f"{redondeo_puntuacion:}"
                  self.puntuacion_imagen = self.fuente.render(puntuacion_str, True,
                                                                self.color_texto,
                                                                self.configuracion.color_fondo)

                  --recorte--
```

La función `round()` normalmente redondea un flotante a un número determinado de decimales dados como segundo argumento. Sin embargo, cuando pasas un negativo número como segundo argumento, función `round()` redondeará el valor al 10, 100, 1000, etc. más cercanos. Este código le dice a Python que redondee el valor de `estadisticas.puntuacion` al 10 más cercano y asígnelo a `redondeo_puntuacion`.



Figura 7.3: Puntuación redondeada con separadores de coma.

Luego usamos el formato de cadena **f-string** para la puntuación. Un formato especificador es una secuencia especial de caracteres que modifica la forma en que una variable se presenta el valor. En este caso la secuencia: le dice a Python que inserte comas en los lugares apropiados del valor numérico proporcionado. Esto da como resultado cadenas como 1,000,000 en lugar de 1000000.

Ahora, cuando ejecutes el juego, debes ver una puntuación redondeada y perfectamente formateada incluso cuando acumule muchos puntos, como se muestra en la Figura 7.3.

7.5.0.5. Puntuaciones altas

Cada jugador quiere superar la puntuación más alta de un juego, así que hagamos un seguimiento e informemos las puntuaciones altas para darles a los jugadores algo por lo que trabajar.

Almacenaremos puntuaciones altas en `stats_juego.py`:

```
stats
_juego.py    def __init__(self, ia_jugar):
              --recorte--
              # Puntuacione altas nunca debe ser eliminadas.
              self.puntuacion_alta = 0
```

Debido a que la puntuación más alta nunca debe ser eliminada, inicializamos `alta_puntuacion` en `__init__()` en lugar de `resetear_stats()`.

A continuación, modificaremos el marcador para mostrar la puntuación más alta.

```
puntuacion.py    def __init__(self, ia_jugar):
                  """Inicializa atributo de puntaje."""
                  --recorte--

                  # Prepara la imagen de puntuacion inicial.
                  self.preparacion_puntuacion()
                  self.preparacion_alta_puntuaciones()
```

La puntuación más alta se mostrará por separado de la puntuación, por lo que necesitamos un nuevo método, `preparacion_alta_puntuacion`, para preparar la imagen de puntuación más alta.

```
puntuacion.py    def preparacion_alta_puntuaciones(self):
                  """Retorna la puntuacion alta dentro de magen renderizada."""
                  puntuacion_alta = round(self.estadisticas.puntuacion_alta, -1)
                  puntuacion_alta_str = f"{puntuacion_alta:,"}
                  self.puntuacion_alta_imagen = self.font.render(puntuacion_alta_str,
                                                                    True,
                                                                    self.color_texto,
                                                                    self.configuracion.color_fondo)

                  # puntuacion_alta al centro y en la parte superior de la pantalla.
                  self.puntuacion_alta_rectang = self.puntuacion_alta_imagen.get_rect()
                  self.puntuacion_alta_rectang.centerx = self.pantalla_rectang.centerx
                  self.puntuacion_alta_rectang.top = pantalla_rectang.top
```

Redondeamos la puntuación más alta a la decena más cercana y la formateamos con comas. Luego generamos una imagen a partir de la puntuación

más alta, centramos la puntuación más alta en el rectángulo horizontalmente y establece su atributo superior para que coincida con el superior de la imagen de la puntuación.

El método `mostrar_puntuacion()` ahora dibuja la puntuación actual en la parte superior derecha y la puntuación más alta en la parte superior central de la pantalla:

```
puntuacion.py    def mostrar_puntuacion(self): # dibuja o muestra
                  """Dibuja puntuacion en pantalla."""
                  self.pantalla.blit(self.puntuacion_imagen, self.rectang_puntuacion)
                  self.screen.blit(self.altapuntuacion_imagen,
                                   self.rectang_puntuacion_alta)
```

Para comprobar puntuaciones altas, escribiremos en `puntuacion.py` un nuevo método, `chequear_altas_puntuaciones()`:

```
puntuacion.py    def chequear_altas_puntuaciones(self):
                  """Compruebe para ver si hay una puntuaciones altas nuevas"""
                  if self.estadisticas.puntuacion > self.estadisticas.puntuacion_alta:
                      self.estadisticas.puntuacion_alta = self.estadisticas.puntuacion
                      self.preparacion_alta_puntuaciones()
```

El método `chequear_altas_puntuaciones()` compara la puntuación actual con la puntuación más alta. Si la puntuación actual es mayor, actualizamos el valor de `puntuacion_alta` y llame `preparacion_alta_puntuacion()` para actualizar la imagen de la puntuación más alta.

Necesitamos llamar a `chequear_altas_puntuaciones()` cada vez que un extraterrestre sea derribado después de la actualización. la puntuación en `_chequear_colisiones_balas_alien()`:

```
invasión        def _chequear_colisiones_balas_alien(self):
_alien.py        --recorte--
                  if colisiones:
                      for aliens in colisiones.values():
                          self.estadisticas.puntuacion += self.\
                              configuracion.puntos_alien * len(aliens)
                          self.Pp.preparacion_puntuacion()
                          self.Pp.chequear_altas_puntuaciones()
                          print(self.Pp.chequear_altas_puntuaciones() )
                  --recorte--
```

Llamamos a `chequear_altas_puntuaciones` cuando el diccionario de colisiones está presente, y lo hacemos después de actualizar la puntuación de todos los alienígenas que han sido alcanzados.

La primera vez que juegues *Invasión Alienígena*, tu puntuación será la puntuación más alta, por lo que se mostrará como la puntuación actual como la puntuación más alta. Pero cuando comienzas un segundo juego, tu puntuación más alta debe aparecer en el medio y tu puntuación actual debería aparecer a la derecha, como se muestra en la Figura 7.4.



Figura 7.4: En la parte superior central de la pantalla se muestra la puntuación más alta.

7.5.0.6. Visualización del nivel

Para mostrar el nivel del jugador, necesitamos definir un atributo en `Estadisticas_Juego()`, que represente el nivel actual. Se inicializa en `resetear_stats()` para restablecer el nivel al inicio de cada juego nuevo:

```
stats      def resetear_stats(self):
    _juego.py      """Inicializa estadísticas que cambian durante el juego"""
                    self.nave_permitidas = self.configuracion.limite_naves
                    self.puntuacion = 0
                    self.nivel = 1
```

Para mostrar el nivel actual, creamos un nuevo método `preparando_nivel()`, en `puntuacion.py`:

```
puntuacion.py      def preparando_nivel(self):
                    """Retorna el nivel en una imagen renderizada."""
                    nivel_str = str(self.estadisticas.nivel)
                    self.nivel_imagen = self.fuente.render(nivel_str,
                                                            True, self.color_texto,
                                                            self.configuracion.color_fondo)

                    # Posicion del nivel bajo la puntuacion
                    self.nivel_rectang = self.nivel_imagen.get_rect()
                    self.nivel_rectang.right = self.rectang_puntuacion.right
                    self.nivel_rectang.top = self.rectang_puntuacion.bottom + 10
```

Y lo llamamos en `__init__(self, ia_jugar)`

```
puntuacion.py      def __init__(self, ia_jugar):
                    --recorte--
                    self.preparando_puntuacion_alta()
                    self.preparando_nivel()
```

El método `preparando_nivel()` crea una imagen a partir del valor almacenado en `estadisticas.nivel` y establece el atributo `rectangulo` de la imagen para que coincida con el atributo `rect` de la puntuación. Luego establece el atributo superior a 10 píxeles debajo de la parte inferior de la imagen de la puntuación para dejar espacio entre la puntuación y el nivel. Y actualizamos el método `mostrar_puntuacion()`:

```
puntuacion.py    def mostrar_puntuacion(self):
                  """Dibuja puntuacion y nivel en la pantalla."""
                  self.pantalla.blit(self.puntuacion_imagen, self.rectang_puntuacion)
                  self.pantalla.blit(self.puntuacion_alta_imagen,
                                      self.puntuacion_alta_rectang )
                  self.pantalla.blit(self.nivel_imagen, self.nivel_rectang)
```

Esta nueva línea de código dibuja la imagen del nivel en la pantalla.

Incrementamos `estadisticas.nivel` y actualizamos la imagen del nivel en `_chequear_colisiones_balas_alien()`:

```
invasión        def _chequear_colisiones_balas_alien(self):
_alien.py      --recorte--
                  if not self.aliens:
                      # Destruye la existencia de las balas y crea una nueva flota.
                      self.balas.empty()
                      self._crear_flota()
                      self.configuracion.incrementar_velocidad()
                      # Incrementar nivel
                      self.estadisticas.nivel += 1
                      self.Pp.preparando_nivel()
```

Si una flota es destruida, incrementamos el valor de `stats.level` y llamamos `prep_level()` para asegurarse de que el nuevo nivel se muestre correctamente. Para garantizar que la imagen del nivel se actualice correctamente al comienzo de un nuevo juego, También llamamos a `prep_level()` cuando el jugador hace clic en el botón Reproducir:

```
invasión        def _chequear_play_boton(self, mouse_pos):
_alien.py      --recorte--
                  if boton_clicked and not self.juego_activo:
                      --recorte--
                      self.Pp.preparacion_puntuacion()
                      self.Pp.preparando_nivel()
                      --recorte--
```

Llamamos a `self.Pp.preparando_nivel()` justo después de llamar a `self.Pp.preparacion_puntuacion()`.

Ahora puedes ver cuántos niveles has completado; el nivel actual aparece justo debajo de la puntuación actual, como se muestra en Figura 7.5.

NOTA

En algunos juegos clásicos, las puntuaciones tienen etiquetas, como Puntuación, Puntuación alta y Nivel. Hemos omitido estas etiquetas porque el



Figura 7.5: El nivel actual aparece justo debajo de la puntuación actual.

significado de cada número queda claro una vez que has jugado. Para incluir estas etiquetas, agréguelas a las cadenas de puntuación justo antes de las llamadas a `font.render()` en el marcador.

7.5.1. Visualización del número de naves

Finalmente, mostremos el número de naves que le quedan al jugador, pero esta vez usemos un gráfico. Para hacerlo, dibujaremos naves en la esquina superior izquierda de la pantalla para representar cuántos naves quedan, tal como lo hacen muchos juegos de arcade clásicos. Primero, debemos hacer que la nave herede de `Sprite` para poder crear un grupo:

```
nave.py
import pygame
from pygame.sprite import Sprite

class Nave(Sprite):
    """Una clase que maneja el comportamiento de la nave"""
    def __init__(self, ia_jugar):
        """Inicia la nave y configura la posición inicial."""
        super().__init__()
        --recorte--
```

Aquí importamos `Sprite`, nos aseguramos de que la nave herede de `Sprite` y llamamos a `super()` al comienzo de `__init__()`.

Y ahora debemos modificar la puntuación para crear un grupo de naves que podamos mostrar:

```
puntuacion.py
```

```
import pygame.font
from pygame.sprite import Group
from nave import Nave
```

Debido a que estamos creando un grupo de naves, importamos el grupo y la nave.

```
puntuacion.py def __init__(self, ia_jugar):
    """Inicializamos los atributos de la puntuacion guardada."""
    self.ia_jugar = ia_jugar
    self.pantalla = ia_jugar.pantalla
    --recorte--
    self.preparando_nivel()
    self.preparando_nave()
```

Asignamos la instancia del juego a un atributo, porque la necesitaremos para crear algunas naves. Llamamos a `preparando_nave()` después de la llamada a `preparando_nivel()`.

Creamos `preparando_nave()`

```
puntuacion.py def preparando_nave(self):
    """Muestra numero de naves permitidas."""
    self.naves = Group()
    for numero_nave in range(self.estadisticas.naves_permitidas):
        nave = Nave(self.ia_jugar)
        nave.rect.x = 10 + numero_naves * nave.rect.width
        nave.rect.y = 10
        self.nave.add(nave)
```

El método `preparando_nave()` crea un grupo vacío, `self.naves`, para almacenar las naves (instancia de `Nave`). Para llenar este grupo, se ejecuta un bucle una vez por cada nave que el jugador ha dejado. Dentro del bucle, creamos una nueva nave y configuramos el valor de la coordenada x de las naves para que aparezcan uno al lado del otro con un espacio de 10 píxeles de margen en el lado izquierdo del grupo de naves³. Establecemos la coordenada y con un valor de 10 píxeles hacia abajo desde la parte superior de la pantalla para que las naves aparezcan en la esquina superior izquierda de la pantalla. Luego agregamos cada nave nueva al grupo.

Ahora necesitamos dibujar las naves en la pantalla:

```
puntuacion.py def mostrar_puntuacion(self):
    """Dibuja puntuaciones, niveles y naves en la pantalla."""
    self.pantalla.blit(self.puntuacion_imagen, self.rectang_puntuacion)
    self.pantalla.blit(self.puntuacion_alta_imagen, self.puntuacion_alta_rectang )
    self.pantalla.blit(self.nivel_imagen, self.nivel_rectang)
    self.nave.draw(self.pantalla)
```

Para mostrar las naves en la pantalla, llamamos a `draw()` en el grupo, y Pygame dibuja cada nave. Para mostrarle al jugador con cuántas naves tiene

para comenzar, llamamos a `preparando_nave()` cuando comienza un nuevo juego. Hacemos esto en `_chequear_play_boton()` en `AlienInvasion`:

```
invasión
_alien.py
def _chequear_play_boton(self, mouse_pos):
    --recorte--
    if boton_clicked and not self.juego_activo:
        --recorte--
        self.Pp.preparando_nivel()
        self.Pp.preparando_nave()
    --recorte--
```

También llamamos a `preparando_nave()` cuando una nave es impactada, para actualizar la visualización de las imágenes de las naves el jugador pierde una nave:

```
invasión
_alien.py
def _nave_golpeada(self):
    """Responde a cuando la nave ha sido golpeada por un alienigena."""
    if self.estadisticas.nave_permitidas > 0:
        # Resta 1 a nave_permitidas y actualiza puntuación
        self.estadisticas.nave_permitidas -= 1
        self.Pp.preparando_nave()
    --recorte--
```

Llamamos al método `preparando_nave()` después de disminuir el valor de `nave_permitidas`, de modo que se muestre el número correcto de naves restantes cada vez que se destruye una nave.

La Figura 7.6 muestra el sistema de puntuación completo, con las naves restantes mostrados en la parte superior izquierda de la pantalla.



Figura 7.6: Las naves disponibles aparecen en la parte superior izquierda de la pantalla.

7.6. INTÉNTALO TÚ MISMO

14-5. Puntuación más alta de todos los tiempos: la puntuación más alta se restablece cada vez que un jugador cierra y reinicia Alien Invasion. Solucione este problema escribiendo la puntuación más alta en un archivo antes llamando a `sys.exit()` y leyendo la puntuación más alta al inicializar su valor en Estadísticas del juego.

14-6. Refactorización: busque métodos que realicen más de una tarea y Refactorícelos para organizar su código y hacerlo eficiente. Por ejemplo, mover parte del código en `_check_bullet_alien_collisions()`, que inicia una nueva nivel cuando la flota de alienígenas ha sido destruida, a una función llamada `inicio_nuevo_nivel()`. Además, mueva las cuatro llamadas a métodos separados en el método `__init__()` en Scoreboard a un método llamado `prep_images()` para acortar `__init__()`. la preparación El método `_images()` también podría ayudar a simplificar `_check_play_button()` o `start_game()` si ya has refactorizado `_check_play_button()`. NOTA Antes de intentar refactorizar el proyecto, consulte el Apéndice D para obtener más información. cómo restaurar el proyecto a un estado de funcionamiento si introduce errores mientras se refactoriza.

14-7. Ampliando el juego: piensa en una manera de expandir Alien Invasion. Para Por ejemplo, puedes programar a los extraterrestres para que disparen balas a tu nave. Tú También puedes agregar escudos para que tu nave se esconda detrás, los cuales pueden ser destruidos por balas de ambos lados. O puedes usar algo como el módulo `pygame.mixer` para agregar efectos de sonido, como explosiones y sonidos de disparos.

14-8. Sideways Shooter, versión final: continúa desarrollando Sideways Shooter, utilizando todo lo que hemos hecho en este proyecto. Agrega un botón Jugar, crea el juego. acelerar en los puntos apropiados y desarrollar un sistema de puntuación. Asegúrate de refactorizar su código mientras trabaja y busca oportunidades

7.7. Resumen

En este capítulo, aprendió cómo implementar un botón Reproducir para iniciar una nueva juego. También aprendió a detectar eventos del mouse y ocultar el cursor en juegos activos. Puedes usar lo que has aprendido para crear otros botones, como un Botón de ayuda para mostrar instrucciones

sobre cómo jugar a tus juegos. tu tambien aprendiste cómo modificar la velocidad de un juego a medida que avanza, implementar un progresivo sistema de puntuación y mostrar información en forma textual y no textual.

Borrador

Capítulo 8

Empaquetando el juego, y creando un archivo EXE

Si ha llegado hasta aquí, seguro logro escribir un juego con PyGame, y ahora quieres compartir y distribuir su obra maestra; la forma más sencilla de distribuir su juego es agrupar el código Python y con sus datos en un archivo comprimido, como **ZIP**, **TAR** o **GZIP**, y cargarlo en su sitio web o envíelo por correo electrónico. Sin embargo, el problema con este enfoque es que Python y cualquier módulo externo que utilice deben ser instalado antes de correr el juego, lo que genera dificultades para el público común que no maneja esas técnicas.

Para distribuir su juego a una audiencia más amplia y no técnica, necesitarás empaquetar tu juego en un formato sencillo y amigable con las plataformas elegidas, por ejemplo en Windows en un archivo **EXE**.

En ésta sección se cubre el tema referente a empaquetar el juego y construir un archivo ejecutable en un formato que permita a cualquier usuario instalarlo y jugarlo.

8.0.1. Creando paquetes en Windows

Instalar un juego en Windows generalmente implica hacer doble clic en un archivo **EXE**, y éste inicia la instalación en forma automática.

El instalador suele tener la forma de un asistente con varias páginas que muestran la licencia, acuerdo y preguntar al usuario dónde copiar los archivos del juego y qué iconos instalar; y finaliza con un botón en la página final para iniciar la copiar de archivos y crear iconos.

Hay dos pasos necesarios para crear un instalador fácil de usar para su juego PyGame en la Plataforma Window:

Primero debe convertir su archivo principal de Python en un ejecutable, que se ejecute sin necesidad de instalar Python.

Segundo, debe utilizar el software de instalación para crear un único archivo EXE que contenga sus archivos del juego.

Con un instalador EXE, hace que su juego sea accesible para una audiencia más amplia, y esencial si su juego está destinado a ser comercial.

Puede omitir el segundo paso si va dirigido a un público es lo suficientemente técnico como para descomprima un archivo ZIP y haga doble clic en un archivo EXE.

8.0.2. Usando cx_Freeze

Para convertir un archivo Python en un archivo ejecutable, puede usar el comando `cx_Freeze`, que es en sí mismo un módulo de Python `cx_Freeze`, no forma parte de la biblioteca estándar de Python, pero se puede instalar fácilmente con el siguiente comando en `cmd.exe`/`bash`/`terminal`:¹

Características importantes de cx_Freeze:

- Crea ejecutables independientes a partir de scripts de Python con el mismo rendimiento.
- Admite versiones de Python de 3.8 a 3.12.
- Es multiplataforma y debería funcionar en cualquier plataforma en la que funcione Python.
- Utiliza una licencia derivada de la Fundación de Software de Python.

¹`cmd.exe`, `bash` y la terminal son términos relacionados con la línea de comandos y la interacción con sistemas operativos: El `cmd.exe` es el ejecutable del Símbolo del Sistema (Command Prompt) de Windows. El `bash` es una shell (interfaz de línea de comandos) popular en sistemas operativos como Linux y macOS. La terminal es un término genérico que se refiere a una aplicación de línea de comandos que se utiliza para interactuar con un sistema operativo. En Windows, el Terminal se conoce también como `cmd` o Símbolo del Sistema. En Linux y macOS, la terminal es una aplicación que se utiliza para ejecutar comandos y scripts en un entorno de texto.

- Tiene una documentación disponible en el repositorio oficial.
- Tiene un canal de discusión para ayuda y soporte.

La última versión estable de `cx_Freeze` es la 7.1.0, lanzada el 26 de mayo de 2024. Esta versión incluye nuevas características como la opción `-zip-filename` en `build_exe`, soporte para `pyproject.toml`, creación de formatos `AppImage` y `DEB` para Linux, mejoras en `bdist_mac`, nuevos `hooks` y soporte para Python 3.12. Muy importante revisar la documentación oficial para obtener más información sobre la instalación y actualizaciones.

Instalación en un Entorno Virtual

Activar el Entorno Virtual: Asegúrate de estar en el entorno virtual donde deseas instalar `cx_Freeze`. Instalar `cx_Freeze` utilizando `pip`:

```
pip install --upgrade cx_Freeze
```

Instalación en un Entorno no Virtual

Instalar `cx_Freeze` utilizando `pip`:

```
python -m pip install --upgrade cx_Freeze}
```

Instalación utilizando Pipenv

Instalar o actualizar `cx_Freeze` utilizando `pipenv`:

```
pipenv install cx_Freeze
```

Antes de crear un ejecutable para su código Python, debe escribir un archivo `setup.py` por convenio, y contiene toda la información de su proyecto y ejecuta `cx_Freeze`.

En mi caso instalé el módulo con el siguiente comando:

```
python -m pip install --upgrade cx_Freeze}
```

Ahora creamos el ejecutable, que por convenio debe llamarse `setup.py`. Primero importamos de `cx_Freeze`, `setup`, y `Executable`, luego especificamos qué vamos a crear un ejecutable;

```
setup.py
```

```

import sys
from cx_Freeze import setup, Executable

ejecutables = [Executable("Invasión_Alienígena.py")]

setup(
    name="Invasión Alienígena",
    version="0.1",
    description="Un Juego",
    options={"build_exe": {"packages": ["pygame"],
                              "include_files": [ "configurar.py",
                                                  "stats_juego.py", "puntuacion.py",
                                                  "nave.py",
                                                  "boton.py", "balas.py", "alien.py",
                                                  "pygame.jpg", "alienigena.bmp",
                                                  "nave.bmp"]}},
    executables=[Executable("Invasión_Alienígena.py", base="Win32GUI")]
)

```

Y configuramos la compilación `setup( )`, con los parámetros de nombre, opciones y ejecutables. Dentro de las `options` (un diccionario) tiene una clave llamada `build_exe` que se refiere a un sub-diccionario que contiene opciones específicas para el proceso de compilación, como `include_files` la clave que se refiere a los módulos o librería de objetos del juego que construimos, y los incluimos como una lista.

Muy importante el parámetro `ejecutables` debe ser una lista de objetos `Executable`, y la base `base= "Win32GUI"`², y todos los archivos en la misma carpeta o directorio. Aquí puede colocar música, imágenes y cualquier otro archivo, como los módulos `py` que necesita su programa.

Bien, una vez creado el archivo `script setup.py`, debe ser ejecutado en `cmd.exe/bash/terminal` con el siguiente comando:

```
python setup.py build
```

Si recibe el error, como que Python no es un comando reconocido, puede hacer lo siguiente:

```
C:/Python34/python setup.py build
```

Esta línea de código busca y copia todos los archivos utilizados por el `script` en una carpeta llamada `build`. La construcción interior es otra carpeta con información de la máquina, y luego allí encontrará el archivo

²En el contexto de `cx_Freeze` el parámetro `Win32GUI` se refiere al subsystem de Windows que se utiliza para crear aplicaciones de escritorio que se ejecutan en modo de ventana (GUI). Este subsystem es compatible con ambas arquitecturas, 32 bits y 64 bits.

`Invasión_Alienígena.exe`, que inicia la simulación de la máquina en estado del Juego, así como otros archivos necesarios para que se ejecute sin instalar Python.

Si comprimes esta carpeta, puedes compartirla con cualquier persona, tenga o no Python o los distintos módulos instalados y pueden ejecutar el juego, pero para darle un toque profesional también debes crear un instalador.

8.0.2.1. Construyendo el instalador

Para un estilo un poco más profesional, puedes crear un instalador, que proporcionará un único instalador para facilitar el uso del juego de manera muy sencilla. Sin necesidad de lidiar con la descompresión con algún extractor de archivos, simplemente haciendo doble clic en la aplicación inicia el proceso de instalación. Para ello, se mantiene todo igual excepto cuando vas a construir el archivo, debe usar el siguiente código:

```
python setup.py bdist_msi
```

Esto crea un directorio `dist`; y dentro está el instalador. De esta forma puedes distribuir sólo este instalador a tu amigos que luego podrán instalarlo y jugar.

Tenga en cuenta que si tiene una versión de Windows de 32 bits, esto supone un instalador de 32 bits para el juego. Windows de 32 o 64 bits puede instalarlo y ejecutarlo. Si está operando con un sistema de 64 bits versión de Windows, su distribución será de 64 bits y no podrá ejecutarse en máquinas de 32 bits.

Puede parecer bastante simple, pero se sabe que construir un ejecutable causa muchos dolores de cabeza. La documentación para obtener más información sobre cómo funciona el módulo `cx_Freeze`, la encuentra en <https://cx-freeze.readthedocs.io/en/stable/>. Un conjunto de módulos y funciones para crear ejecutables compatibles con varias plataformas, incluyendo Windows, Mac OS X y Linux. La documentación también cubre cómo utilizar `cx_Freeze` con otras herramientas y tecnologías, como PyInstaller, Setuptools y Virtualenv.

En general, la documentación de `cx_Freeze` es una guía detallada sobre cómo utilizar el herramienta para congelar scripts de Python en ejecutables y es un recurso valioso para cualquier persona que busque crear ejecutables de Python utilizando `cx_Freeze`.

La documentación proporciona información detallada sobre cómo utilizar `cx_Freeze`, incluyendo:

Iniciación: Esta sección proporciona una visión general de cómo utilizar `cx_Freeze`, incluyendo cómo instalarlo y cómo crear un ejecutable básico.

Uso básico: Esta sección explica cómo utilizar `cx_Freeze` para congelar un script de Python en un ejecutable.

Uso avanzado: Esta sección cubre cómo utilizar `cx_Freeze` para congelar un script de Python en un ejecutable con opciones avanzadas, como personalizar el icono del ejecutable y crear un archivo `setup.exe`.

Solución de problemas: Esta sección proporciona información sobre cómo solucionar problemas comunes con `cx_Freeze`, como errores y problemas de compatibilidad.

Preguntas frecuentes: Esta sección responde a preguntas frecuentes sobre `cx_Freeze`.

La siguiente tabla muestra una lista de algunos métodos y funciones de `cx_Freeze` que podrían ayudarte para crear ejecutables en Windows:

Table 8.0.2.1. Lista de métodos del módulo `cx_Freeze`:

	Descripción
<code>pip install cx_Freeze</code>	Comando para instalar <code>cx_Freeze</code> usando <code>pip</code> . Requiere tener Python agregado al PATH durante la instalación
<code>setup()</code>	Función principal para configurar el proceso de empaquetado. Recibe varios argumentos como <code>name</code> , <code>version</code> , <code>description</code> , <code>author</code> , <code>url</code> , etc
<code>Executable()</code>	Clase para definir un ejecutable a partir de un script Python. Recibe argumentos como <code>script</code> , <code>base</code> , <code>icon</code> , etc.
<code>build_exe</code>	Comando para construir el ejecutable a partir de la configuración definida en <code>setup.py</code> . Define el directorio de salida para el ejecutable.
<code>include_files</code>	Permite incluir archivos adicionales en el ejecutable, como imágenes o archivos de configuración
<code>include_msvcr</code>	Incluye la biblioteca de runtime de Microsoft Visual C++ (MSVCR) en el ejecutable
<code>msi_name</code>	Define el nombre del instalador MSI
<code>msi_version</code>	Define la versión del instalador MSI
<code>msi_description</code>	Define la descripción del instalador MSI
<code>msi_author</code>	Define el autor del instalador MSI
<code>msi_url</code>	Define la URL del autor del instalador MSI

Estos métodos y funciones permiten personalizar el proceso de empaquetado y crear un instalador MSI ("Microsoft Software Installer") más avanzado.

Como una técnica alternativa para generar un ejecutable de Python se tiene a `PyInstaller`, que se puede usar como:

```
pyinstaller --windowed --onefile --icon= nombre.ico script.py
```

`-windowed` oculta la consola, `-onefile` crea un archivo único, `-icon` agrega un ícono al ejecutable.

Ventajas de `cx_Freeze`

Facilidad de uso: `cx_Freeze` es fácil de instalar y utilizar, especialmente para usuarios con experiencia en Python. Compatibilidad multiplataforma: Puede generar ejecutables para Windows, Linux y Mac.

Flexibilidad: Permite personalizar la configuración de los ejecutables, como la inclusión de iconos y la ocultación de la consola. Soporte para instaladores: Permite crear instaladores de Windows (MSI) para facilitar la distribución de aplicaciones.

Ventajas `PyInstaller`

Soporte amplio: `PyInstaller` es ampliamente utilizado y tiene una gran comunidad que lo mantiene actualizado y mejorado.

Flexibilidad: Permite generar ejecutables que incluyan todas las dependencias necesarias para su funcionamiento.

Opción de crear un único archivo ejecutable: Permite crear un archivo ejecutable único que incluya todas las dependencias necesarias. Soporte para iconos: Permite agregar iconos a los ejecutables.

Desventajas de `cx_Freeze`:

No tan ampliamente utilizado como `PyInstaller`: Esto puede significar que no hay tanto apoyo y actualizaciones como `PyInstaller`. Puede requerir configuración adicional: Para generar ejecutables que incluyan todas las dependencias necesarias, puede ser necesario realizar configuración adicional.

`PyInstaller` puede generar ejecutables grandes, si se incluyen todas las dependencias necesarias.

Puede requerir configuración adicional: Para generar ejecutables que incluyan todas las dependencias necesarias, puede ser necesario realizar configuración adicional.

Ambas herramientas son útiles para empaquetar scripts de Python en ejecutables, pero `PyInstaller` es más ampliamente utilizado y tiene una mayor comunidad detrás.